Chapter 26 - Stage 7 of SKDL -- Executable Knowledge (Part 3): Data Integrity & Comprehensive Architecture

Completing the Executable Ontology

Chapter 24 built the boundary layer of an executable ontology. Chapter 25 built the value layer. This chapter builds the last layer -- **data integrity** -- and then steps back to view the whole of Stage 7 as a single architecture.

By the end of this chapter, we will have consolidated **five execution primitives** into a single anatomy, examined executability from four complementary perspectives (mathematical, historical, industrial, and EKA), and closed the design-pattern family that began in Chapter 22.

This is the final chapter of Stage 7. It completes the SKDL lifecycle and prepares the synthesis that follows in Chapter 27

- 26.1 Chapter Introduction
- 26.2 Learning Objectives
- 26.3 Exercise 26 -- Cardinality Restrictions and Data Completeness
  - 26.3.1 The Engineering Objectives
  - 26.3.2 Implementation: Creating the Cardinality Restriction
  - 26.3.3 Why the Reasoner Did Not Classify the Three Pizzas
  - 26.3.4 How to Make the Cardinality Fire
    - Attempt 1 -- Add a closure axiom (the pattern from Chapter 24)
    - Attempt 2 -- Correct the first Stage 1 modeling error, then re-test
      - Attempt 2.1 -- Diagnose the remaining two pizzas
      - Attempt 2.2 -- Apply the remaining corrections and re-test
      - Attempt 2.3 -- Investigate the remaining two classes
      - Attempt 2.4 -- Investigate the apparent non-classification
      - The lesson: verify inference in the right view
    - Approach 2 -- Assert the cardinality directly (an alternative)
  - 26.3.5 Understanding Protégé UI Behavior: Why the Description Panel Can Mislead
    - 26.3.5.1 The Mechanics of Conversion: Why "SubClass Of" Becomes Empty
    - 26.3.5.2 The Inferred-Ancestor Behavior: Why a Subclass May Not Show Its Inferred Parent
    - 26.3.5.3 The Underlying Principle: Local View vs. Global View
    - 26.3.5.4 A Verification Recipe
    - 26.3.5.5 Applying the Lesson
- 26.4 The Anatomy of an Executable Ontology
  - 26.4.1 The Five Primitives
  - 26.4.2 The Three Layers
  - 26.4.3 The Anatomy in the EKA Framework
  - 26.4.4 A Minimal Worked Example
  - 26.4.5 What the Anatomy Enables
  - 26.4.6 From Anatomy to Perspective
- 26.5 Mathematical Perspective
  - 26.5.1 OWL as Set Theory
  - 26.5.2 Closure as Set Restriction
  - 26.5.3 Enumeration as Finite Extension
  - 26.5.4 `hasValue` as Singleton Binding
  - 26.5.5 Cardinality as Bounded Extension
  - 26.5.6 The Mathematical Arc of Stage 7
  - 26.5.7 Distinctness -- The Underlying Condition
  - 26.5.8 What the Mathematics Teaches
  - 26.5.9 The Mathematical Arc Across Stage 7
- 26.6 Historical Perspective
  - 26.6.1 The AI Lineage -- From Production Rules to Rule Engines
  - 26.6.2 The Industrial Lineage -- From Taylor to BPMN to Model-Driven Engineering

    - 26.6.3 The Convergence at Stage 7
    - 26.6.4 What the Historical Perspective Teaches
    - 26.6.5 From Lineage to Industrial Practice
- 26.7 Industrial Perspective
    - 26.7.1 The Manufacturing Leg: ISA-95 and the Reasoning-to-Execution Pipeline
    - 26.7.2 The Compliance Leg: Closed-World Regulation vs. Open-World Reasoning
    - 26.7.3 The Process Graph Leg: BPMN, DMN, and the Separation of Concerns
    - 26.7.4 The Three Legs Together
- 26.8 EKA Perspective -- Stage 7 Establishes Θ + Φ
    - 26.8.1 The Two Halves of the Executable Knowledge Architecture
    - 26.8.2 The Five Primitives in the EKA Framework
    - 26.8.3 The Anatomy Revisited: EKA in Action
    - 26.8.4 The EKA Perspective Across the Book
    - The EKA Architecture in Industrial Practice
    - 26.8.6 What the EKA Perspective Teaches
    - 26.8.7 Looking Ahead -- From EKA to SKDL Synthesis
- 26.9 Engineering Guidelines for Executable Knowledge
- 26.10 Key Concepts
- 26.11 Chapter Summary
- 26.12 Looking Ahead -- Toward SKDL Synthesis
- Reference

26.1 Chapter Introduction

We have now built two layers of an executable ontology.

The boundary layer (Chapter 24). Universal restrictions told the reasoner what was *allowed*; closure axioms closed the world around those boundaries. The ontology could classify `MargheritaPizza` and `SohoPizza` as `VegetarianPizza`, deterministically and without ambiguity.

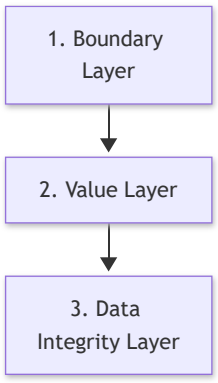
The value layer (Chapter 25). Enumerated classes closed the *set of possible values* for a property (`Spiciness = {Hot, Medium, Mild}`). `hasValue` restrictions bound a property to a *specific value* from that set (`JalapenoPepperTopping hasSpiciness value Hot`). The ontology could fire a trigger on a specific value rather than a category.

But a boundary and a value together are still not enough to execute safely.

Consider what happens when the ontology classifies a pizza as `SpicyPizza` because it has at least one `Hot` topping. What if the pizza has *no* topping? What if it has *two* `hasBase` assertions? What if the property that the rule depends on is simply *missing*?

The reasoner under the Open World Assumption will not complain. It will simply *not* fire the rule, and the execution system will silently do nothing. That is the quiet failure mode of an ontology that has not been engineered for **integrity**.

The third layer of executability solves this. It is the **data integrity layer** -- the layer that guarantees the *presence and cardinality* of the data that triggers depend on.



This chapter introduces the last execution primitive -- the **cardinality restriction** (**Exercise 26**) -- and then consolidates all five primitives into a single architecture.

Key Insight:

> "A boundary tells the system what is permitted. A value tells the system what to act on. Cardinality tells the system *when it has enough to act*. Only all three together make execution safe."

26.2 Learning Objectives

After completing this chapter, you should be able to achieve the following learning outcomes:

Knowledge:

- Explain why data integrity is the third layer of an executable ontology
- Distinguish the three cardinality restrictions: `min`, `max`, and `exactly`
- Understand how cardinality differs from the closure and value layers
- Explain how the five execution primitives of Stage 7 form a single anatomy
- Trace the two historical lineages of executability: AI reasoning and industrial process
- Apply executable knowledge to industrial scenarios (manufacturing, compliance, process)

Practical Skills:

- Create cardinality restrictions (`min` , `max` , `exactly`) in Protégé
- Use `hasTopping min 3 PizzaTopping` and other cardinality patterns
- Convert a class using cardinality to a defined class
- Test cardinality-based classification by synchronizing the reasoner
- Verify that reasoner-inferred classifications match the intended integrity constraints

Mathematical Perspective:

- Understand cardinality as the bounded-extension restriction on a property
- Explain the relationship between cardinality, functional properties, and `hasValue`
- Understand how cardinality restrictions close the "number of values" dimension

Engineering Perspective:

- Understand data integrity as the *last* layer, not the first
- Map the five Stage 7 primitives to the Θ/Φ architecture
- Apply executable ontologies in industrial contexts

26.3 Exercise 26 -- Cardinality Restrictions and Data Completeness

Having established closure in **Exercise 23** and value binding in **Exercise 24 and 25**, we now turn to the final primitive of Stage 7: the **cardinality restriction**.

This exercise follows the tutorial's **Exercise 26**, which introduces `InterestingPizza` -- a pizza with three or more toppings.

26.3.1 The Engineering Objectives

Exercise 26 introduces the **Cardinality Restriction** -- a restriction on the *number* of values a property may have.

OWL provides three forms:

Form	Meaning	Syntax
Minimum	At least *n* values	`hasTopping min 3 PizzaTopping`
Maximum	At most *n* values	`hasTopping max 5 PizzaTopping`
Exactly	Exactly *n* values	`hasTopping exactly 2 PizzaTopping`

A cardinality restriction for an "interesting pizza", for example, would be:

```
Class: InterestingPizza
    EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
```

This means: "An `InterestingPizza` is a `Pizza` with at least three toppings."

The engineering objective of **Exercise 26** is to demonstrate how cardinality restrictions **guarantee data completeness** -- the third and final requirement for **execution**.

Why does execution need cardinality?

Consider an execution rule that fires only when a pizza has enough toppings to qualify as "interesting". Without a cardinality restriction, the ontology cannot determine *how many* toppings the pizza has -- it can only say that *some* exist. A rule that depends on count cannot fire reliably.

Cardinality closes the *number* dimension. Where the closure axiom closes the *set* of permitted toppings and `hasValue` closed the *specific value*, cardinality closes *how many* values the property may take.

Key Insight:

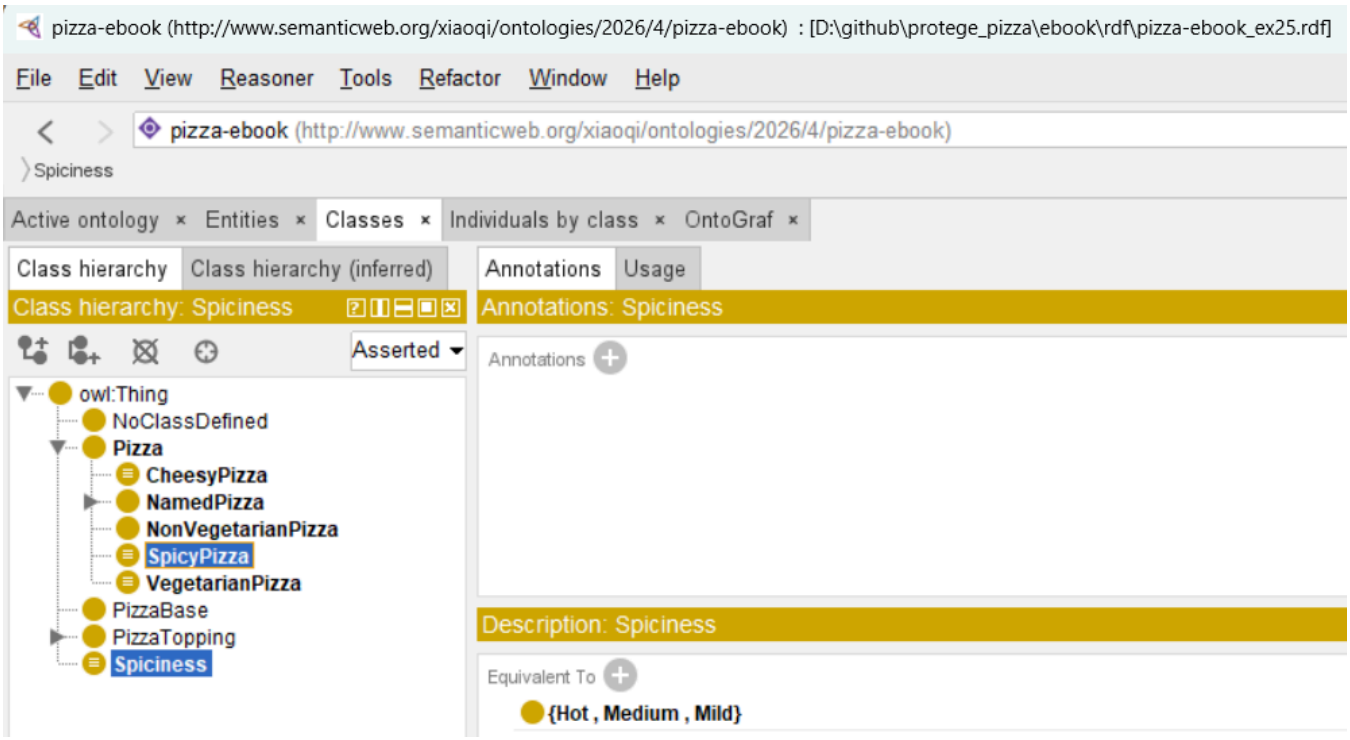
> "Cardinality is the last mile of executability. A rule that fires on 'at least three toppings' requires the ontology to know that count is enforced."

26.3.2 Implementation: Creating the Cardinality Restriction

Let's walk through the implementation step by step.

Step 1: Open the `Pizza` ontology

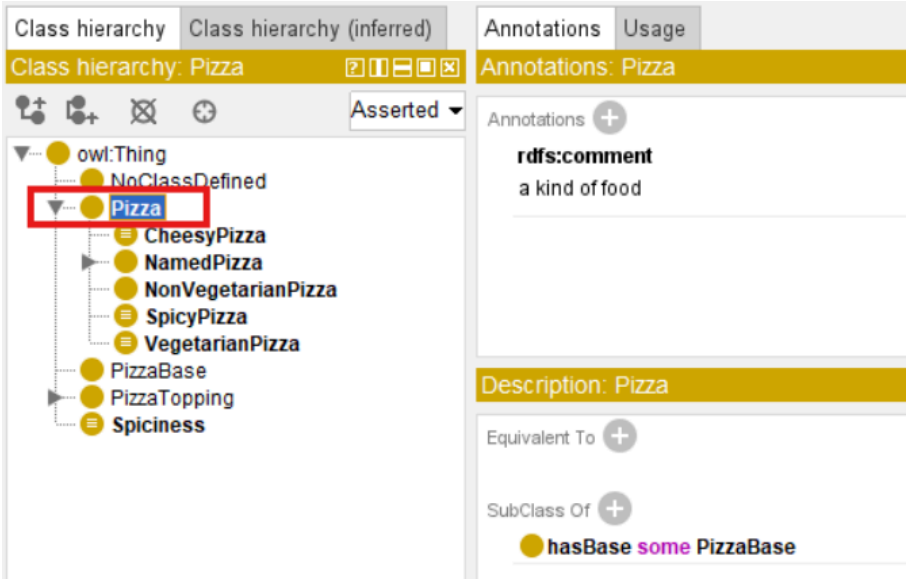
Open `pizza-ebook_ex25.rdf` in Protégé -- the file from the previous exercise, which now contains the enumerated class `Spiciness`, the `hasSpiciness` property, and the defined class `SpicyPizza`.



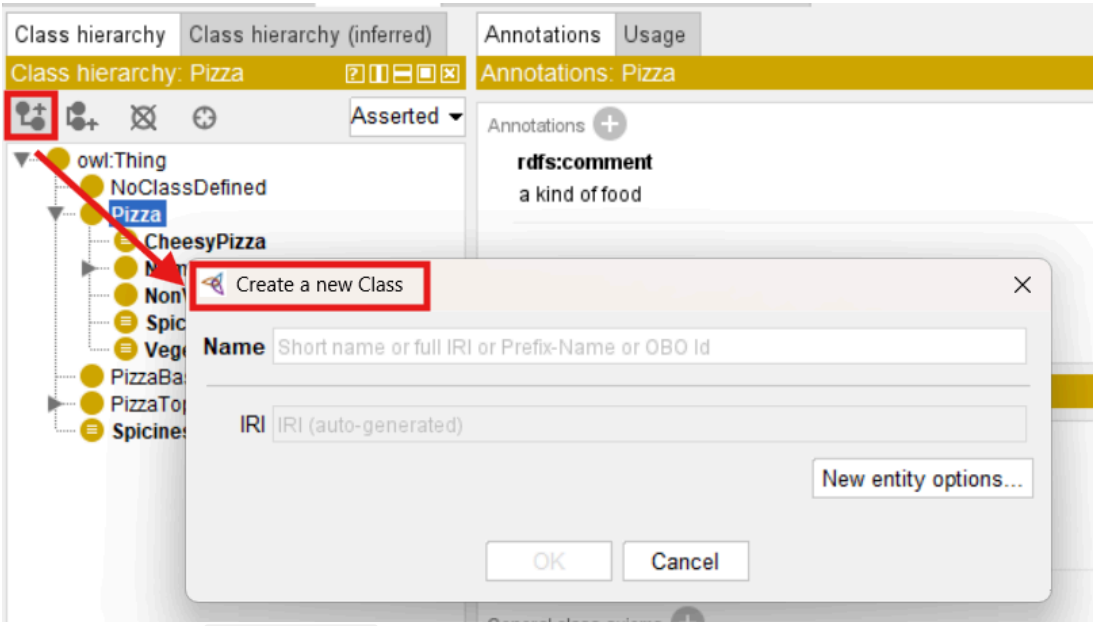
> **Note**
>
> The finished state of **Exercise 26** will be saved as `pizza-ebook_ex26.rdf`.

Step 2: Create the `InterestingPizza` class

1. Make sure `Pizza` is selected in the class hierarchy

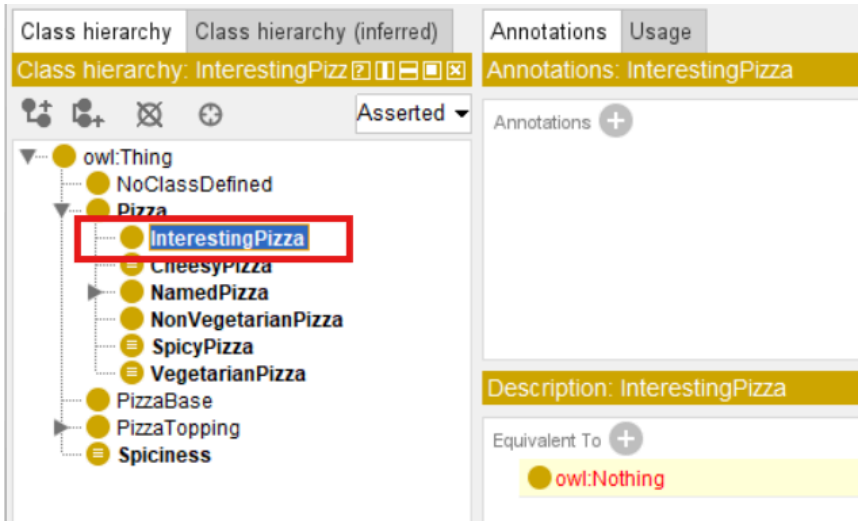


2. Click the **Add subclass** icon



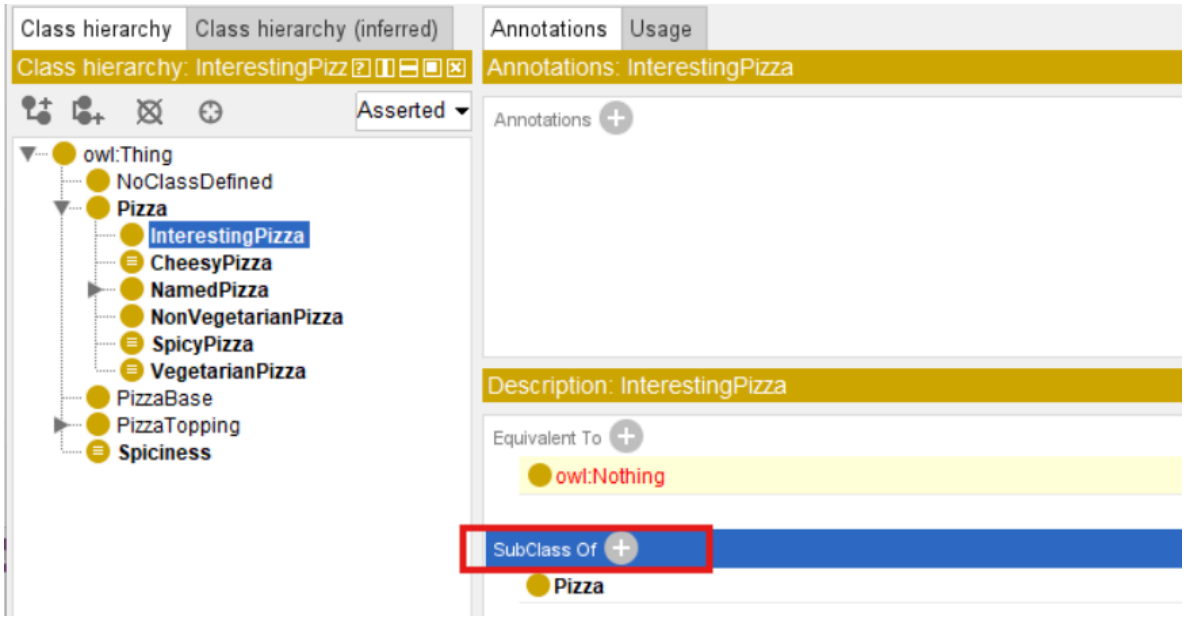
3. Name the new class `InterestingPizza`
4. 

5. Click `OK`

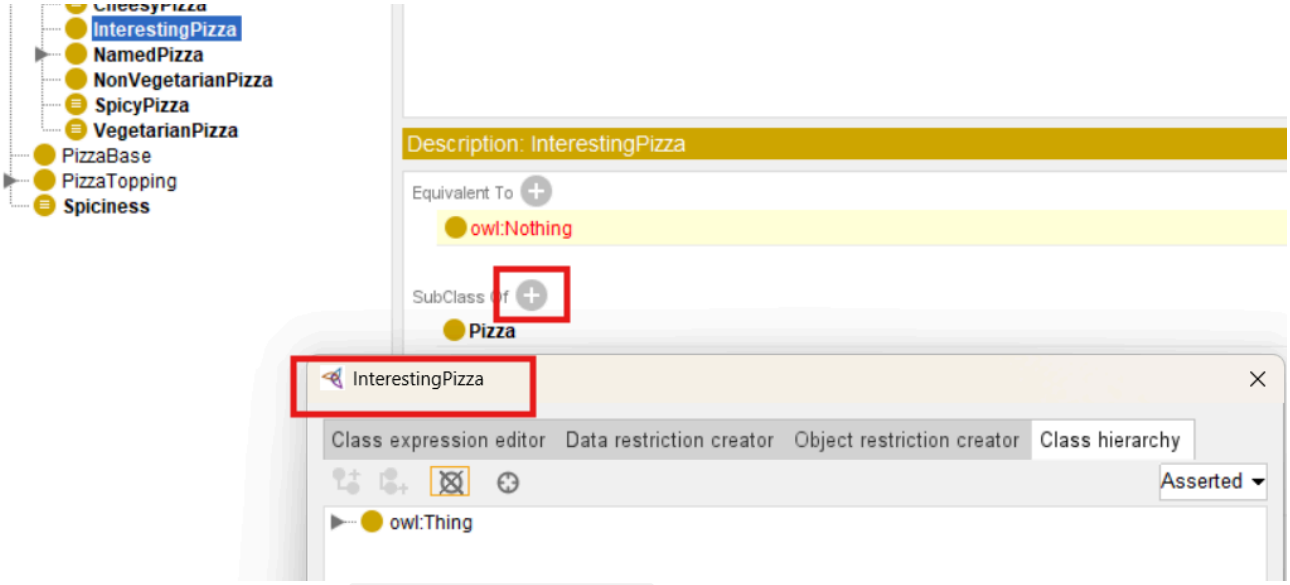


Step 3: Add the cardinality restriction

1. Select `InterestingPizza` (should already in focus after previous step)
2. In the Description view, locate the **SubClass Of** section



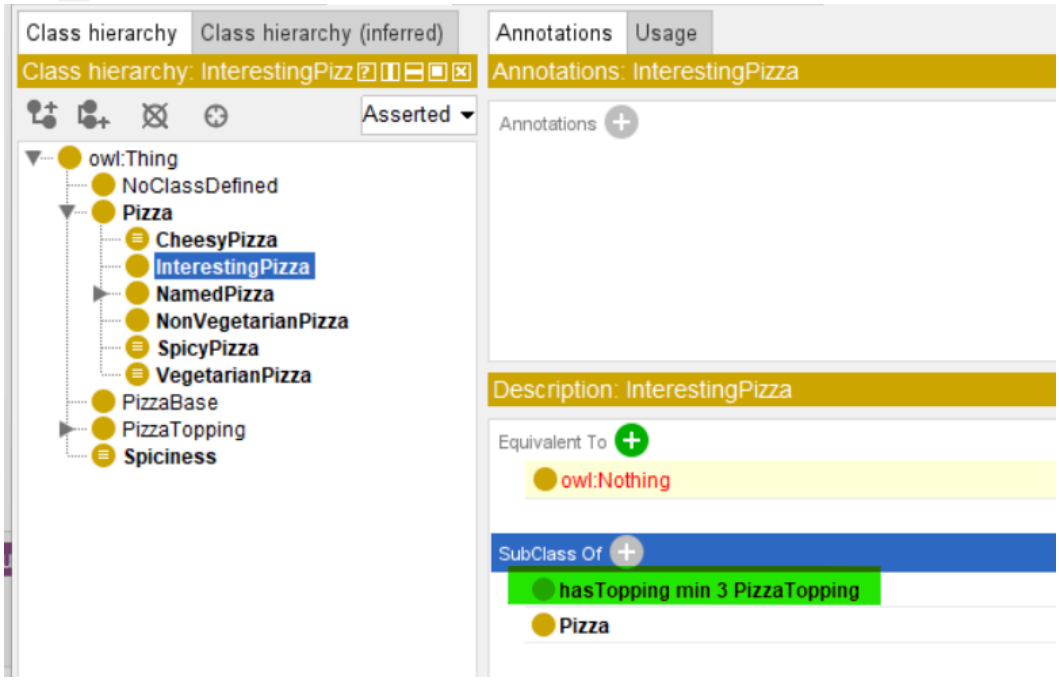
3. Click the Add icon (+)



4. In the Class Expression Editor, enter: `hasTopping min 3 PizzaTopping`



5. Click `OK`



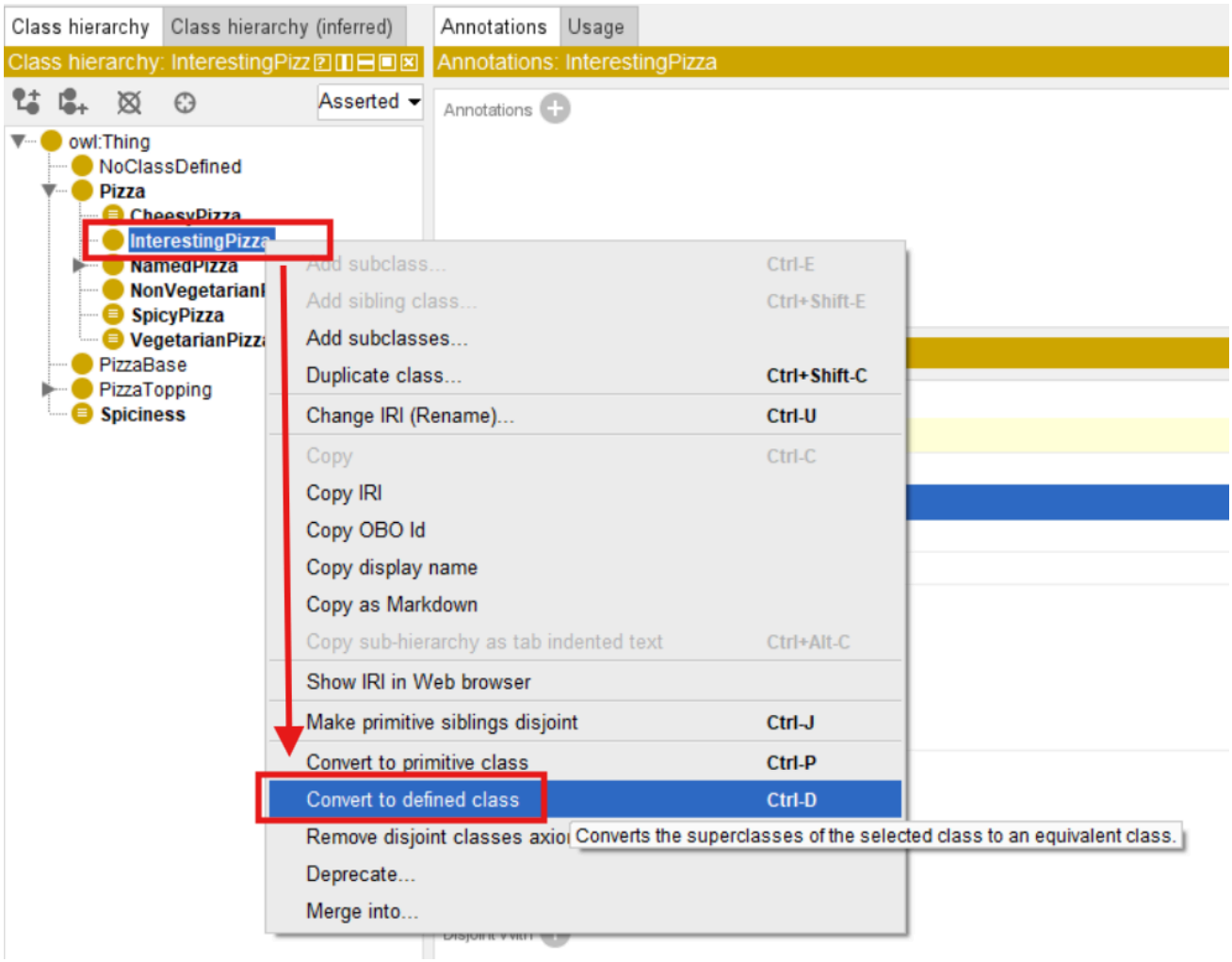
After this step, `InterestingPizza` has:

```
Class: InterestingPizza
    SubClassOf: Pizza
    SubClassOf: hasTopping min 3 PizzaTopping
```

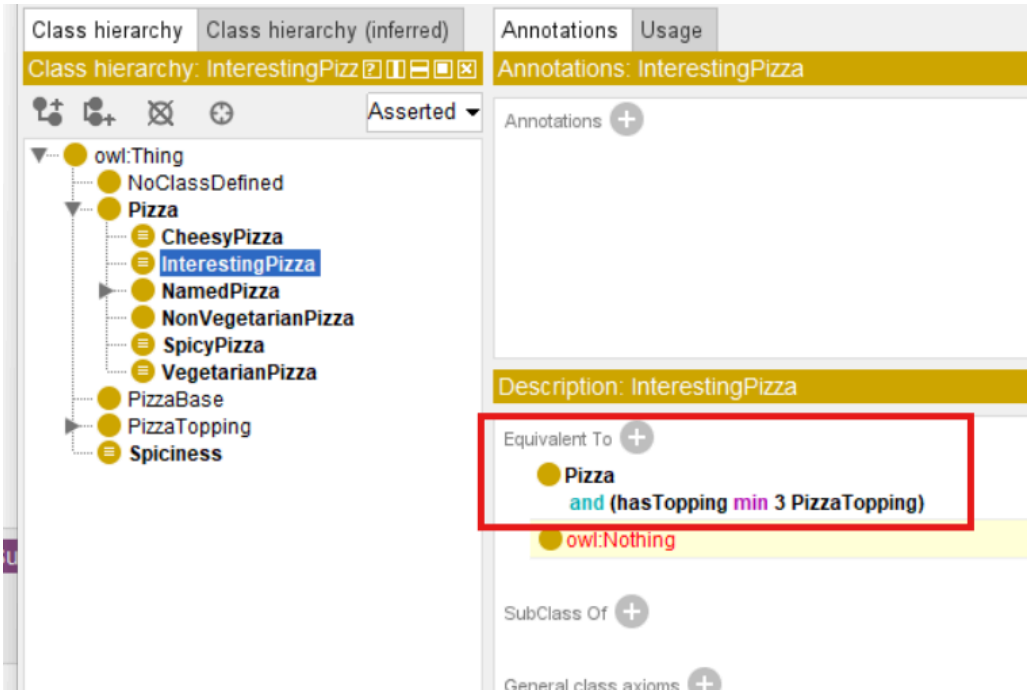
This says: "An `InterestingPizza` is a pizza and must have at least three toppings."

Step 4: Convert `InterestingPizza` to a defined class

1. Make sure `InterestingPizza` is still selected
2. Select `Edit > Convert to defined class`



Observe what happens in the Description view. The `SubClassOf` axiom for the cardinality restriction is moved to `EquivalentTo`:

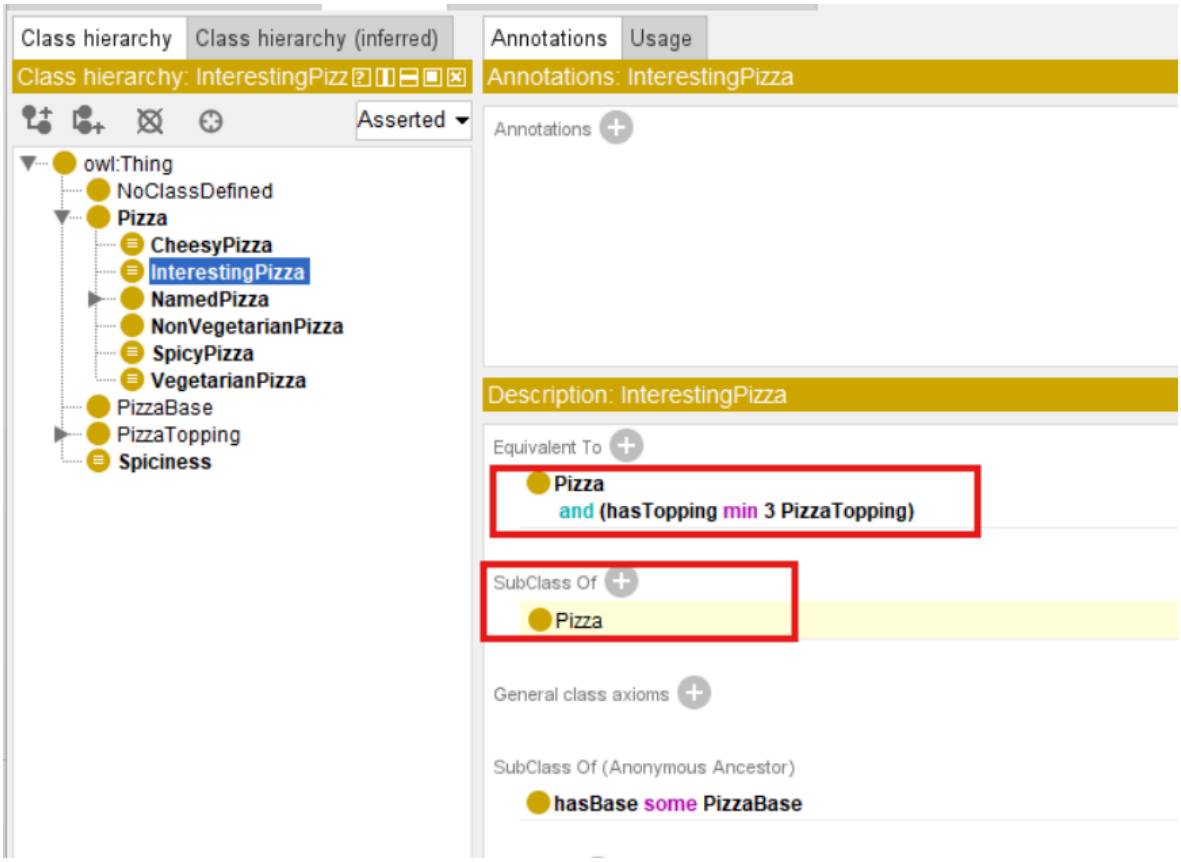


```
Class: InterestingPizza
    EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
    EquivalentTo: owl:Nothing
    SubClassOf: <Empty>
```

> **Note**
>
> You may notice the **SubClass Of** section is empty, to learn why, check later §26.3.5 for some clarification.

Step 5: Synchronize the reasoner

Select `Reasoner > Synchronize reasoner` (Ctrl+R)



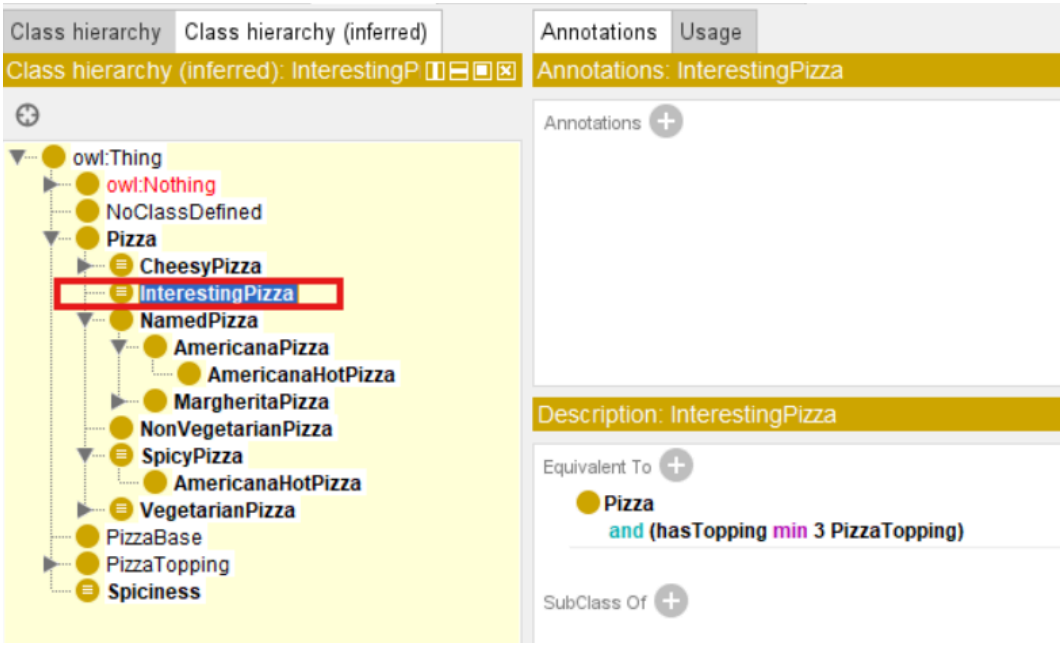
After reasoner synchronized, the **SubClass Of** section contains `Pizza` again.

```
Class: InterestingPizza
    EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
    SubClassOf: Pizza
```

Step 6: Observe the result

We would assume that the reasoner classifies pizzas with three or more toppings as subclasses of `InterestingPizza`. In the tutorial's data, these include `AmericanaHotPizza`, `AmericanaPizza`, and `SohoPizza`.

However, look at the **Class hierarchy (inferred)** tab:



You will see something surprising: **None of `AmericanaHotPizza`, `AmericanaPizza`, or `SohoPizza` are classified as subclasses of `InterestingPizza`.**

This is not as tutorial mentioned and unexpected. All three pizzas have three or more toppings assertions:

Pizza	Toppings
`AmericanaPizza`	Mozzarella, Pepperoni, Tomato (3)
`AmericanaHotPizza`	Mozzarella, Pepperoni, Tomato, Jalapeno (4)
`SohoPizza`	Mozzarella, Tomato, Olive, Parmesan (4)

And yet the reasoner does not classify any of them under `InterestingPizza`.

No worry! This is not a bug in Protégé. It is a subtle consequence of how cardinality restrictions interact with existential restrictions under the Open World Assumption. It deserves its own explanation -- see **§26.3.3**.

26.3.3 Why the Reasoner Did Not Classify the Three Pizzas

The behavior in above Step 6 is one of the most instructive moments in this chapter. It looks like a failure, but it is a faithful reflection of OWL's semantics -- and understanding it is essential to writing executable ontologies that work in production.

The apparent contradiction. Each of the three pizzas has three or four existential toppings restrictions, e.g. `AmericanaPizza`:

```
Class: AmericanaPizza
    SubClassOf: hasTopping some MozzarellaTopping
    SubClassOf: hasTopping some PepperoniTopping
    SubClassOf: hasTopping some TomatoTopping
```

A human reads this and concludes: "This pizza has three toppings." The `InterestingPizza` definition asks for `hasTopping min 3 PizzaTopping`. The count matches. So why doesn't the reasoner agree?

The OWL semantics. Existential restrictions do **not** commit to the number of distinct fillers. `hasTopping some X` means "there exists at least one value of `hasTopping` that is of type `X`." It does not say "there is exactly one" or "this is the only one of its kind." Under the Open World Assumption, the reasoner is forbidden from assuming the three existentials refer to three distinct individuals unless it can prove it from the axioms.

Even when the filler classes are declared disjoint -- as `MozzarellaTopping`, `PepperoniTopping`, and `TomatoTopping` are -- disjointness establishes that *if* three disjoint individuals exist, they are of different types. It does not establish that three individuals exist. The three

existential restrictions could, in the reasoner's view, all be satisfied by the same single individual if that individual happened to be a member of all three classes. (In practice, disjointness rules that out here, but the reasoner cannot bridge from "there exist some X, some Y, some Z" to "there exist at least three individuals" without additional axioms.)

The `min 3 PizzaTopping` restriction, therefore, has nothing to count. The reasoner sees three *existential restrictions*, not three *toppings*, and it cannot satisfy the min-3 cardinality.

Why this is important? This is the single most common source of confusion when engineers first write cardinality restriction in OWL. It looks like the ontology should be able to count; semantically, it cannot, without more information. The tutorial's expectation that these pizzas would be inferred as `InterestingPizza` reflects human intuition about the data, not the reasoner's semantics.

This is not a flaw in OWL. It is a *design choice*: OWL refuses to commit to the number of values a property has unless the ontology explicitly closes the world. The consequence for execution is direct: **if a trigger depends on a count, the ontology must be engineered so that the count is provable.**

Key Insight:

> "The reasoner counts what the ontology proves, **not** what the engineer intends. Existential restrictions prove existence, not count. Only closure or explicit cardinality makes the count provable."

26.3.4 How to Make the Cardinality Fire

The fix is not to remove the cardinality restriction. The fix is to give the reasoner enough information to evaluate it. But as we are about to see, the "obvious" fix does not work -- and reason it does not work is one of the most important lessons in this chapter.

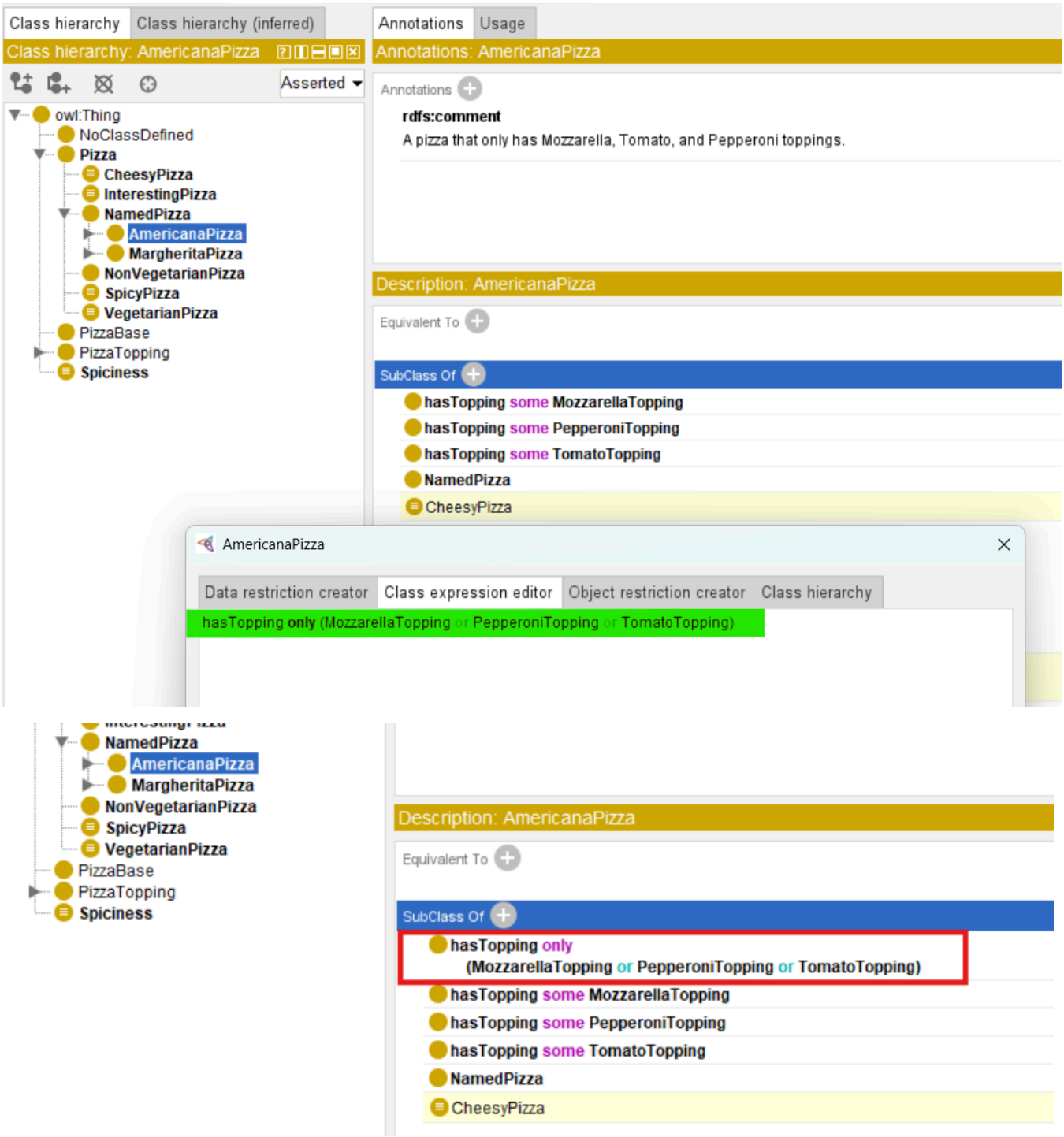
We will walk through the fix in stages. Each stage tries something, observes the result, and diagnoses why it did or did not work. This is what semantic engineering looks like in practice: **Not one big correct answer, but a disciplined sequence of hypotheses, tests, and corrections.**

Attempt 1 -- Add a closure axiom (the pattern from Chapter 24)

> ✎ **Keep `pizza-ebook_ex26.rdf` as one snapshot. From now on, the fix for Attempt 1 is saved in `pizza-ebook_ex26-approach1.rdf`**

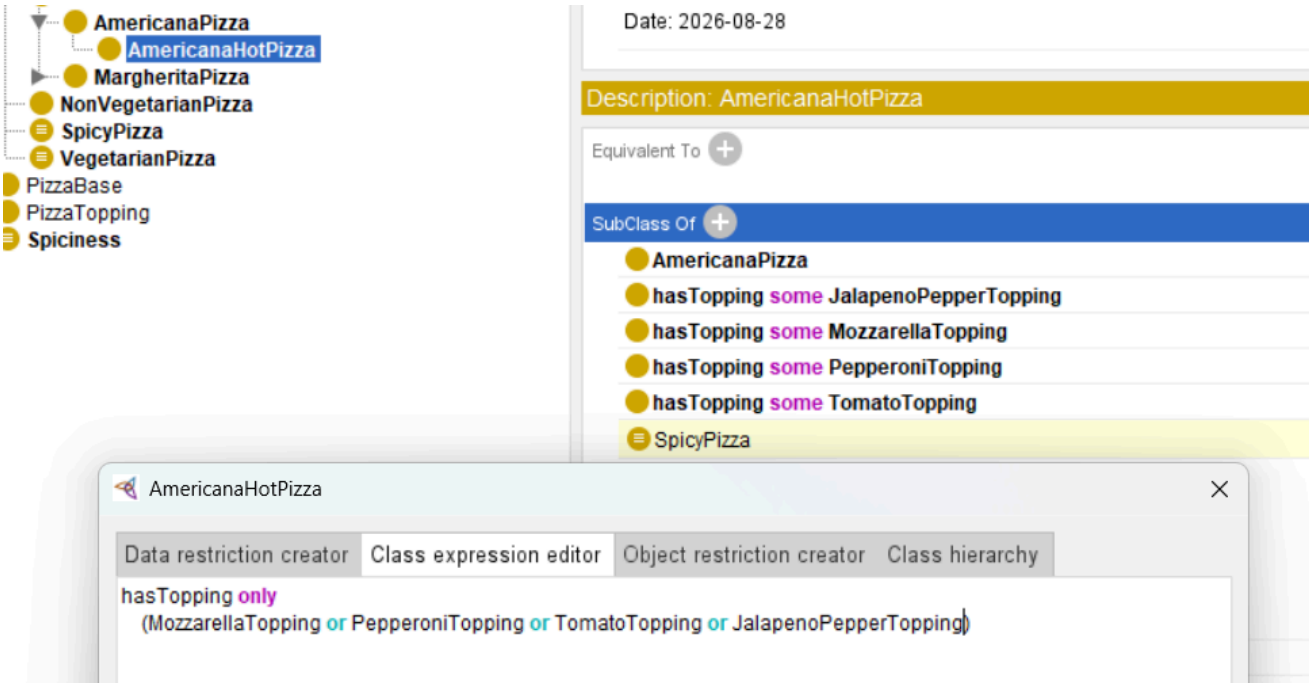
The natural first move is to apply the closure pattern from Chapter 24. If the reasoner cannot prove the count because it does not know the toppings are exhaustively the listed ones, then close the world for each pizza class using a universal restriction:

```
Class: AmericanaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping)
```

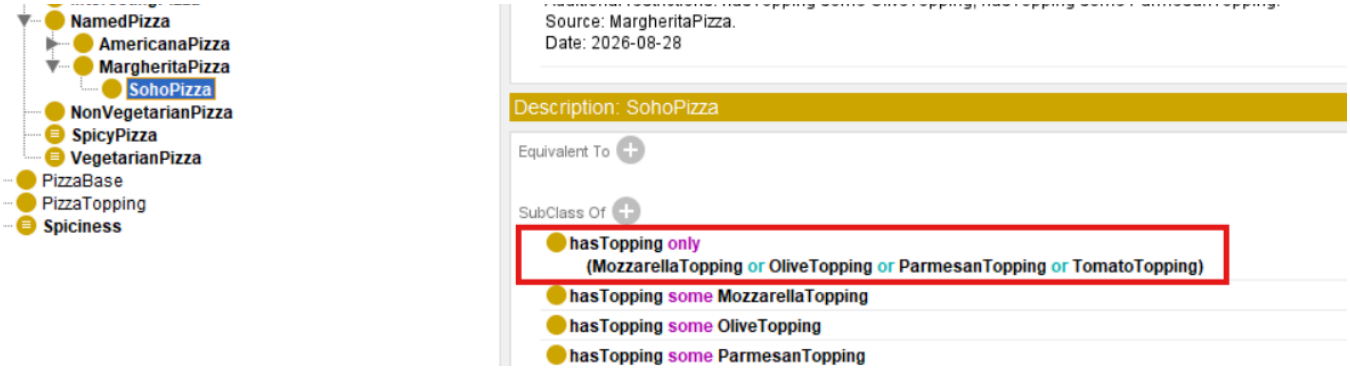


Apply the same closure to `AmericanaHotPizza` (Mozzarella, Pepperoni, Tomato, JalapenoPepper) and `SohoPizza` (Mozzarella, Tomato, Olive, Parmesan). This is exactly the pattern that worked in **Chapter 24** for `MargheritaPizza` and `SohoPizza`. It should work here too.

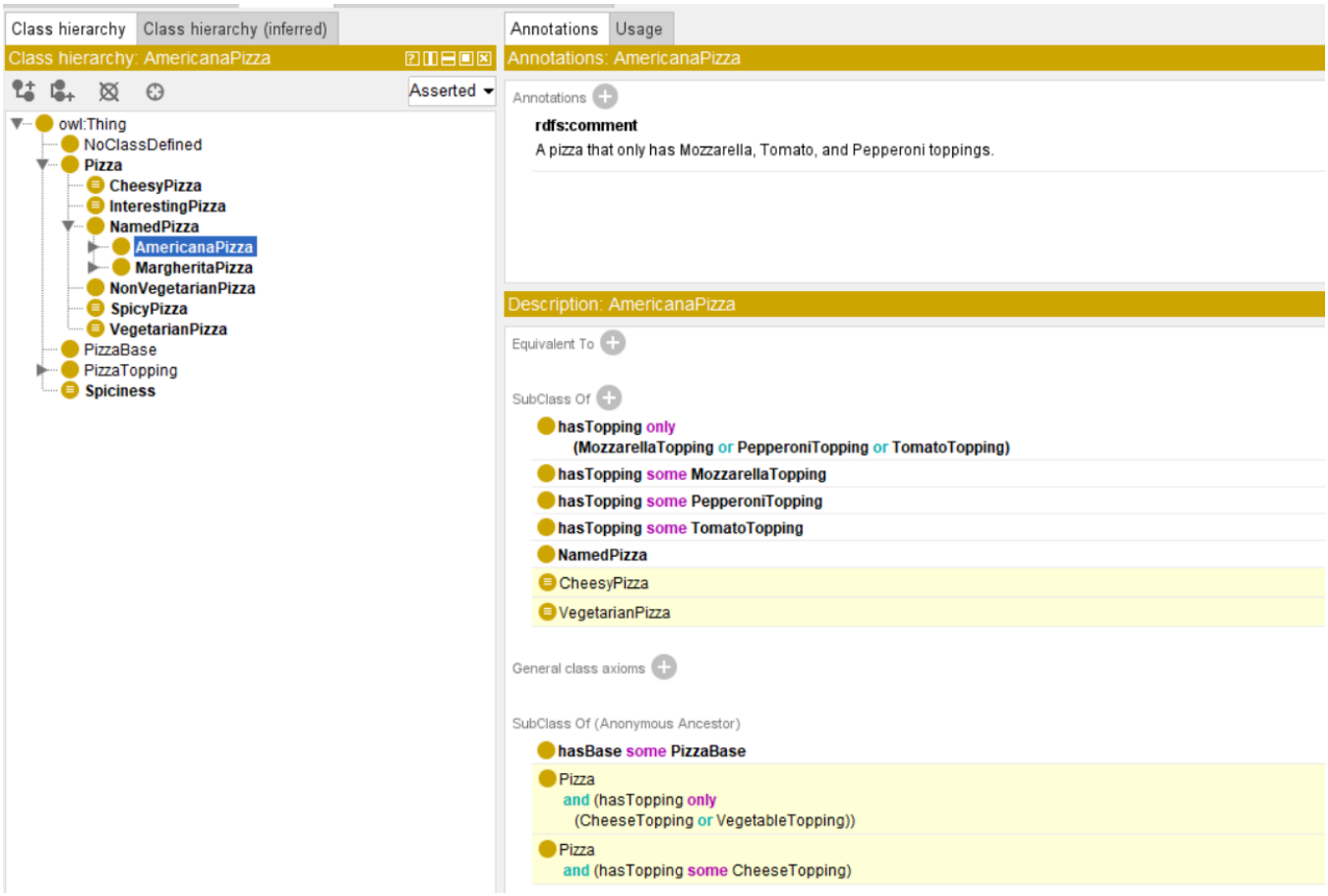
```
Class: AmericanaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping or JalapenoPepperTopping)
Class: SohoPizza
    SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping or OliveTopping or ParmesanTopping)
```



You may find `SohoPizza` has already had the needed universal restriction in the working ontology.



Synchronize the reasoner and observe. For `AmericanaPizza`, the result is **not** what we expect:



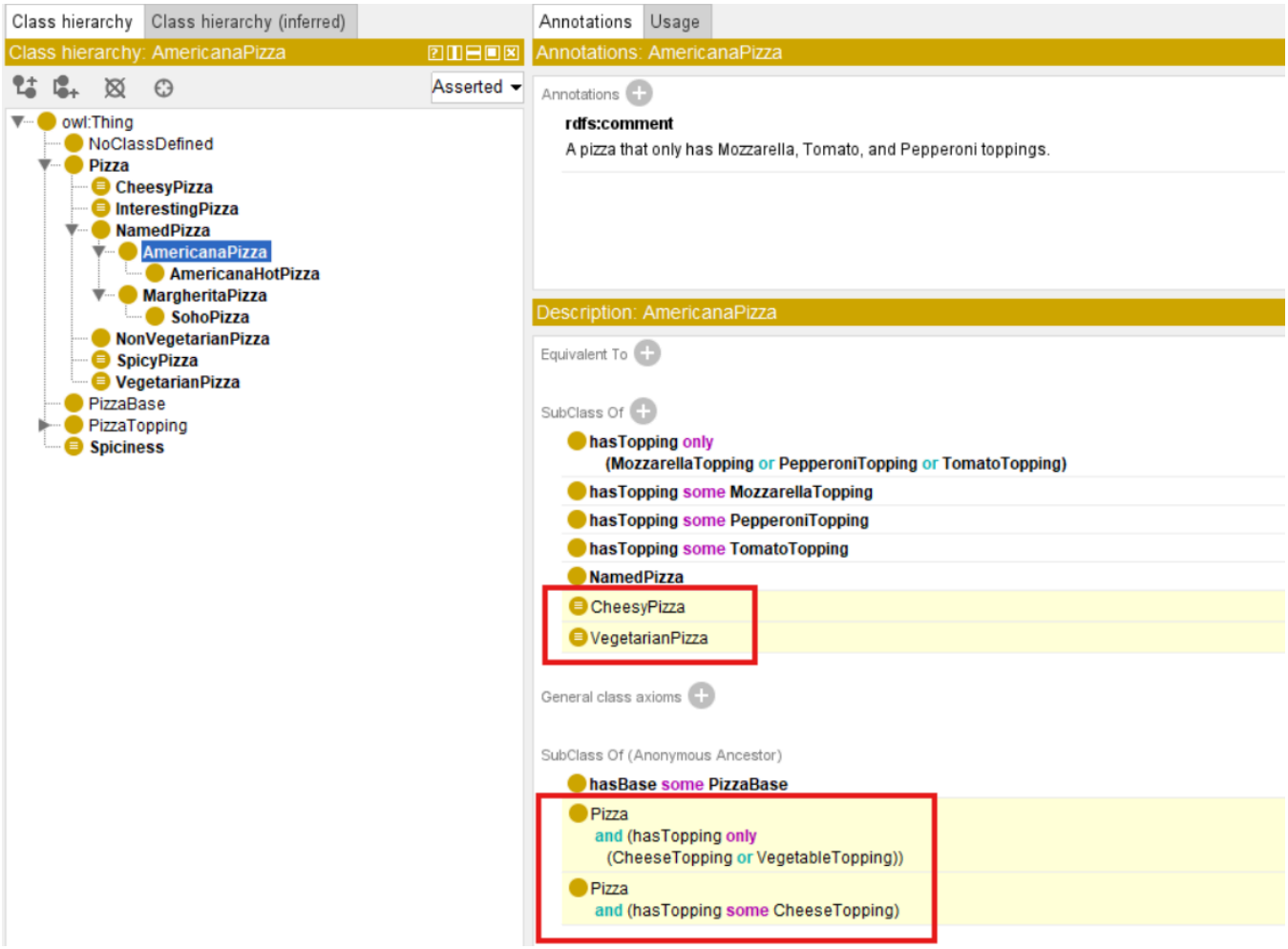
None of the three pizzas are classified as `InterestingPizza`. The closure axiom is in place -- we verified in the Protégé UI (the `hasTopping only (...)` restriction is visible in the `SubClass Of` section of `AmericanaPizza`) -- and yet the min-3 cardinality still does not fire.

Attempt 1 has failed. This is worth pausing on, because the failure is not a bug. It is a ***signal***.

Why Attempt 1 failed -- and what the failure reveals?

The closure axiom restricts the *range* of `hasTopping`: every topping must be in the listed union. This is necessary for the reasoner to bound the possible toppings. But for the min-3 cardinality to fire, the reasoner must still prove that **three distinct topping individuals exist**.

Even with the closure in place, the reasoner can prove at most **two** distinct toppings for `AmericanaPizza`, see from its `SubClass Of` section below:



Here is why.

The reasoner's ability to count distinct individuals depends on being able to prove that two values cannot be the same individual. It proves this from **disjointness axioms**. If two classes `A` and `B` are declared disjoint, then no individual can be both an `A` and a `B`, so any individual of type `A` is distinct from any individual of type `B`.

For `AmericanaPizza`'s three toppings -- `MozzarellaTopping`, `PepperoniTopping`, `TomatoTopping` -- the reasoner needs to know that all three are pairwise disjoint. It checks the ontology:

- `MozzarellaTopping` is a subclass of `CheeseTopping`.
- `PepperoniTopping` is declared (in the working ontology) as a subclass of `VegetableTopping`.
- `TomatoTopping` is a subclass of `VegetableTopping`.

The ontology's disjointness declarations establish that `CheeseTopping`, `MeatTopping`, and `VegetableTopping` are pairwise disjoint. So:

- `MozzarellaTopping` (Cheese) is provably distinct from `PepperoniTopping` (Vegetable). ✅
- `MozzarellaTopping` (Cheese) is provably distinct from `TomatoTopping` (Vegetable). ✅
- `PepperoniTopping` (Vegetable) and `TomatoTopping` (Vegetable) are **not** declared disjoint from each other. ❌

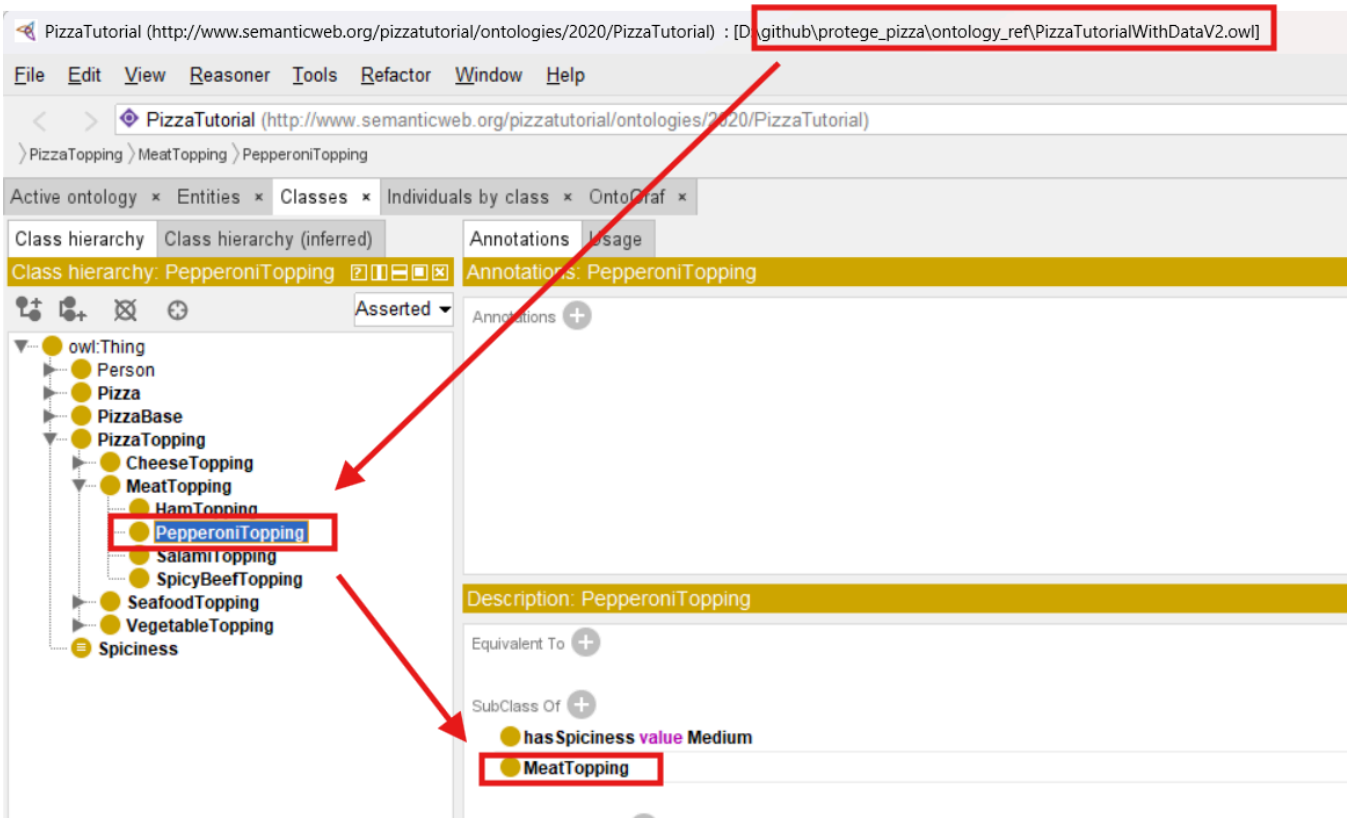
The reasoner can prove at most **two** distinct toppings for `AmericanaPizza` -- Mozzarella is distinct from the others, but Pepperoni and Tomato might be the same individual as far as the reasoner can tell. The min-3 cardinality cannot be satisfied.

But, wait, there is a deeper problem here. The reason `PepperoniTopping` and `TomatoTopping` are not disjoint is not an accident of the ontology's design. It is a **modeling error**.

Pepperoni is a meat product. It should be subclass of `MeatTopping`, not `VegetableTopping`. The working ontology got this wrong. This is the same kind of error the SKDL was designed to catch -- but it has slipped through to Stage 7.

Let's verify the correct taxonomy. Michael DeBellis' reference ontology (`PizzaTutorialWithDataV2.owl`) declares:

```
<owl:Class rdf:about="...#PepperoniTopping">
    <rdfs:subClassOf rdf:resource="...#MeatTopping"/>
    ...
</owl:Class>
```



The reference is unambiguous: `PepperoniTopping` is a subclass of `MeatTopping`. The working ontology has it under `VegetableTopping`. **This is a Stage 1 modeling error.**

Stage 7 cannot fix a Stage 1 error.

This is the moment where the SKDL's discipline shows its value.

The SKDL is a **lifecycle**, not a checklist. Stage 7 (Executable Knowledge) depends on Stage 6 (Reasoning), which depends on Stage 5 (Validation), which depends on Stage 4 (Governance), which depends on Stage 3 (Reuse), which depends on Stage 2 (Description), which depends on Stage 1 (Conceptual Modeling).

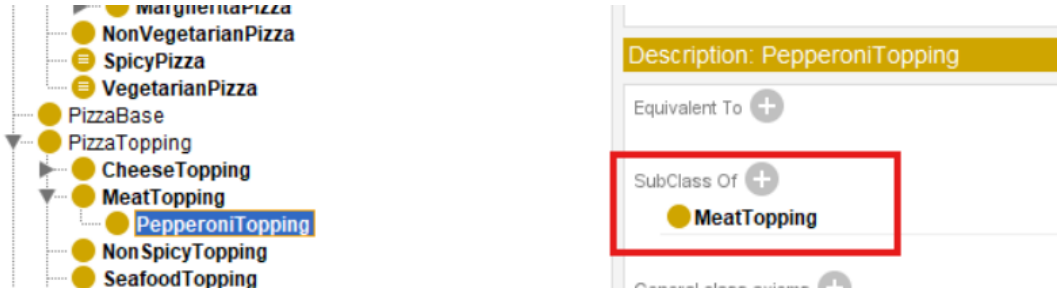
If a modeling error exists at Stage 1, **no amount of engineering at Stage 7 will fix it**. The reasoner will faithfully compute the logical consequences of the axioms it is given -- including the axioms that encode the wrong taxonomy. Attempt 1 failed because the closure axiom's effect was undermined by a Stage 1 classification error that the closure could not compensate for.

The honest engineering response is to go back to Stage 1, correct the model, and then re-run the Stage 7 work.

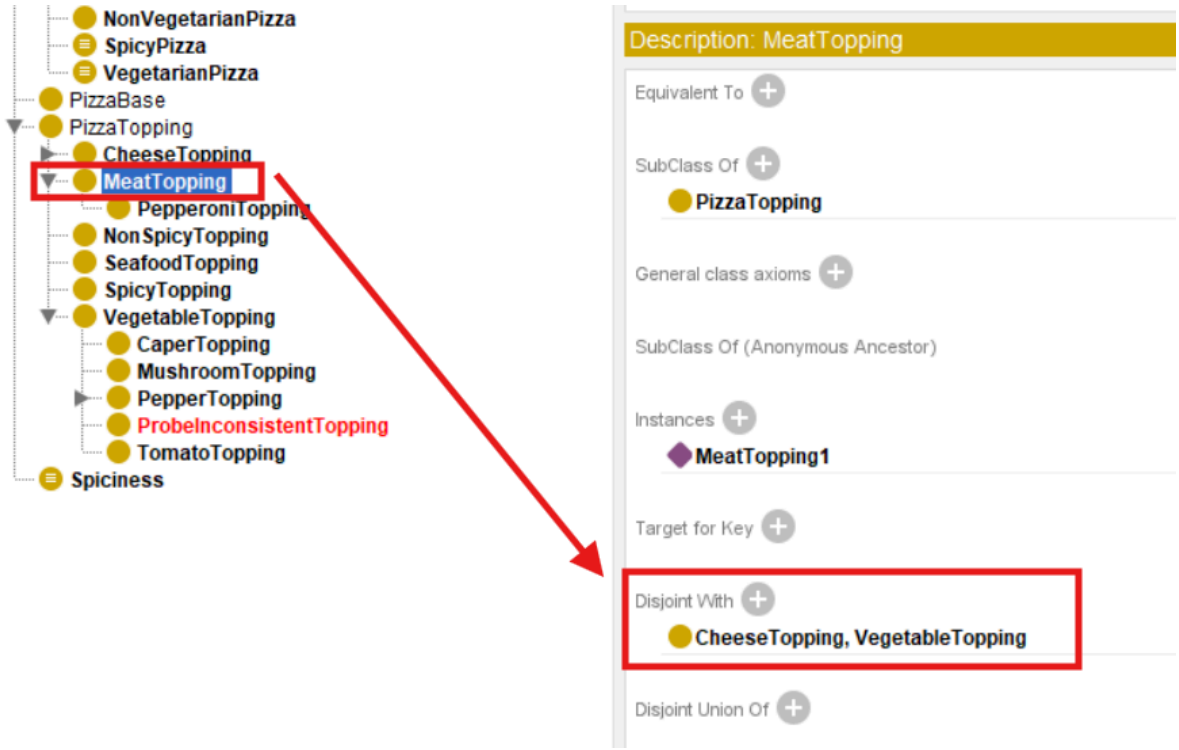
Attempt 2 -- Correct the first Stage 1 modeling error, then re-test

The fix is a one-line change in the class hierarchy:

```
Class: PepperoniTopping
    SubClassOf: MeatTopping     # was: VegetableTopping
```



This moves `PepperoniTopping` out from under `VegetableTopping` and into its correct position under `MeatTopping`. Now `PepperoniTopping` is a subclass of `MeatTopping`, and `MeatTopping` is disjoint from `CheeseTopping` and `VegetableTopping` (per the ontology's `AllDisjointClasses` axioms).

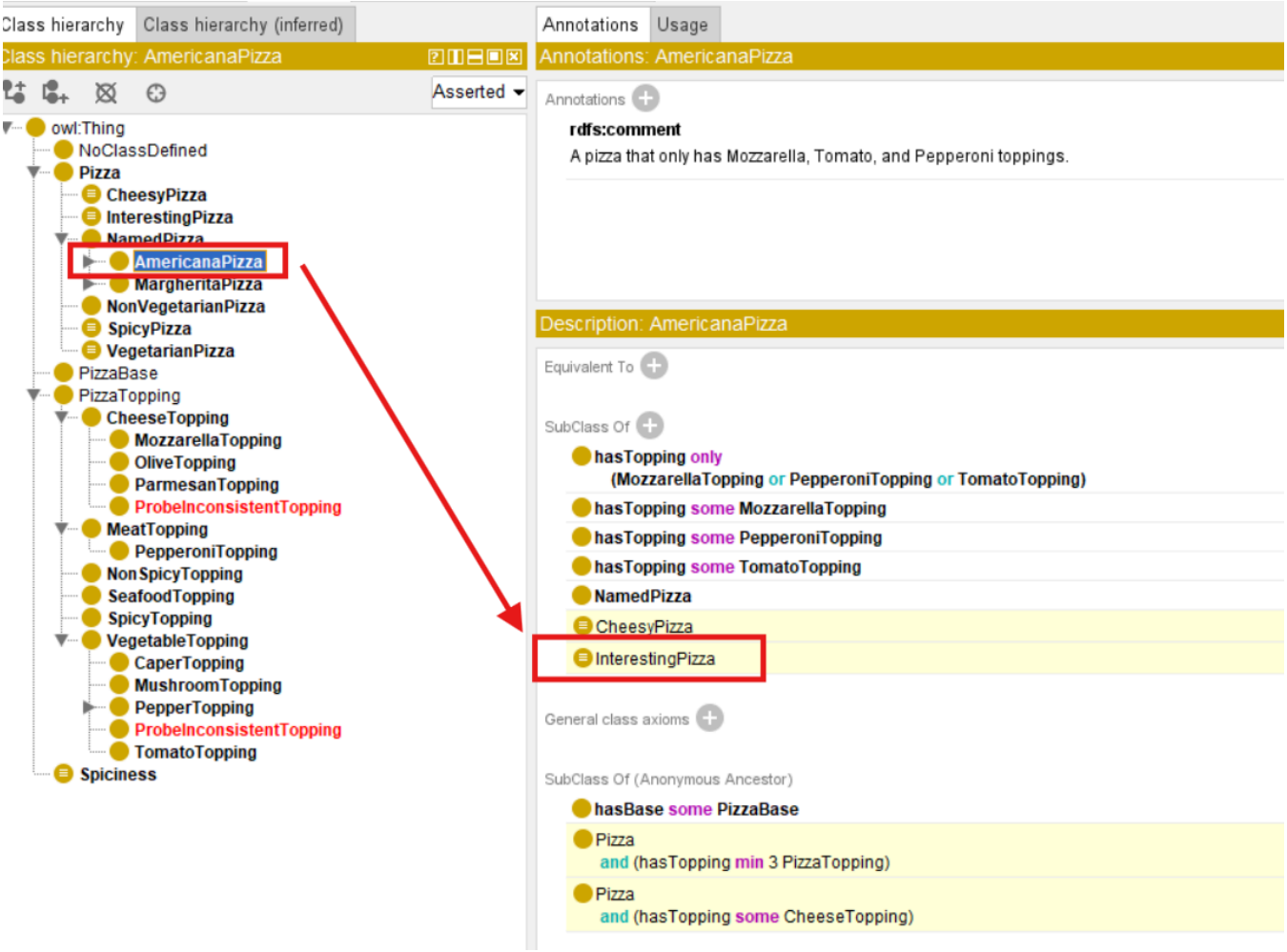


With this correction:

- `MozzarellaTopping` (Cheese) is provably distinct from `PepperoniTopping` (Meat). ✅
- `MozzarellaTopping` (Cheese) is provably distinct from `TomatoTopping` (Vegetable). ✅
- `PepperoniTopping` (Meat) is provably distinct from `TomatoTopping` (Vegetable). ✅

All three of `AmericanaPizza`'s toppings are now provably pairwise distinct. The reasoner can count them. Combined with the closure axiom from Attempt 1, the min-3 cardinality should now fire for `AmericanaPizza`.

Synchronize the reasoner and observe. This time, the result of `AmericanaPizza` is what we expect:



`AmericanaPizza` is now classified as subclass of `InterestingPizza`. **Good progress!** Its three toppings {Mozzarella, Pepperoni, Tomato} are pairwise provably distinct (Cheese ⊥ Meat ⊥ Vegetable), and the closure from Attempt 1 supplies the exhaustive count.

However, `AmericanaHotPizza` and `SohoPizza` are still not classified. Correcting one modeling error has fixed one pizza, but not the other two. The journey continues.

Attempt 2.1 -- Diagnose the remaining two pizzas

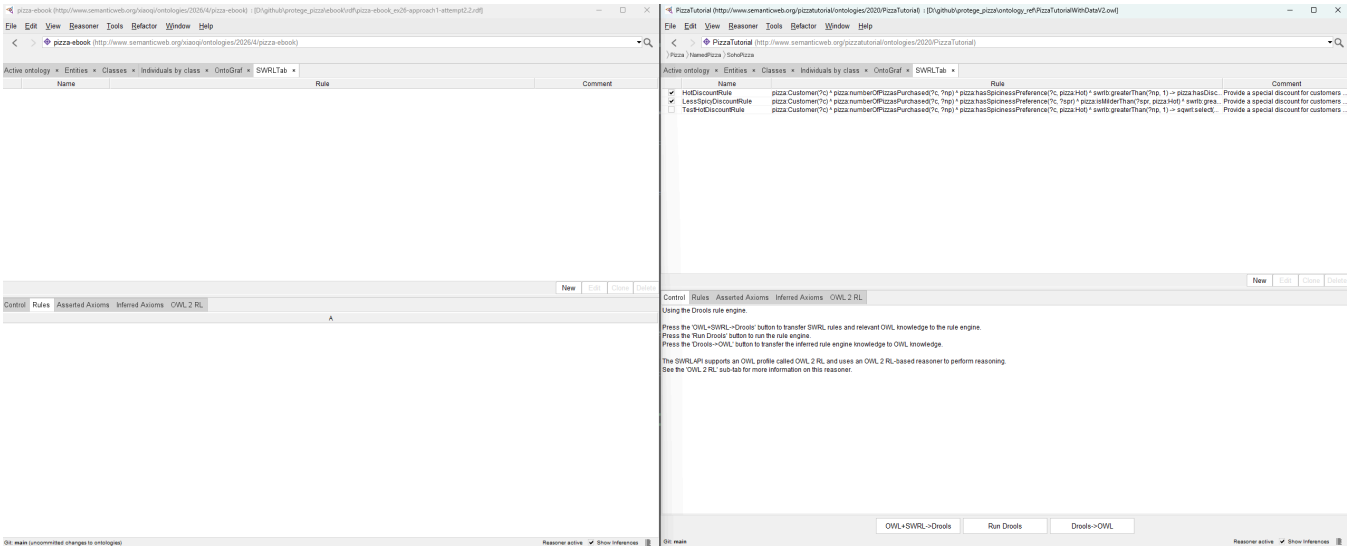
This is a **diagnostic** step. No new corrections are applied here. We investigate *why* the other two pizzas still fail.

A preliminary tooling issue

Before we trust the reasoner's output, there is one thing to check. The Protégé log shows:

```
ERROR An error occurred during reasoning: A SWRL rule uses a built-in atom,
but built-in atoms are not supported yet.
java.lang.IllegalArgumentException: A SWRL rule uses a built-in atom,
but built-in atoms are not supported yet.
```

The active reasoner is **HermiT**, which does not support SWRL built-ins. The reference ontology contains two SWRL rules (`HotDiscountRule`, `LessSpicyDiscountRule`), both using `swrlb:greaterThan`. HermiT fails to load the ontology. Protégé falls backs to Pellet, but the reasoner state is unreliable.



After comparing from above screenshot, the problem is not in the working ontology. The working ontology has no SWRL rules so far. The `A SWRL rule uses a built-in atom` error appeared only when the **reference ontology (`PizzaTutorialWithDataV2.owl`)** was open in a separate Protégé window during earlier work. In that session, HermiT refused to load the reference ontology, but the error is unrelated to the working ontology

The working ontology has no SWRL rules and no reasoner configuration issue. Pellet or HermiT both work.

> **Note 1: Not all reasoners support all features.**
>
> HermiT supports OWL 2 DL but not SWRL built-ins. Pellet supports both. When the ontology uses SWRL, select Pellet. Reading the reasoner log -- rather than guessing -- is a discipline that pays off repeatedly in ontology engineering.

> **Note 2: A note on multi-window Protégé sessions.**
>
> When the working ontology and the reference ontology are open in separate Protégé windows, errors from one window can appear in the log of the other, causing confusion. The SWRL error above was from the reference ontology, not the working ontology. As a general practice, work with one ontology per Protégé session when diagnosing reasoner behavior. This avoids cross-contamination of logs and reasoner state.

Why `AmericanaHotPizza` still fails

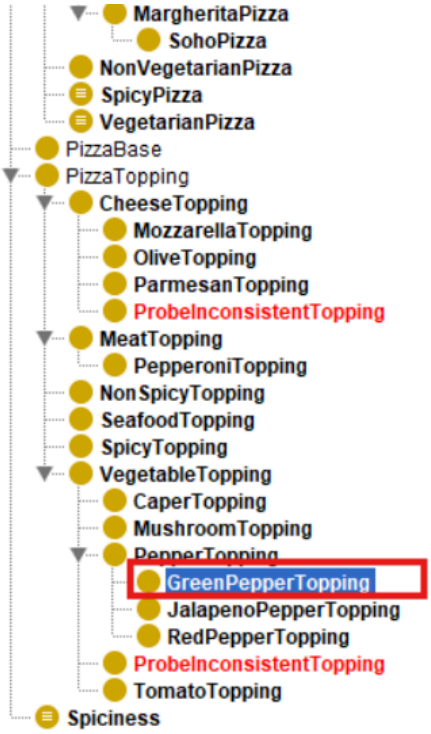
`AmericanaHotPizza` has four toppings: Mozzarella, Pepperoni, Tomato, JalapenoPepper.

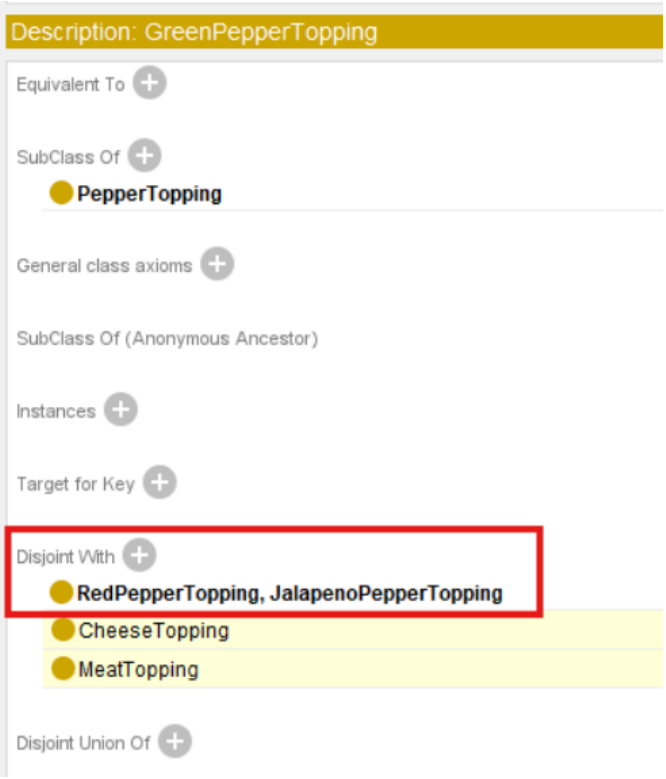
> **The `Topping` postfix is omitted for simplified display purposes only.**

After the Pepperoni fix (Correction 1), the disjointness checks are:

- Mozzarella (Cheese) vs. Pepperoni (Meat) ✅
- Mozzarella (Cheese) vs. Tomato (Vegetable) ✅
- Mozzarella (Cheese) vs. JalapenoPepper (Vegetable > Pepper) ✅
- Pepperoni (Meat) vs. Tomato (Vegetable) ✅
- Pepperoni (Meat) vs. JalapenoPepper (Vegetable > Pepper) ✅
- **Tomato (Vegetable) vs. JalapenoPepper (Vegetable > Pepper)** ❌

Both `TomatoTopping` and `JalapenoPepperTopping` sit under `VegetableTopping`. The working ontology declares only the three pepper subclasses (`GreenPepper` , `JalapenoPepper` , and `RedPepper`) as pairwise disjoint.





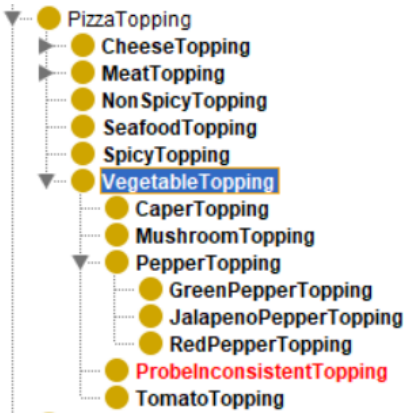
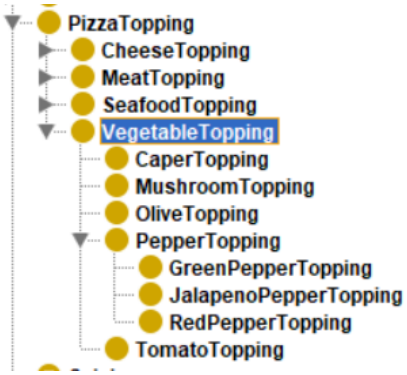
It does **not** declare the broader vegetable subclasses (`Caper` , `Mushroom` , `Pepper` , `Tomato`) as disjoint.

> **You may still see that `ProbeInconsistentTopping` as subclass of `VegetableTopping` , leave it as that's not impact this fixing result.**

The reference ontology has this broader declaration for `VegetableTopping` :

```
<owl:members rdf:parseType="Collection">
    <rdf:Description rdf:about="...#CaperTopping"/>
    <rdf:Description rdf:about="...#MushroomTopping"/>
    <rdf:Description rdf:about="...#OliveTopping"/>
    <rdf:Description rdf:about="...#PepperTopping"/>
    <rdf:Description rdf:about="...#TomatoTopping"/>
</owl:members>
```

> **The working ontology doesn't have `OliveTopping` as a subclass of `VegetableTopping` so far, which is mentioned below as another modeling difference.**

Working Ontology	Reference Ontology
	

The working ontology is **missing this broader disjointness declaration**, also `OliveTopping` is wrongly modeled as a subclass of `CheeseTopping` while should be a subclass of `VegetableTopping`. Without the correct modeling and proper broader disjointness declaration, the reasoner cannot prove that Tomato and JalapenoPepper are disjoint.

> **You may still see `ProbeInconsistentTopping` as a subclass of `VegetableTopping`. Leave it -- it does not impact this fixing result.**

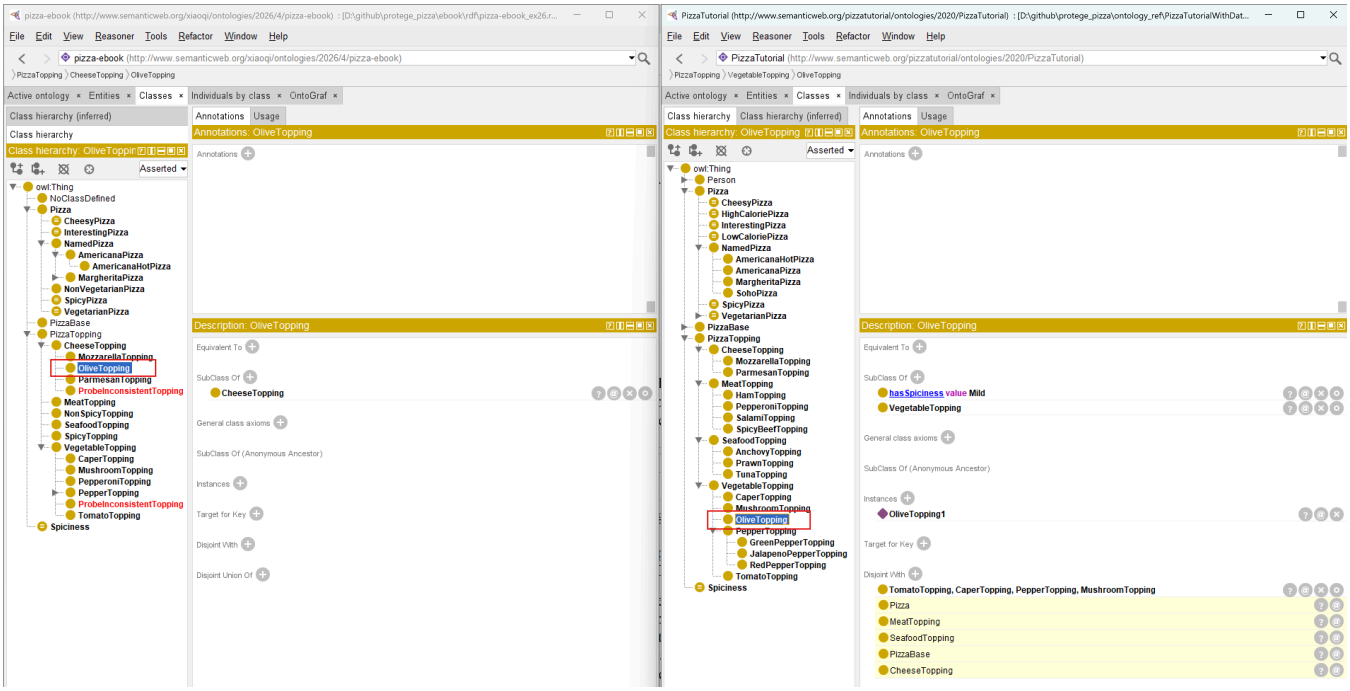
Why `SohoPizza` still fails

`SohoPizza` also has four toppings: Mozzarella, Olive, Parmesan, Tomato. Two problems, of different kinds.

Problem 1 -- a modeling error in the topping taxonomy. In the working ontology, as found in above `AmericanaHotPizza` analysis, `OliveTopping` is declared as a subclass of `CheeseTopping`:

```
<owl:Class rdf:about="...#OliveTopping">
    <rdfs:subClassOf rdf:resource="...#CheeseTopping"/>
</owl:Class>
```

But Olive is not a cheese. In the reference ontology, `OliveTopping` is a subclass of `VegetableTopping` (see table above). **This is a second Stage 1 modeling error.**



With Olive misplaced under Cheese, `SohoPizza`'s toppings become: Mozzarella (Cheese), Tomato (Vegetable), Olive (Cheese), Parmesan (Cheese). The reasoner checks:

- Mozzarella (Cheese) vs. Tomato (Vegetable) ✅
- Mozzarella (Cheese) vs. Olive (Cheese) -- both cheese, no disjointness declared ❌
- Mozzarella (Cheese) vs. Parmesan (Cheese) -- both cheese, no disjointness declared ❌
- Olive (Cheese) vs. Parmesan (Cheese) ❌
- Tomato (Vegetable) vs. all cheeses ✅

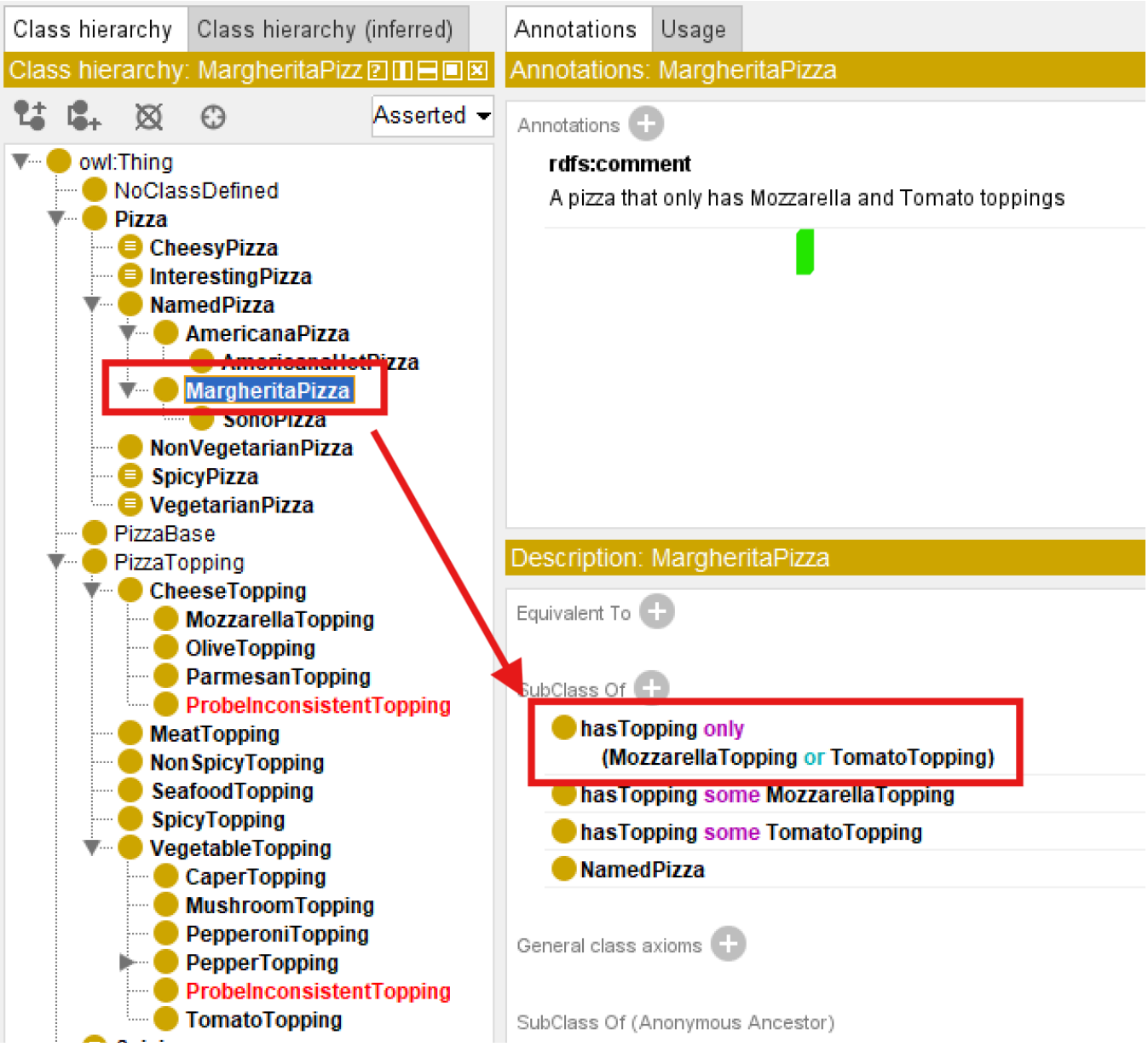
The reasoner can prove at most **two** distinct toppings for `SohoPizza`. The min-3 fails.

Problem 2 -- a closure-inheritance conflict. `SohoPizza` is declared as a subclass of `MargheritaPizza`:

```
<owl:Class rdf:about="...#SohoPizza">
    <rdfs:subClassOf rdf:resource="...#MargheritaPizza"/>
    ...
</owl:Class>
```

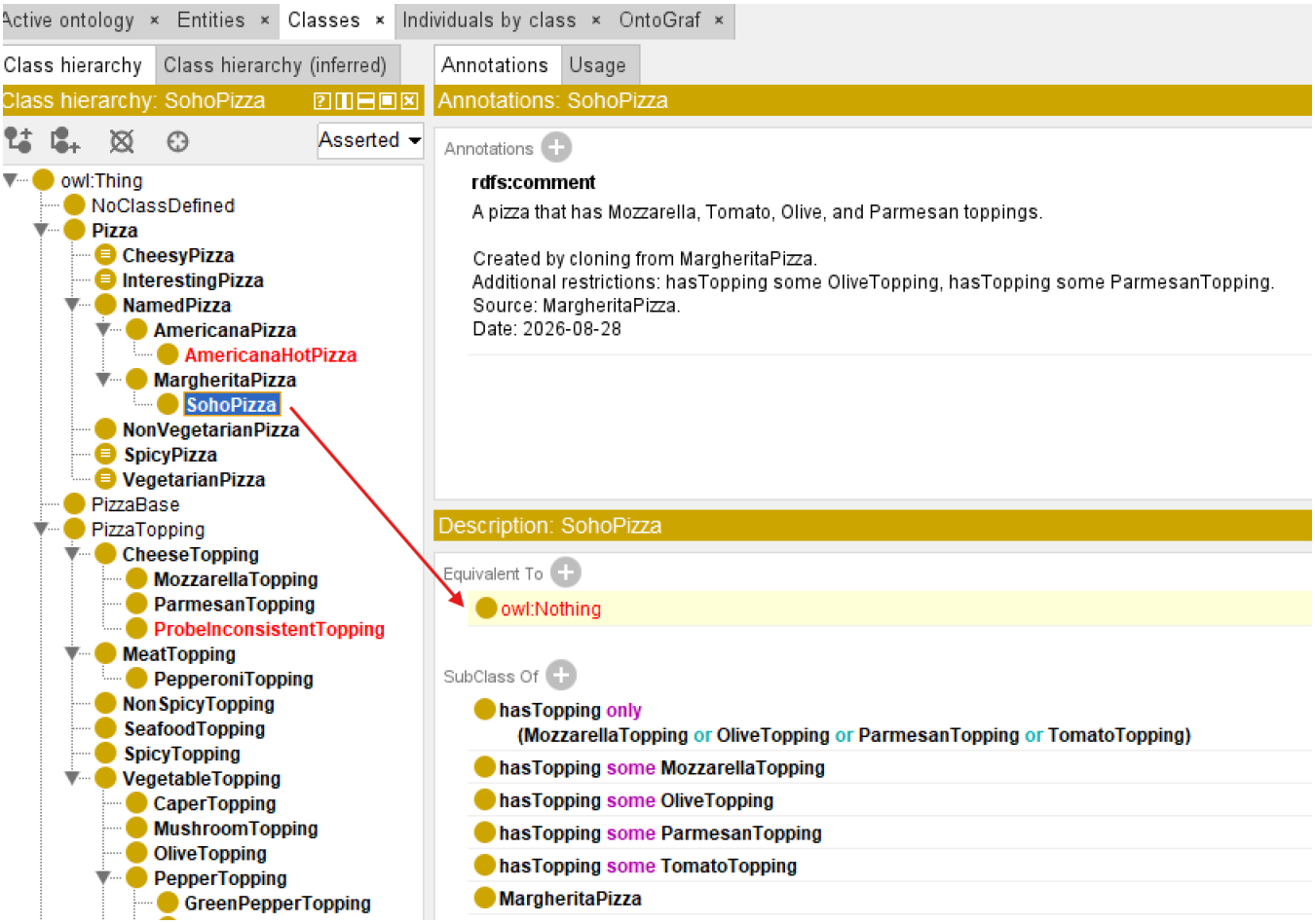
And, in working ontology, `MargheritaPizza` carries a closure axiom from Chapter 24:

```
Class: MargheritaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping)
```

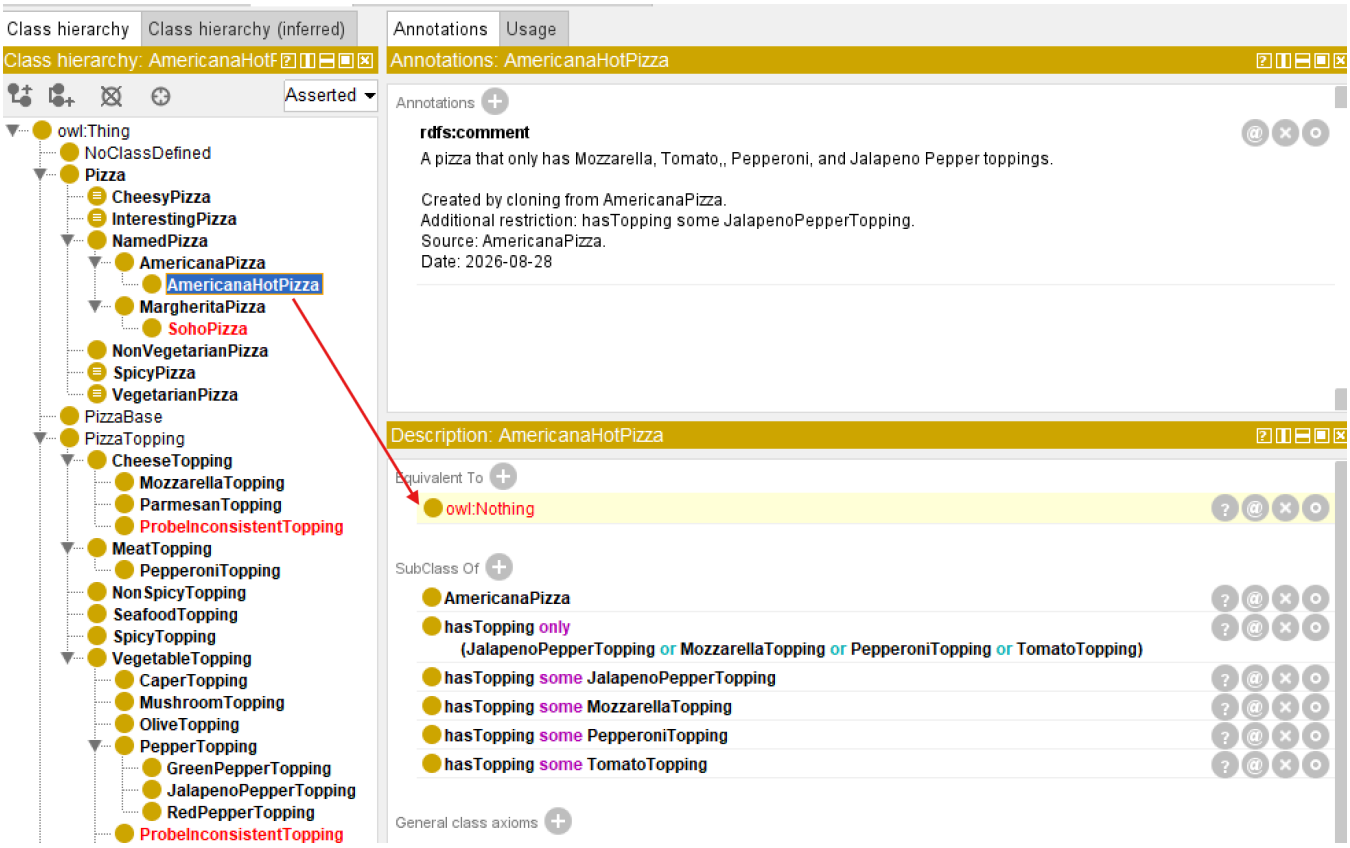


This closure, due to `SohoPizza` $\sqsubseteq$ `MargheritaPizza`, restricts every `MargheritaPizza`'s toppings to {Mozzarella, Tomato}. `SohoPizza` inherits this. But `SohoPizza` also has `hasTopping some OliveTopping` and `hasTopping some ParmesanTopping`, requiring at least one of each. Neither Olive nor Parmesan is in the inherited union.

The two axioms cannot both be satisfied. The reasoner correctly infers `SohoPizza` to be `owl:Nothing` -- an unsatisfiable class.



The same closure-inheritance conflict affects `AmericanaHotPizza`. `AmericanaHotPizza` is declared as a subclass of `AmericanaPizza`, which carries a closure to {Mozzarella, Pepperoni, Tomato}. `AmericanaHotPizza` adds `hasTopping some JalapenoPepperTopping`. Jalapeno is not in the inherited union. `AmericanaHotPizza` is also inferred to be `owl:Nothing`.



Summary of Attempt 2.1

We now know:

- `AmericanaPizza` was fixed in Attempt 2 (Correction 1).
- `AmericanaHotPizza` needs the broad vegetable disjointness (Correction 3) **and** a resolution of the closure-inheritance conflict (Corrections 4-6).
- `SohoPizza` needs the Olive reclassification (Correction 2), the broad vegetable disjointness (Correction 3), **and** a resolution of the closure-inheritance conflict (Corrections 4-6).
- The reasoner must be switched to use Pellet.

No fix is applied in this step. Attempt 2.2 applies them.

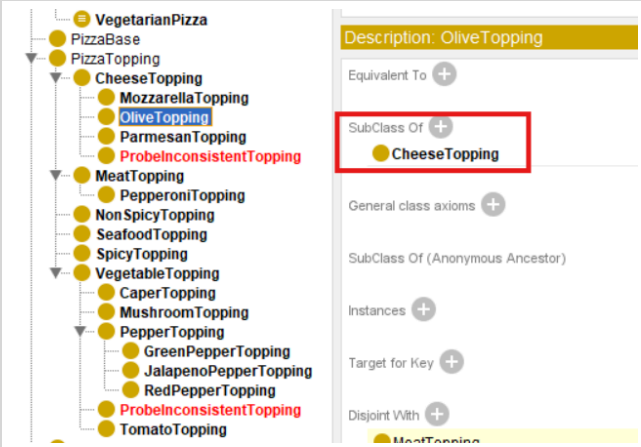
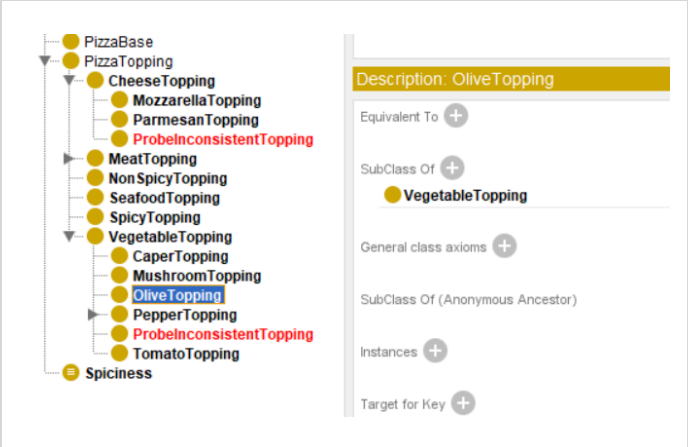
Attempt 2.2 -- Apply the remaining corrections and re-test

The complete set of corrections, with the order in which they will be applied:

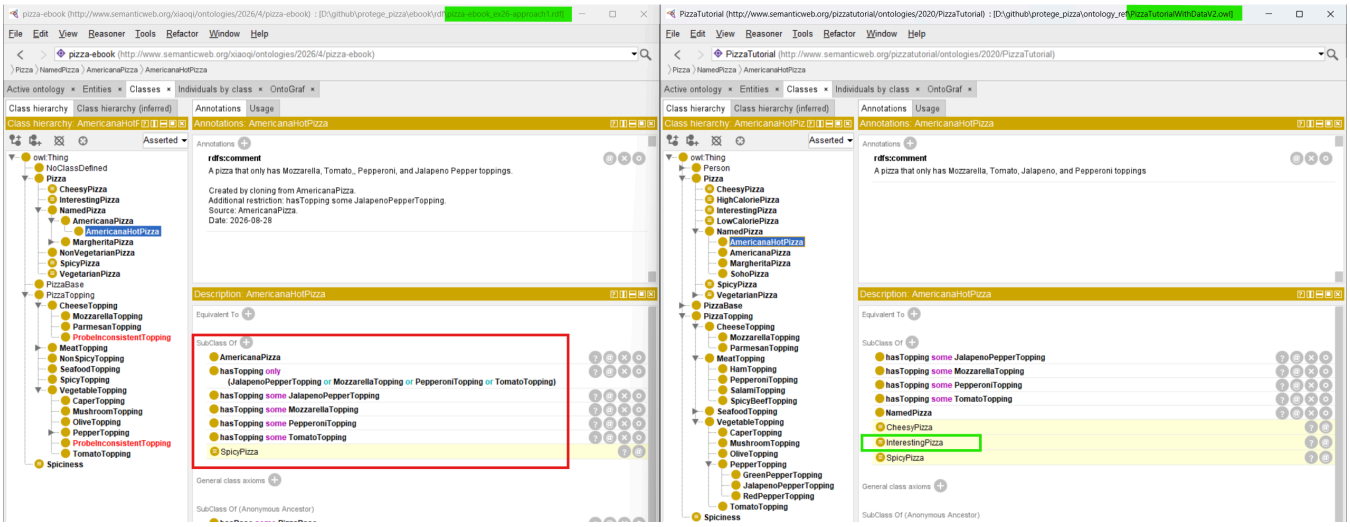
#	Correction	Type	Applied in
1	`PepperoniTopping` ⊑ `MeatTopping` (was `VegetableTopping`)	Taxonomy	Attempt 2
2	`OliveTopping` ⊑ `VegetableTopping` (was `CheeseTopping`)	Taxonomy	Attempt 2.2
3	`DisjointClasses: Caper, Mushroom, Olive, Pepper, Tomato`	Disjointness	Attempt 2.2
4	`SohoPizza` ⊑ `NamedPizza` (was `MargheritaPizza`)	Hierarchy	Attempt 2.2
5	Remove closure from `AmericanaPizza`	Closure	Attempt 2.2
6	Introduce `PlainAmericanaPizza` with closure {M, P, T}	Structure	Attempt 2.2

Correction 2 -- Move `OliveTopping` to `VegetableTopping`

```
Class: OliveTopping
    SubClassOf: VegetableTopping    # was: CheeseTopping
```

Before Fixing	After Fixing
	

After this correction, synchronizing the reasoner. `AmericanaHotPizza` and `SohoPizza` **still do not classify**:

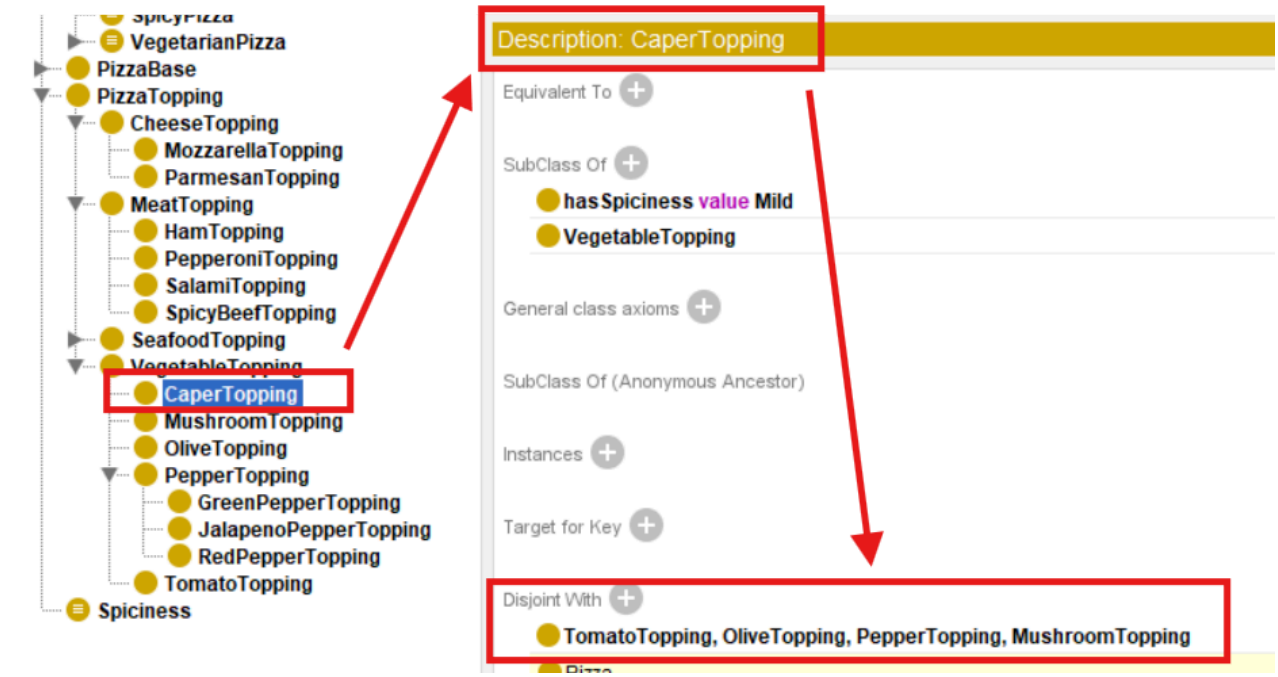


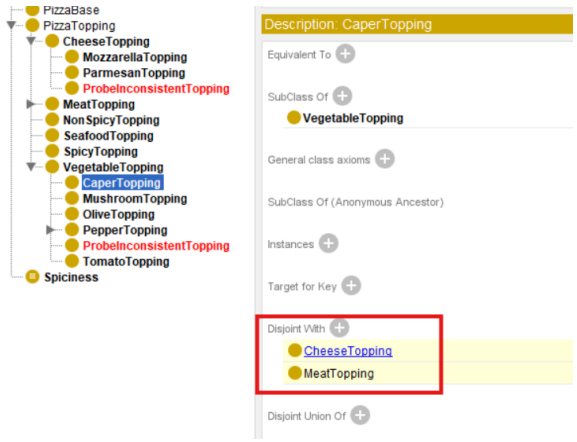
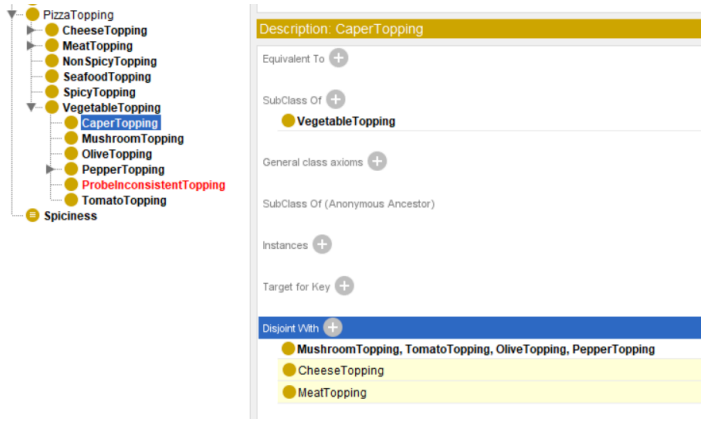
The fixing is not completed. This is expected -- the closure-inheritance conflict remains, and the broader disjointness is still missing.

Correction 3: Add the broad vegetable disjointness declaration

```
DisjointClasses: CaperTopping, MushroomTopping, OliveTopping, PepperTopping, TomatoTopping
```

The reference ontology shows this kind of broad vegetable disjointness:



Before adding broad Disjointness	After adding broad Disjointness
	

After this correction, synchronize the reasoner again. Still, `AmericanaHotPizza` and `SohoPizza` do not classify. **The disjointness was not the only blocker.** The remaining blocker is the closure-inheritance conflict.

Diagnosing the closure-inheritance conflict

Let us name the pattern we have discovered, because it is general:

> **A closure axiom on a parent class propagates to every subclass. If any subclass extends the parent's topping set, the inherited closure contradicts the subclass' own restrictions, and the subclass becomes unsatisfiable (`owl:nothing`).**

Both branches of our pizza hierarchy exhibit this conflict:

- **Americana branch**: `AmericanaPizza` has closure {M, P, T}; `AmericanaHotPizza` extends it with Jalapeno. **Conflict.**
- **Margherita branch**: `MargheritaPizza` has closure {M, T} (added in Chapter 24); `SohoPizza` extends it with Olive and Parmesan. **Conflict.**

The reference ontology avoids this by **flattening the structure** -- all four named pizzas are siblings under `NamedPizza`. Our working ontology uses a richer, hierarchical structure, which is a legitimate modeling choice -- an Americana Hot Pizza *is a kind of* Americana Pizza with jalapeño added. But the hierarchy forces us to resolve the closure-inheritance conflict correctly.

There are four candidate fixes.

Option A -- Flatten the structure

Move `AmericanaHotPizza` and `SohoPizza` to be direct subclasses of `NamedPizza`, matching the reference ontology.

- *Pros*: Simple, known to work, no new classes.
- *Cons*: Loses the "is a kind of" semantic relationship.

Option B -- Widen the parent's closure to include the subclass's toppings

Change `AmericanaPizza`'s closure to include Jalapeno:

```
Class: AmericanaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping or JalapenoPepperTopping)
```

- *Pros*: Keeps the hierarchy. Small textual change.
- *Cons*: **This is semantically wrong.** A closure axiom is a universal restriction -- `hasTopping only X` says *for every value v of hasTopping, v ∈ X*. Widening the parent's closure to include Jalapeno says "every Americana Pizza may have a Jalapeno topping." But a plain Americana Pizza does not have jalapeño. We have leaked the subclass's extension into the parent's definition. The parent's closure no longer describes the parent; it describes the union of all descendants. The axiom loses its semantic precision.

We reject Option B. It resolves the reasoner's complaint by weakening the semantics of the parent.

Option C -- Introduce "plain" leaf classes that carry the closures

Keep the hierarchy, but remove the closure from the parent. Introduce a leaf class representing "plain Americana Pizzas with exactly the standard three toppings":

```
Class: PlainAmericanaPizza
    SubClassOf: AmericanaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping)
```

Closures now live only on the leaves. **No inheritance conflict.**

- *Pros*: Semantically clean. This is the well-known **two-tier** or **definition/closure separation** pattern.
- *Cons*: Introduces new classes. Existing instances would need to be reclassified.

Option D -- Hybrid: keep inheritance where it does not conflict; revert to flat where it does

Observe that the two branches have different historical constraints. `MargheritaPizza`'s closure was placed in Chapter 24 for the `VegetarianPizza` classification -- it is used elsewhere in the book. `AmericanaPizza`'s closure was placed in Attempt 1 of this exercise.

The pragmatic hybrid:

- **Americana branch:** keep the hierarchy, introduce `PlainAmericanaPizza` for the closure. This is Option C applied to one branch.
- **Margherita branch:** match the reference. Move `SohoPizza` to be a direct subclass of `NamedPizza`. This preserves Chapter 24's `MargheritaPizza` closure and its `VegetarianPizza` classification.

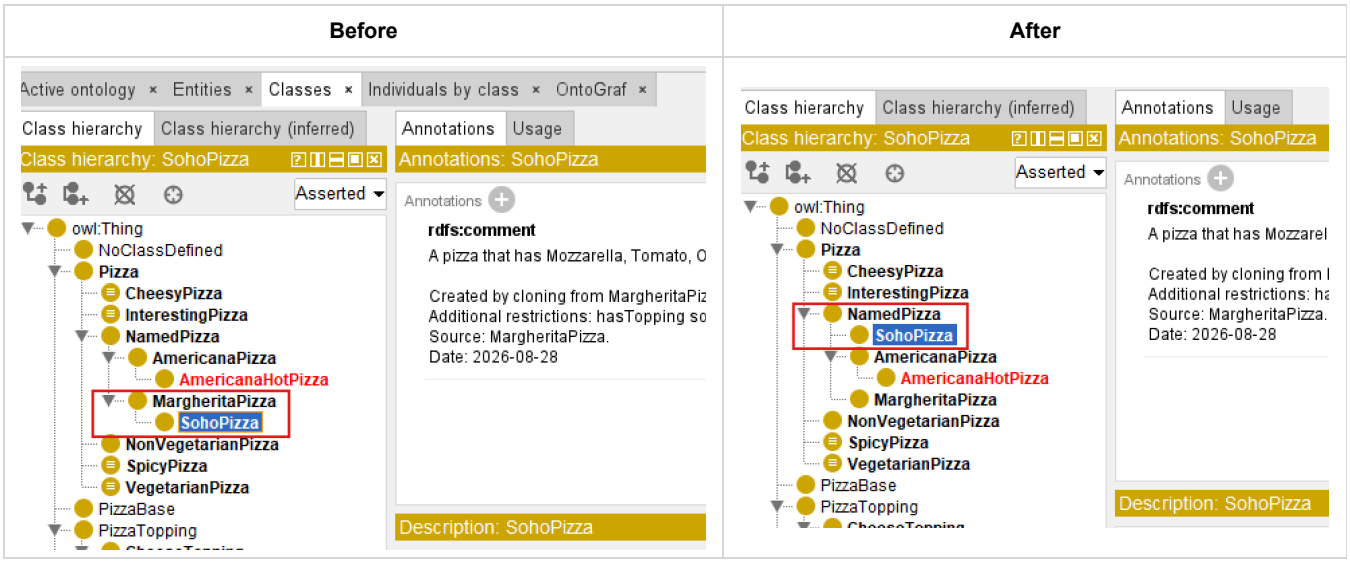
The result is asymmetric -- and that asymmetry is honest. It reflects the specific interactions between closures, inheritance, and the chapters already written.

Applying the hybrid fix (Option D)

> **In `rdf` folder, I've saved previous `approach1` ontology to a new file named as `pizza-ebook_ex26-approach1-attempt2.2.rdf`, which you can use for practicing below corrections.**

Correction 4 -- Move `SohoPizza` to `NamedPizza`

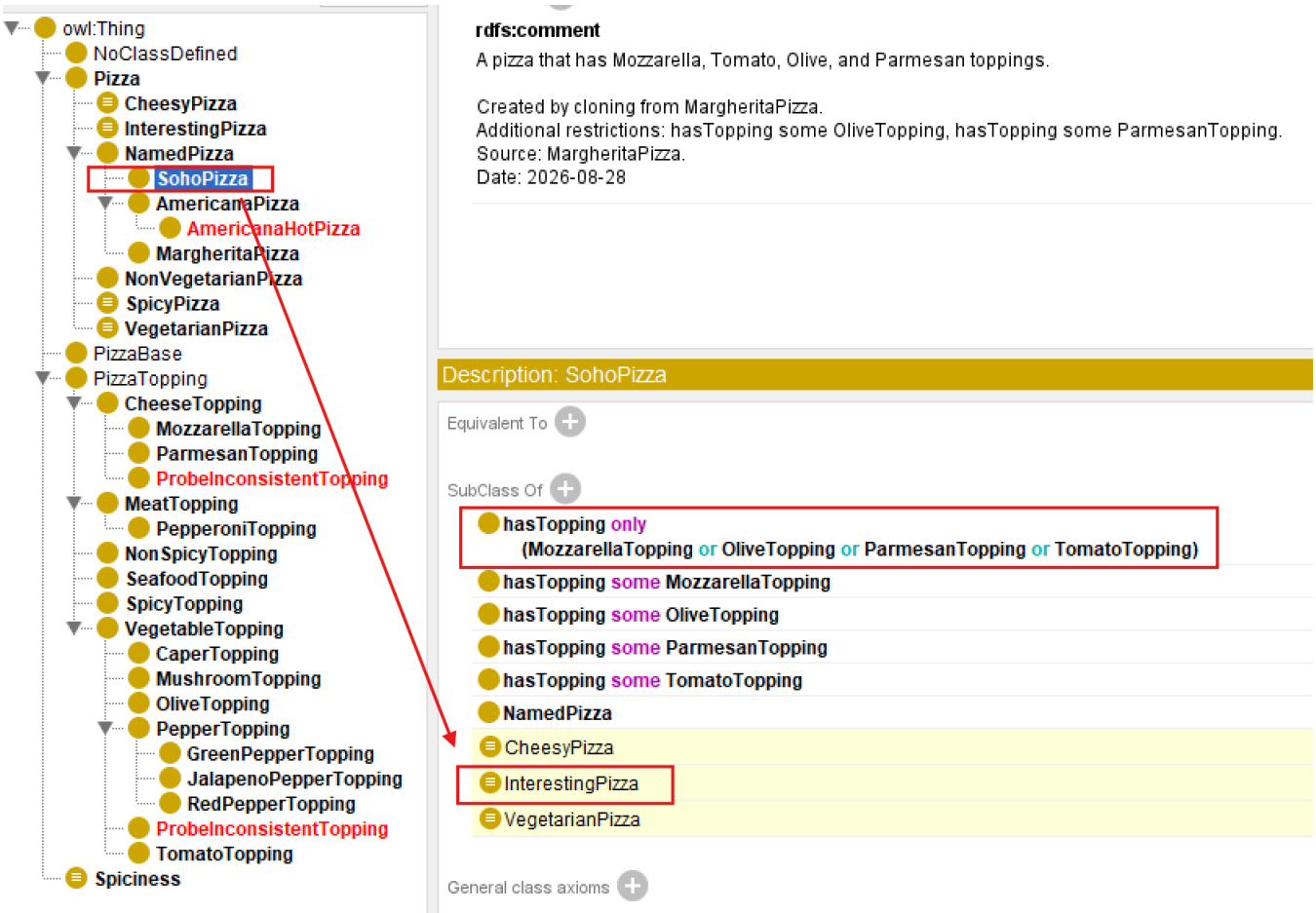
```
Class: SohoPizza
    SubClassOf: NamedPizza      # was: MargheritaPizza
```



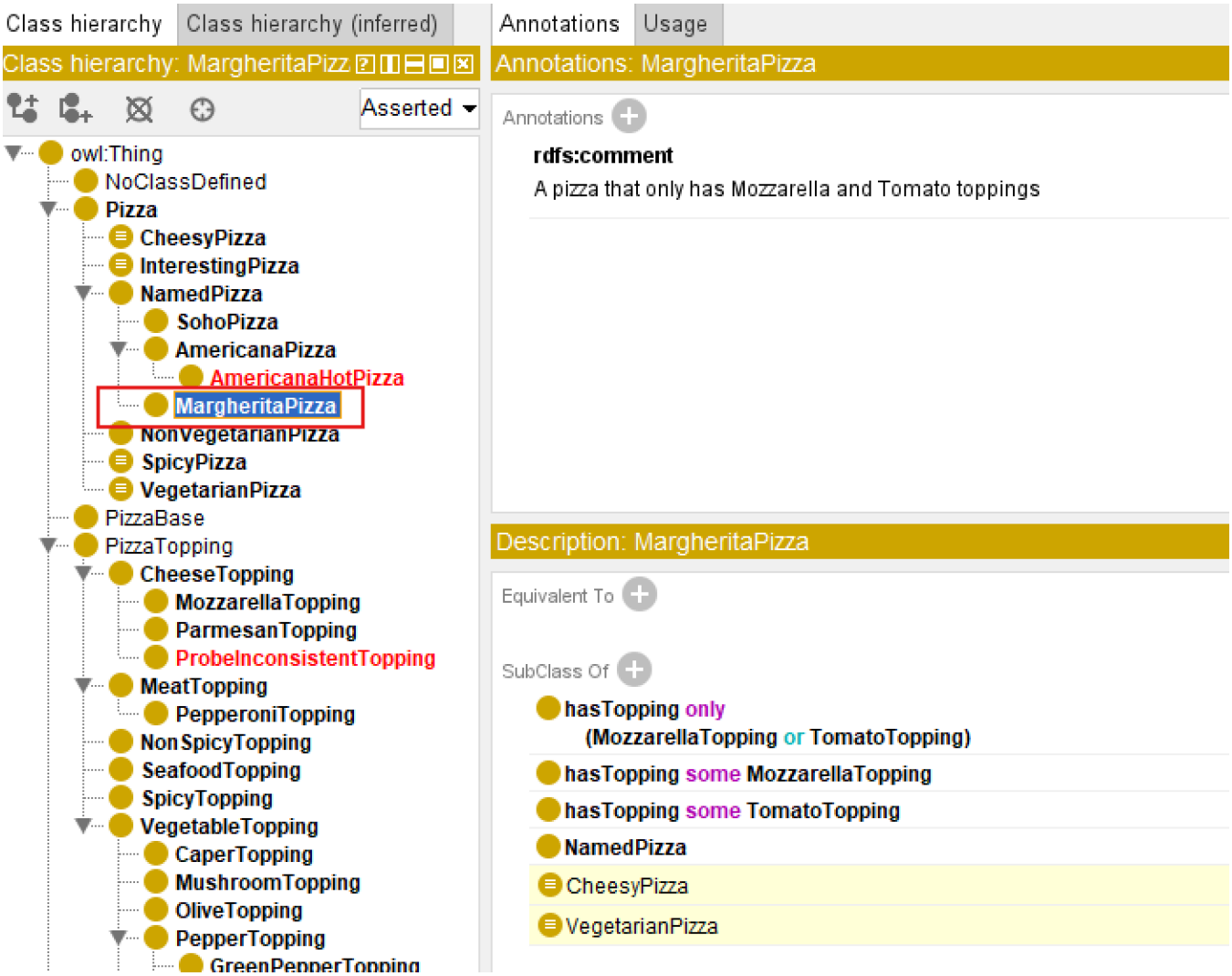
`SohoPizza` no longer inherits `MargheritaPizza`'s closure. Its own closure to {Mozzarella, Tomato, Olive, Parmesan} is the only closure that applies to it, and it is sound. `MargheritaPizza` retains its Chapter 24 closure -- no conflict, because it now has no subclass that extends its topping set.

Synchronize the reasoner and observe

`SohoPizza` (M, T + O + Pa): three provably distinct (M, T, O via the vegetable disjointness). ✅



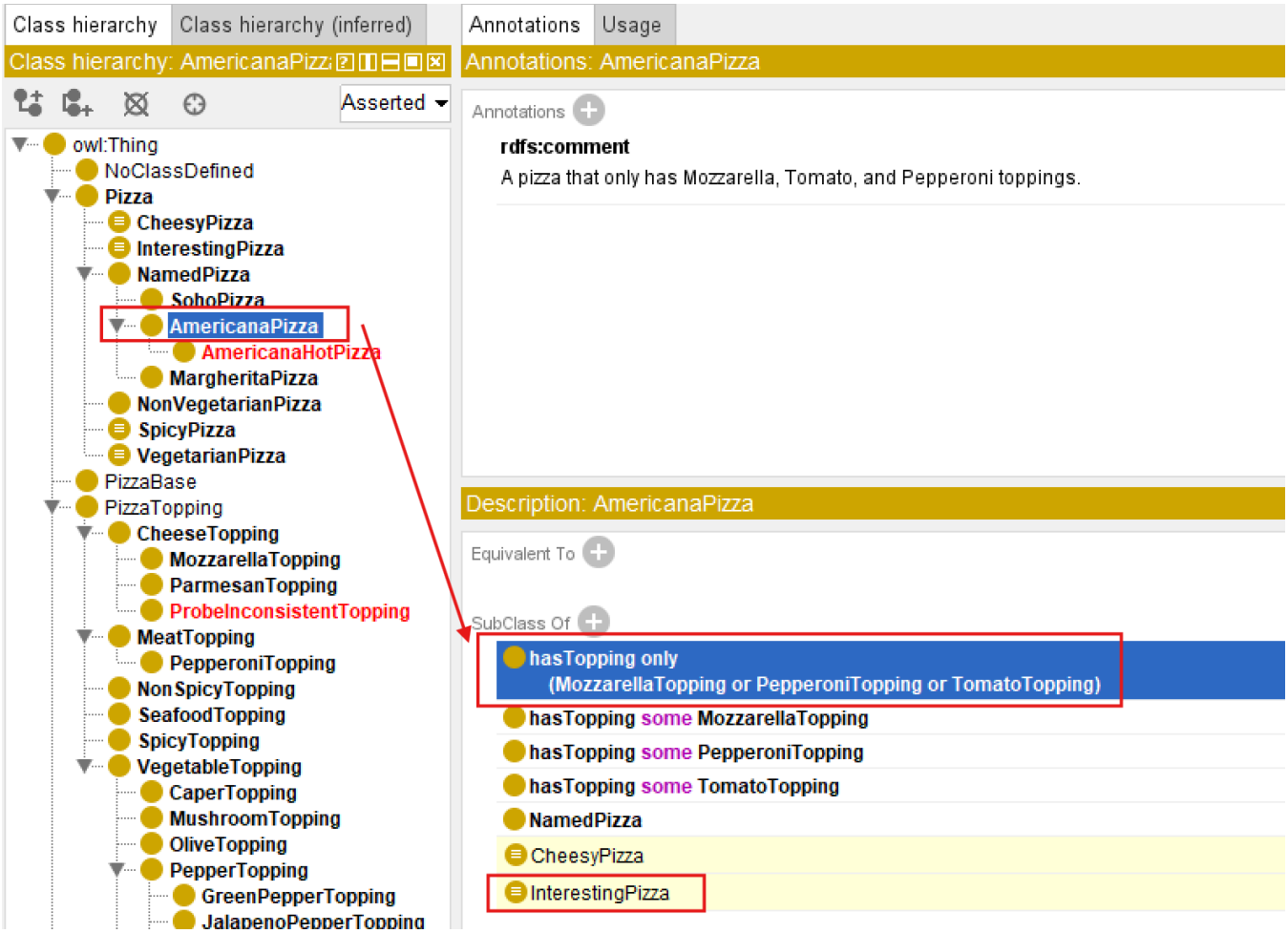
`MargheritaPizza` (M, T): two toppings -- correctly **not** classified as `InterestingPizza`.



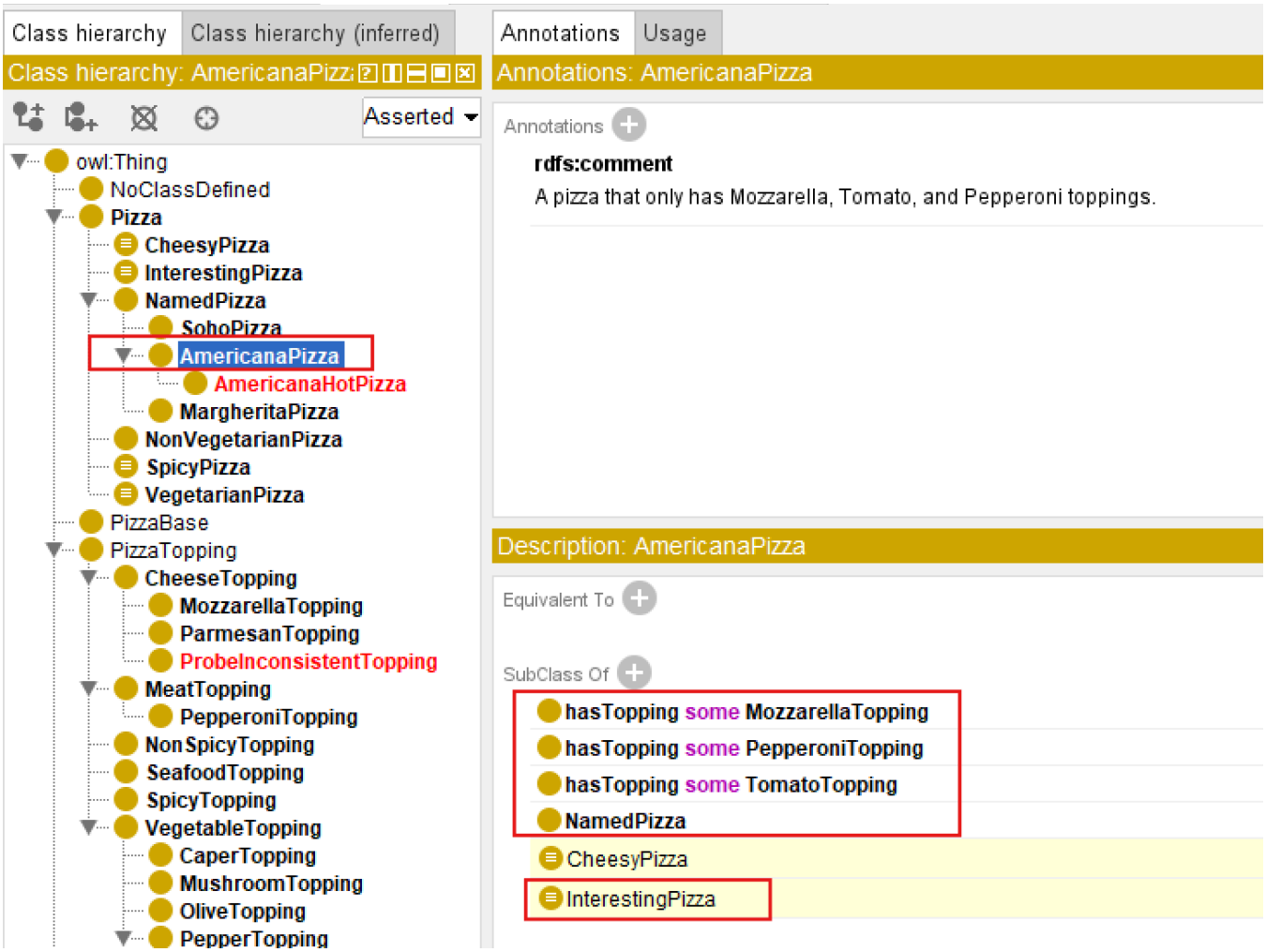
Correction 5 -- Remove the closure from `AmericanaPizza`

```
Class: AmericanaPizza
    SubClassOf: NamedPizza
    SubClassOf: hasTopping some MozzarellaTopping
    SubClassOf: hasTopping some PepperoniTopping
    SubClassOf: hasTopping some TomatoTopping
    # closure removed
```

Before Correction 5:

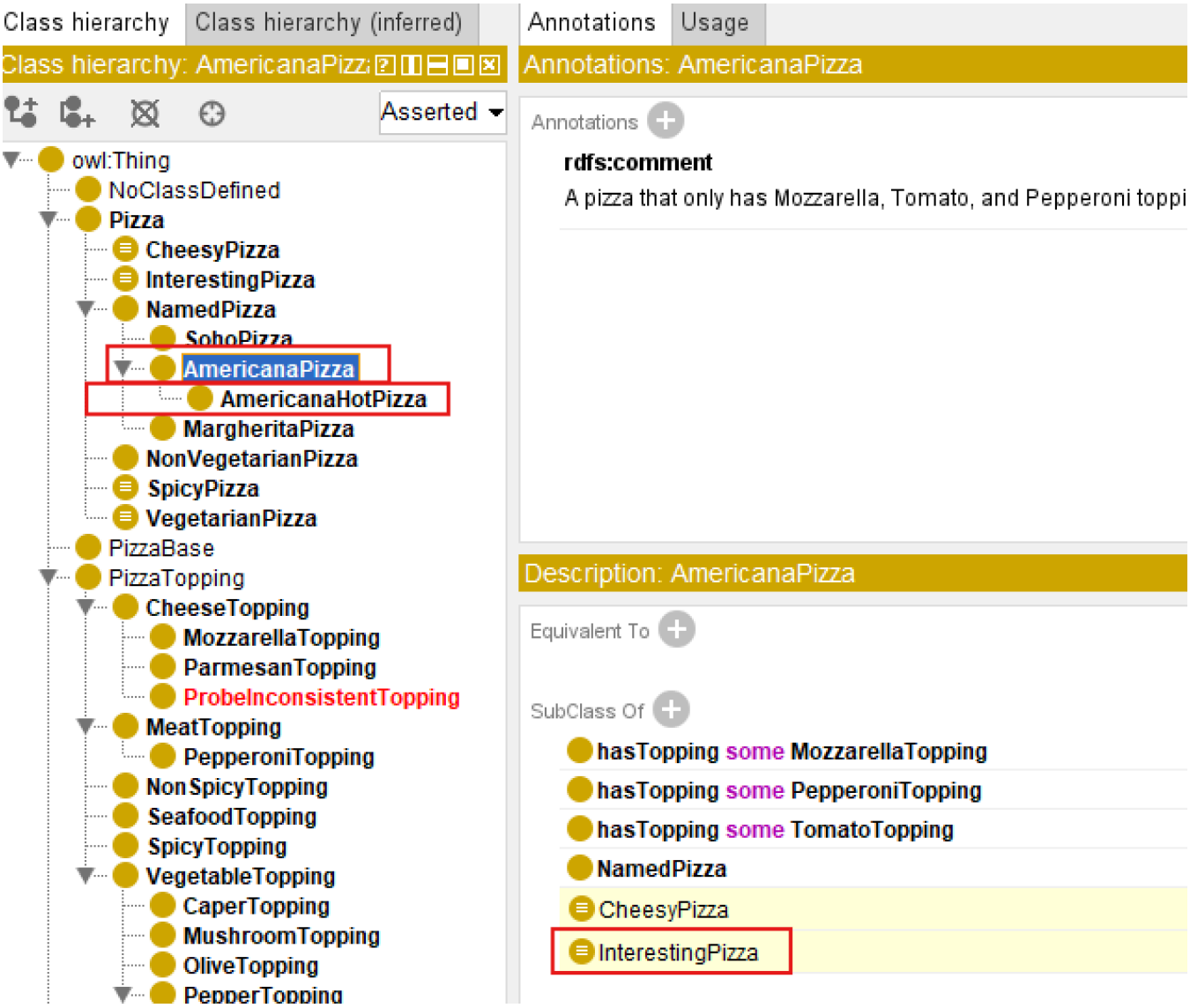


After Correction 5:

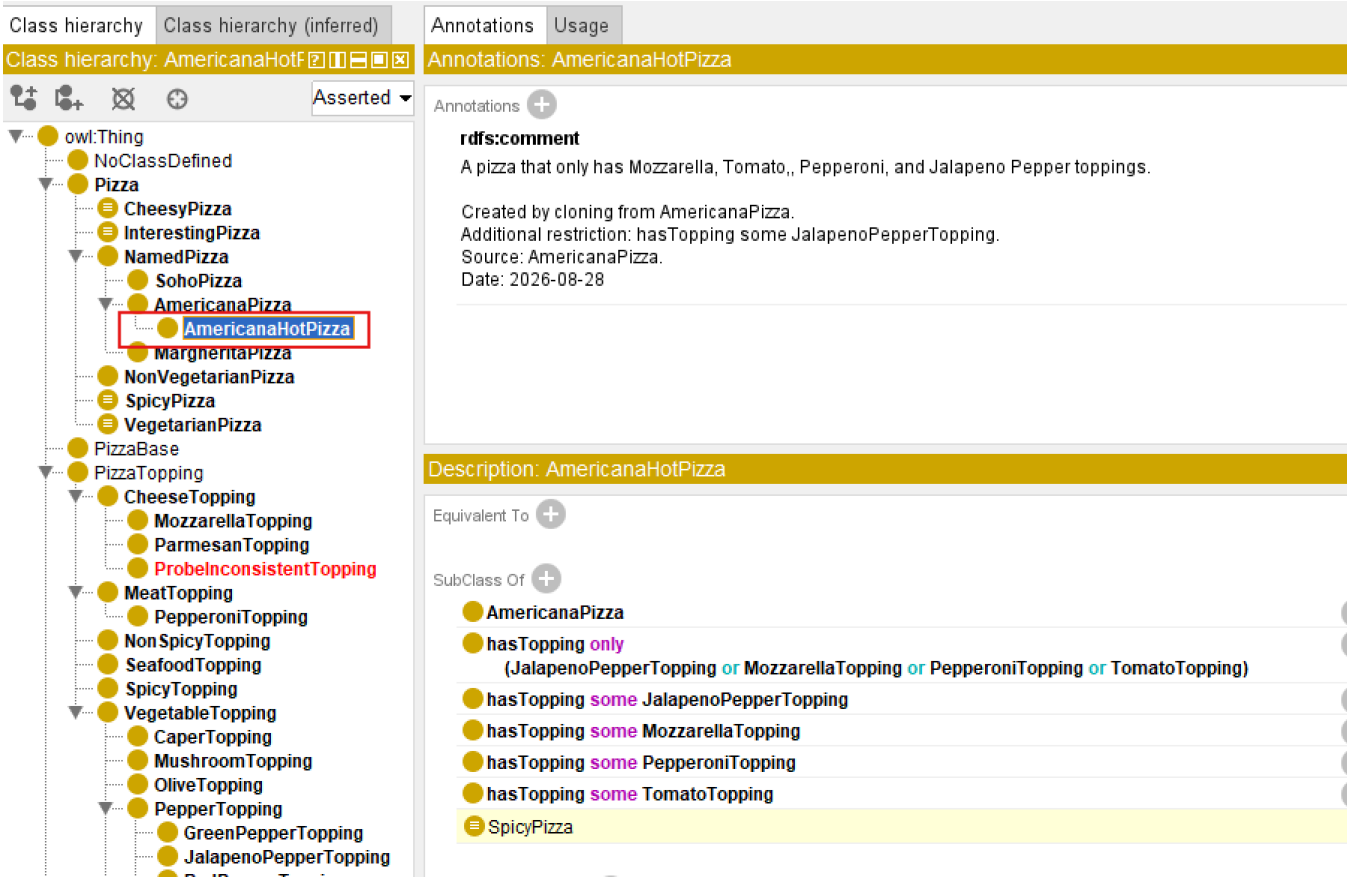


Synchronize the reasoner and observe

`AmericanaPizza` is still classified as subclass of `InterestingPizza`, since it fulfills the 3 distinct toppings criteria; and you may notice the inconsistency (red color) on `AmericanaHotPizza` has disappeared, means there's no axiom inheritance conflicts after this correction.

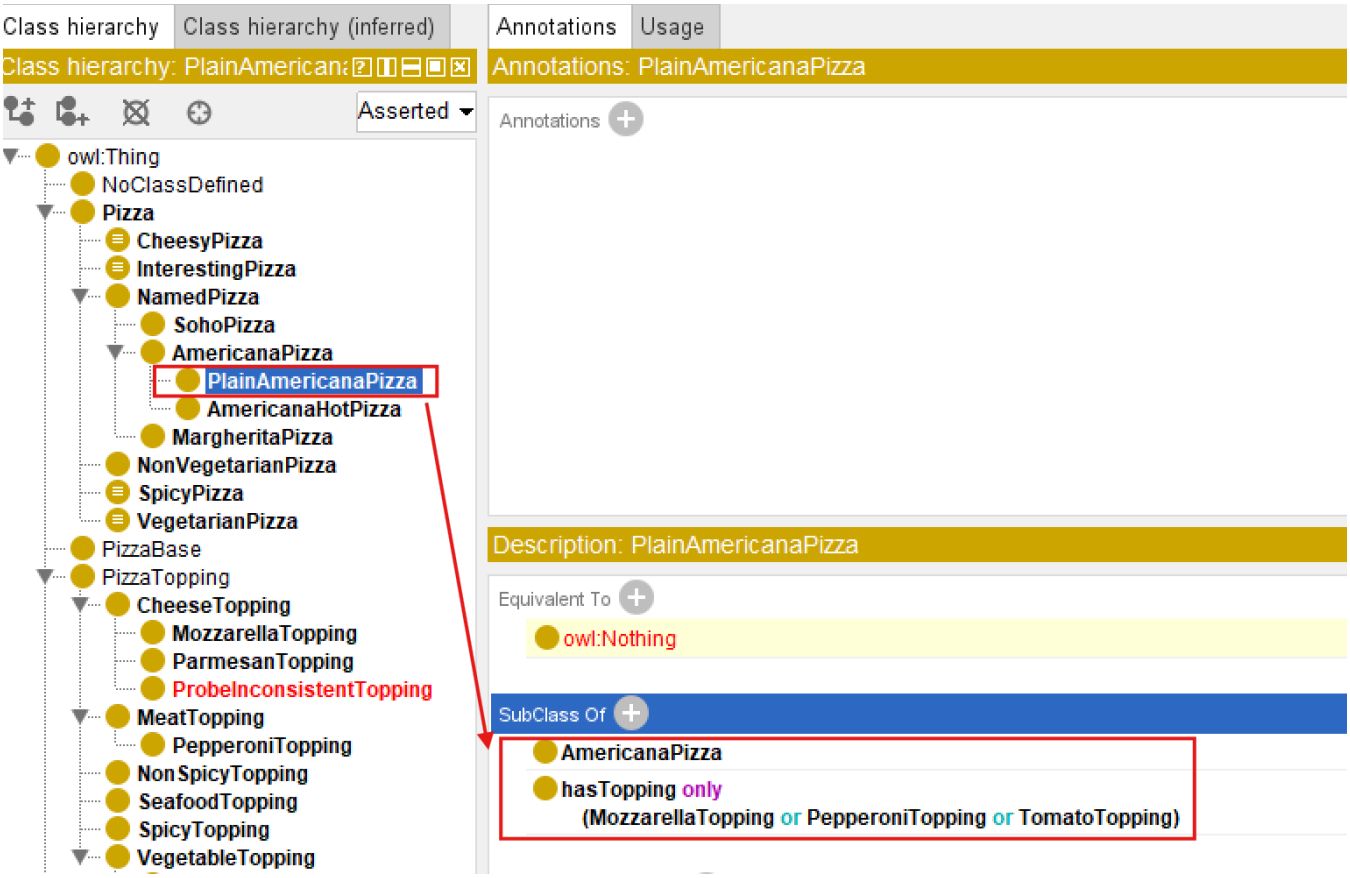


However, `AmericanaHotPizza` is still not classified as subclass of `InterestingPizza`:



Correction 6 -- Introduce `PlainAmericanaPizza` with the closure

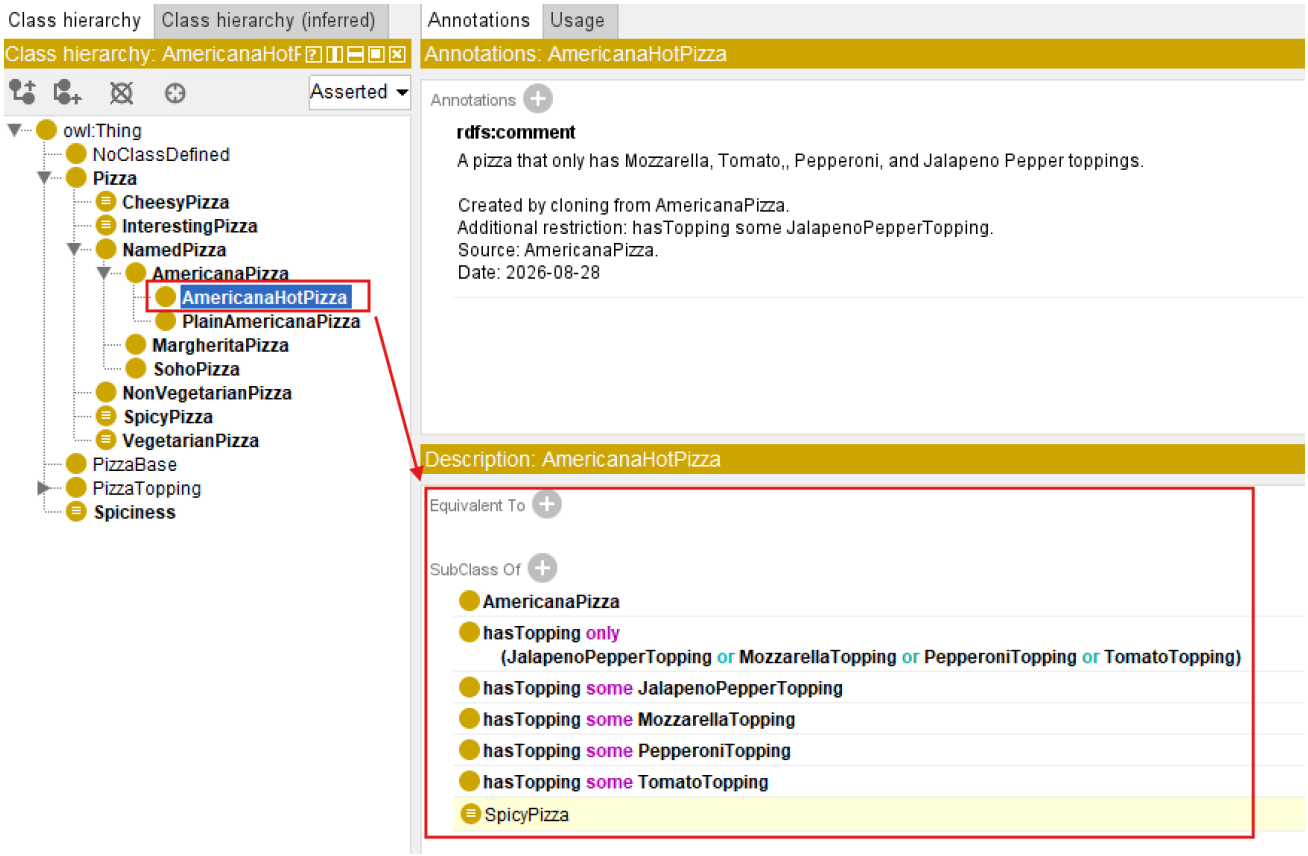
```
Class: PlainAmericanaPizza
    SubClassOf: AmericanaPizza
    SubClassOf: hasTopping only (MozzarellaTopping or PepperoniTopping or TomatoTopping)
```



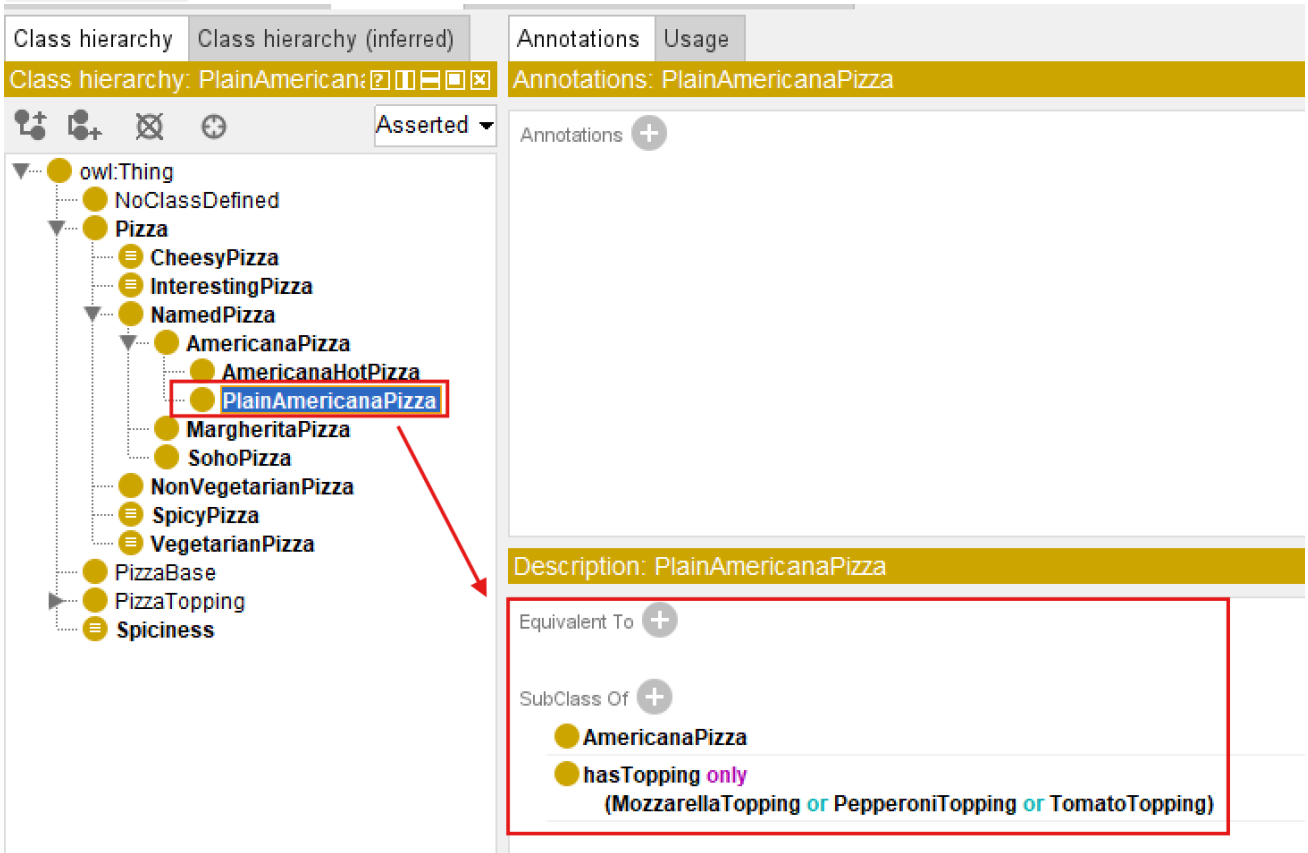
The closure now lives on the leaf. `AmericanaHotPizza` keeps its own closure to {M, P, T, J}. Neither conflicts with the other.

Synchronize the reasoner and observe

- `AmericanaPizza`, no closure: classified as subclass of `InterestingPizza`. ✅
- `PlainAmericanaPizza` (M, P, T): three provably distinct toppings. ❌



- `AmericanaHotPizza` (M, P, T + J): four provably distinct toppings. ❌



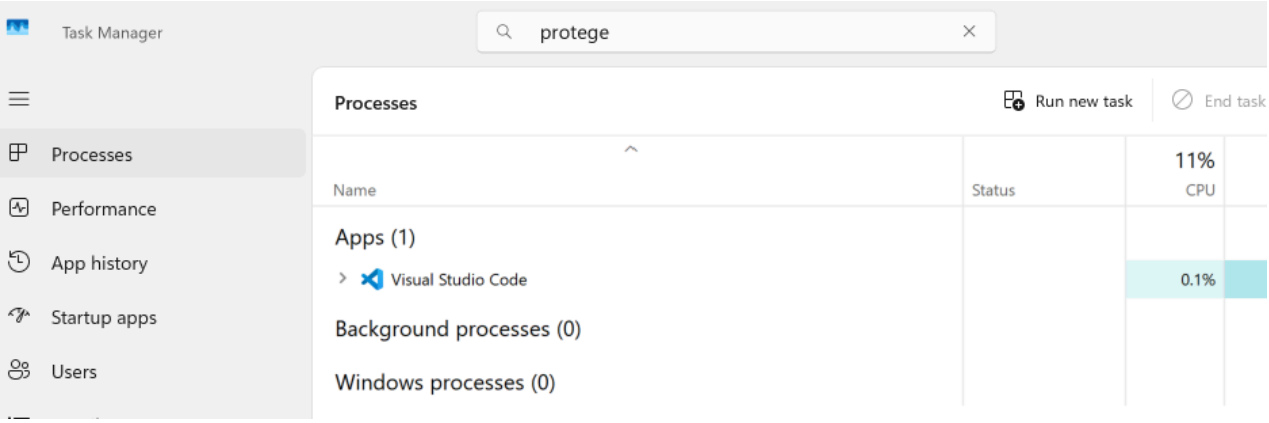
After applying all six corrections, `AmericanaPizza` and `SohoPizza` classify correctly. However, `PlainAmericanaPizza` and `AmericanaHotPizza` do not, despite having logically correct axioms.

Attempt 2.3 -- Investigate the remaining two classes

Two steps:

1. **Workspace hygiene.** Confirm that only the working ontology is open. Confirm no SWRL rules exist. Confirm the reasoner is Pellet or HermiT (both work for this ontology).
2. **Restart and re-synchronize.** Clear the reasoner cache by restarting Protégé.

Below Task Manager in Windows shows no `Protégé` process (except VS Code is used to editing this book) exists:

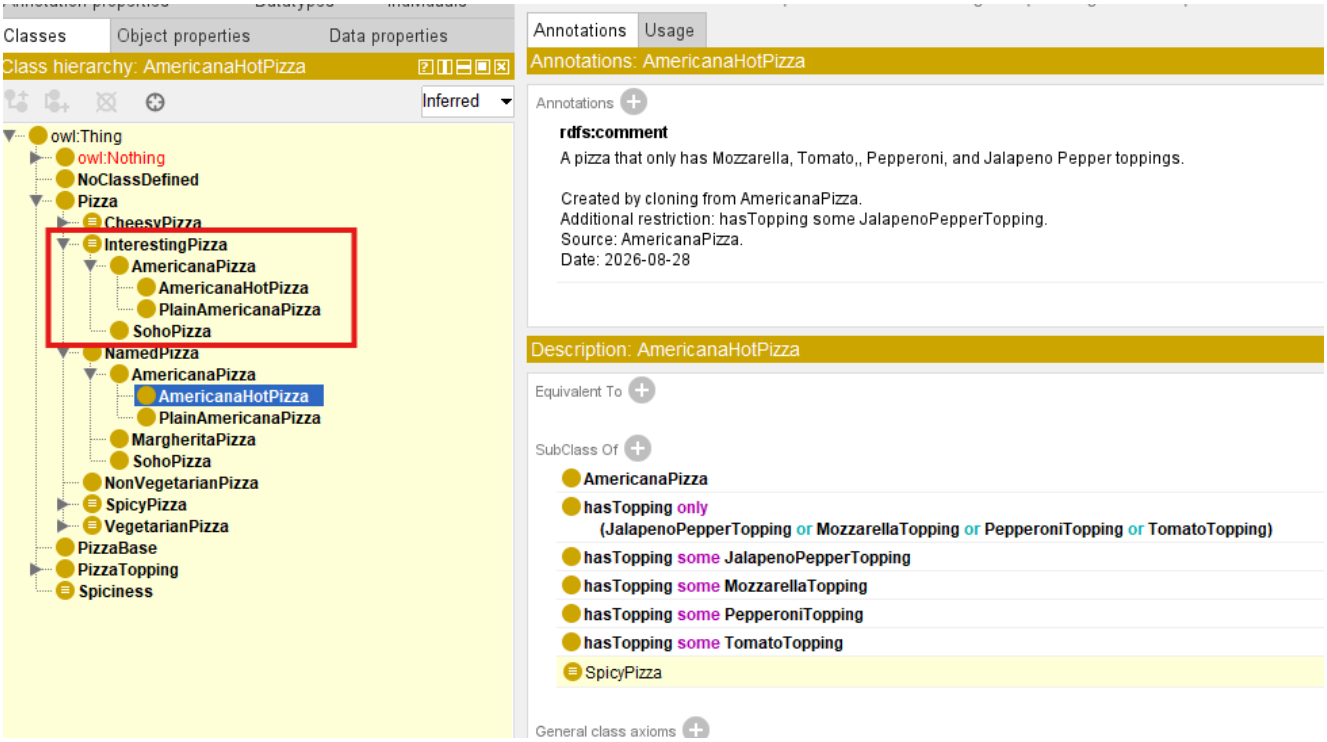


However, after the clean restart both reasoners agree the classes still do not classify.

Attempt 2.4 -- Investigate the apparent non-classification

After applying Corrections 1-6, `AmericanaPizza` and `SohoPizza` appear to classify correctly -- their inferred position in the tree is under `InterestingPizza`. `AmericanaHotPizza` and `PlainAmericanaPizza` appear **not** to classify, because their Description panels do not show `InterestingPizza` in the `SubClass Of` field.

Before assuming a reasoner failure, verify by switching to the **Class hierarchy (inferred)** tab and inspecting the class hierarchy around `InterestingPizza`.



The inferred tree shows `InterestingPizza` with **all four of the target pizzas nested under it**: `AmericanaPizza`, `AmericanaHotPizza`, `PlainAmericanaPizza`, and `SohoPizza`. The reasoner **has** inferred the classification.

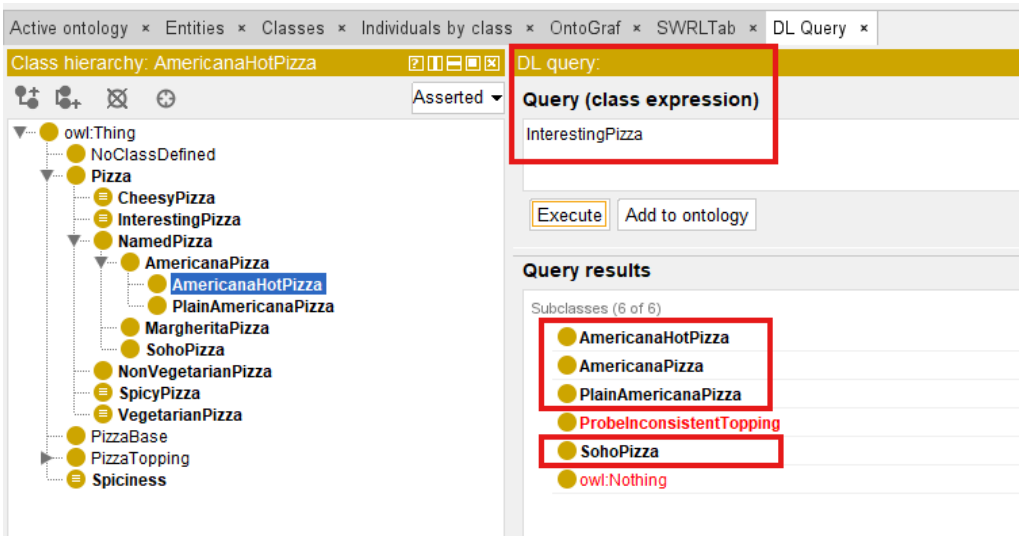
The apparent failure was a Protégé UI display artifact. When the reasoner infers $\mathrm{A} \sqsubseteq \mathrm{C}$ transitively (through $\mathrm{A} \sqsubseteq \mathrm{B} \sqsubseteq \mathrm{C}$), Protégé's **inferred class hierarchy tree** shows A under C, but the **Description panel** for A may not show C in the `SubClass Of` field. This is because the Description panel displays asserted axioms and directly inferred axioms, not all transitive entailments.

To verify an inferred ancestor, use one of:

- The **inferred class hierarchy tree** (authoritative global view).
- The **DL Query** tab -- query for the candidate superclass, check "Subclasses".
- Select the **candidate superclass** in the tree and inspect its inferred children.

Here, from the 2nd verification method:

1. Use the `DL Query` tab
2. Enter the query: `InterestingPizza`
3. Check the `Instances` and `Subclasses` boxes.
4. The result should list the four pizzas.



The full correction set worked. All four pizzas with three or more toppings classify as `InterestingPizza`. `MargheritaPizza` with 2 toppings correctly does not.

The lesson: verify inference in the right view

When working with reasoner-inferred classifications in Protégé, always verify in the **inferred class hierarchy tree**, not in the Description panel. The Description panel tells you what the class *has been asserted to be*. The inferred tree tells you what the reasoner has *derived*. These are different views, and Protégé does not always surface every inference in the Description panel.

> **A Protégé UI behavior**
>
> The Description panel shows the asserted axioms of the selected class and its directly inferred axioms. When a superclass is inferred **transitively** (through another superclass), it may not appear in the Description panel.

Key Insight:

> "The reasoner's output is in the inferred hierarchy tree. The Description panel shows you what you said about a class; the tree shows you what the reasoner derived. When they seem to disagree, trust the tree -- or better, verify in a third view (DL Query) to be sure."

Approach 2 -- Assert the cardinality directly (an alternative)

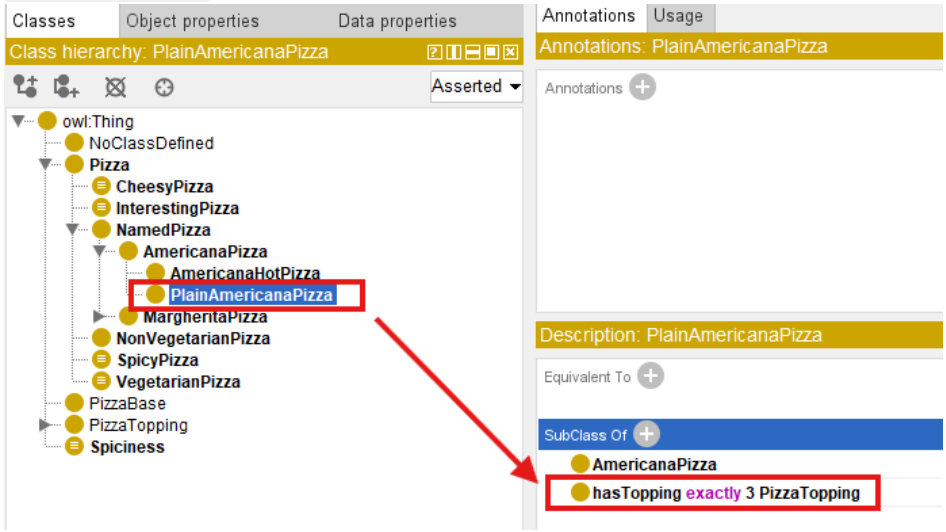
In some cases, the reasoner does not derive the intended classification, even when the axioms are correct. When this happens, one pragmatic alternative is to assert the cardinality directly on the leaf classes:

```
Class: PlainAmericanaPizza
    SubClassOf: hasTopping exactly 3 PizzaTopping
Class: AmericanaHotPizza
    SubClassOf: hasTopping exactly 4 PizzaTopping
```

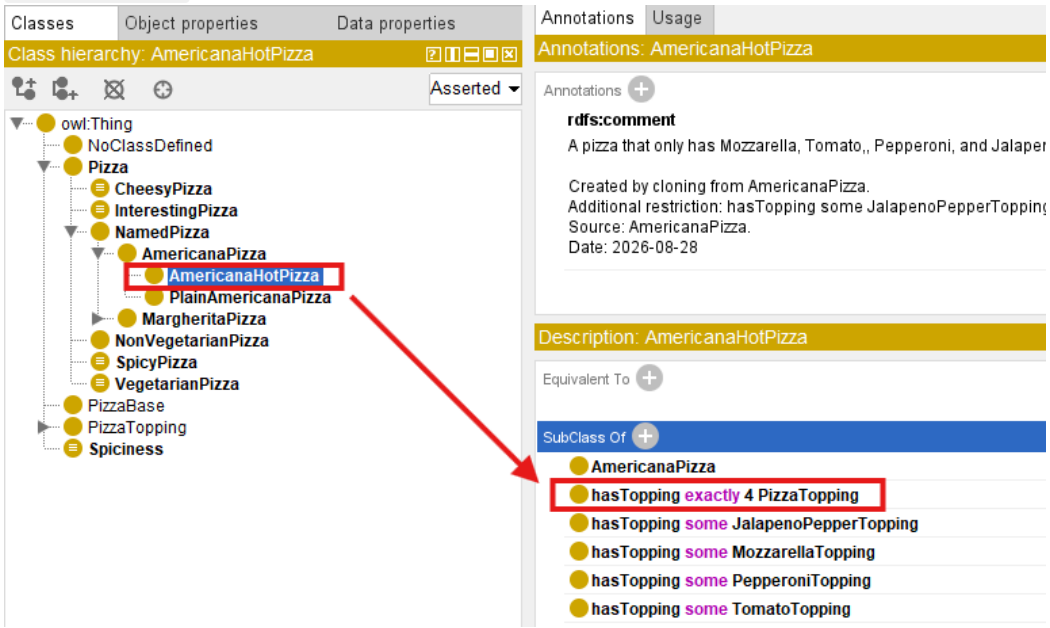
> Start from `pizza-ebook_ex26.rdf` in folder `rdf`, save it to `pizza-ebook_ex26-approach2.rdf` for this approach.

New create `PlainAmericanaPizza` and set as above direct cardinality:

- `PlainAmericanaPizza`



- `AmericanaHotPizza`

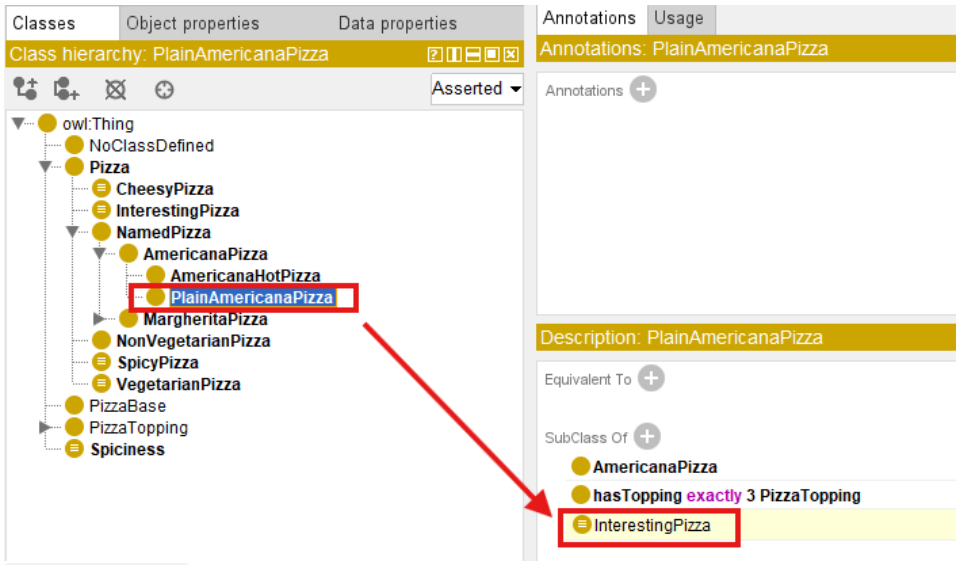


This tells the reasoner the count without requiring it to derive the count from existentials and disjointness. It is more direct but less flexible -- if the pizza's topping set changes, the cardinality axiom must be updated manually.

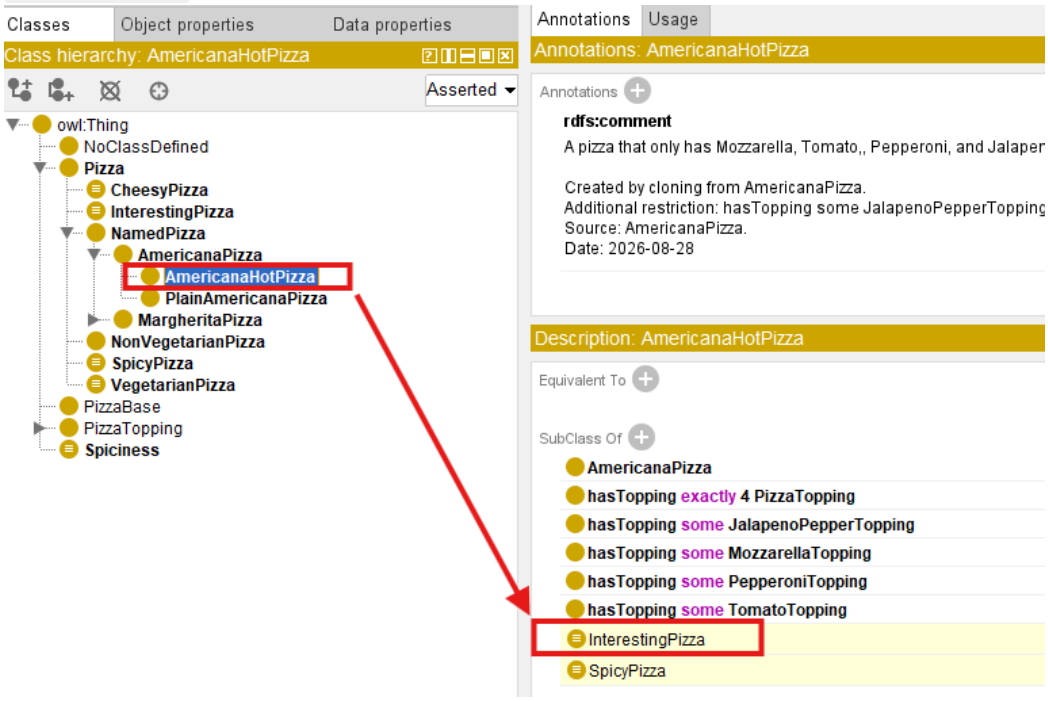
Synchronize reasoner and observe

Both `PlainAmericanaPizza` and `AmericanaHotPizza` are classified directly as subclass of `InterestingPizza`:

- `PlainAmericanaPizza`



- `AmericanaHotPizza`



Which to use. In this exercise, the closure-based approach (Approach 1) succeeded. But in production ontologies, when the reasoner does not fire the intended inference, Approach 2 is a legitimate fallback. Both approaches are used in practice; the choice depends on whether the engineer values derivability (Approach 1) or robustness (Approach 2).

Key Insight:

> "**When the reasoner refuses to derive what the axioms entail**, asserting the intended fact directly is a legitimate engineering fallback. The primitives are the correct pattern; the fallback is the pragmatic response."

26.3.5 Understanding Protégé UI Behavior: Why the Description Panel Can Mislead

Protégé is a powerful environment, but its UI displays information from multiple views that do not always agree. Two behaviors in particular confuse new users and can lead to wrong conclusions about what the reasoner has actually derived. We encountered both during Exercise 26. This section explains them together, because they share a common root.

26.3.5.1 The Mechanics of Conversion: Why "SubClass Of" Becomes Empty

When you select **Edit > Convert to defined class**, a subtle but crucial change occurs in the Protégé interface:

- **What Protégé does.** Converting a class to a defined class automatically *moves* the restrictions or superclasses from the **SubClass Of** section up into the **Equivalent To** section. Consequently, the **SubClass Of** box for that class will often appear completely empty immediately after conversion.
- **Why this happens logically.** A *primitive* class only states necessary conditions (it is a subclass of something). A *defined* class states necessary *and* sufficient conditions (`EquivalentTo`). By shifting `hasTopping min 3 PizzaTopping` into `EquivalentTo` , the reasoner now knows it can both recognize pizzas that fit this rule automatically and ensure full logical equivalence.
- **How to verify.** If you notice your axioms have "disappeared" from *SubClass Of*, check the **Equivalent To** field to confirm they safely relocated there. Once you start the reasoner, any class instance meeting this criteria will correctly and dynamically inherit its classification under the target class.

This behavior is expected and correct. The axioms are not lost; they are reorganized into their logically stronger form.

26.3.5.2 The Inferred-Ancestor Behavior: Why a Subclass May Not Show Its Inferred Parent

A second, more subtle behavior emerges when the reasoner infers a superclass **transitively** — that is, through an intermediate class.

Consider a class `A` with $A \sqsubseteq B$ and $B \sqsubseteq C$. The reasoner derives $A \sqsubseteq C$. In Protégé:

- The **inferred class hierarchy tree** correctly shows `A` nested under `C` (as well as under `B`). This is the authoritative global view of the inferred hierarchy.
- The **Description panel** for `A` may show only $A \sqsubseteq B$, not $A \sqsubseteq C$. The Description panel displays the asserted axioms of the selected class and some inferred axioms — but not necessarily all transitive entailments. Whether $A \sqsubseteq C$ appears depends on the Protégé version and settings.

This behavior can cause a false alarm. During **Exercise 26**, we initially believed `AmericanaHotPizza` and `PlainAmericanaPizza` were not classified as `InterestingPizza` , because their Description panels did not show `InterestingPizza` in the `SubClass Of` field. In fact, the reasoner had correctly inferred the relationship -- the inferred class hierarchy tree showed all four pizzas nested under `InterestingPizza` . The apparent failure was a UI display artifact, not a reasoning failure.

26.3.5.3 The Underlying Principle: Local View vs. Global View

The two behaviors share a root: **Protégé's Description panel is a *local* view; the inferred class hierarchy tree and the DL Query tab are *global* views.**

View	What it shows	When to use it
Description panel	Asserted axioms of the selected class, plus some inferred axioms (version-dependent)	Inspecting what *you* wrote about a class; checking the definition's form
Inferred class hierarchy tree	The reasoner's full subsumption hierarchy — every inferred parent and child relationship	Verifying classifications; inspecting where the reasoner placed a class
DL Query tab	Direct answers to queries about subclasses, superclasses, instances	Confirming a specific inference; getting a clean list of inferred subclasses

When these views seem to disagree, the principle is: **trust the global view.** The inferred class hierarchy tree and the DL Query tab are the reasoner's output. The Description panel is a projection of that output onto the selected class.

26.3.5.4 A Verification Recipe

When you are unsure whether a class has been correctly classified by the reasoner, use this sequence:

1. **Switch to the inferred class hierarchy tree.** Locate the candidate parent class (e.g., `InterestingPizza`) and expand it. The class should appear as a child if it was inferred.
2. **If you need a clean list, use the DL Query tab.** Enter the class expression (e.g., `InterestingPizza`), check the **Subclasses** box, and click **Execute**. The result lists all inferred subclasses.
3. **If you want to see the class's own inferred ancestors, select the *candidate parent* and inspect its inferred children** — or use the DL Query tab with the child class as the query and check **Superclasses**.

A Protégé UI behavior

The Description panel shows a local view of a class. When a superclass is inferred transitively (through another superclass), it may not appear in the Description panel, even though the reasoner has derived the relationship. To see the reasoner's full output for a class, use the inferred class hierarchy tree or the DL Query tab.

Key Insight:

> "Protégé shows you many views of the same ontology. The Description panel tells you what you said about a class. The inferred class hierarchy tree tells you what the reasoner derived. When they seem to disagree, trust the tree — or better, verify in a third view (the DL Query tab). The reasoner's output is in the tree; the local view can mislead."

26.3.5.5 Applying the Lesson

The lesson is not specific to Protégé. Any ontology development environment has this split between local views (what the engineer asserted) and global views (what the reasoner derived). Knowing which view to consult — and knowing when the two can legitimately disagree — is part of the semantic engineering discipline.

In **Exercise 26**, the two views temporarily disagreed. Investigating the discrepancy revealed a real behavior of the reasoner that had been correct all along. Understanding the discrepancy is what closed the exercise — not a change to the ontology, but a change to how we read it.

26.4 The Anatomy of an Executable Ontology

We have now introduced all five execution primitives of Stage 7. In this section we step back and view them as a single architecture -- the **anatomy** of an executable ontology.

The purpose of the anatomy is not to introduce new constructs. It is to show how the five primitives we have built up across **Chapter 24, 25, and 26** fit together, and to name the layers they form.

26.4.1 The Five Primitives

The five execution primitives, and the exercises that introduced them:

#	Primitive	Exercise	Role
1	**Universal Restriction** (`only`)	Ex 22	Defines the boundary: what values are permitted
2	**Closure Axiom**	Ex 23	Closes the boundary: what values exhaustively exist
3	**Enumerated Class**	Ex 24	Closes the value domain: the finite set of possible values
4	`hasValue` **Restriction**	Ex 25	Binds the value: the specific value to act on
5	**Cardinality Restriction**	Ex 26	Enforces completeness: how many values are required

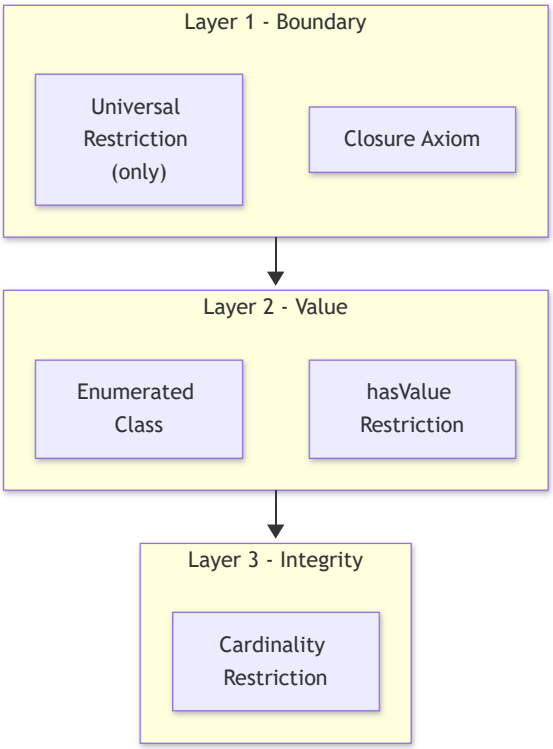
Read down the list, and Stage 7 reads as a single sentence:

> *First define what is permitted. Then close the boundary. Then close the value domain. Then bind the specific value. Then guarantee the count.*

That sentence is the anatomy of an executable ontology.

26.4.2 The Three Layers

The five primitives organize into three layers, each building on the one before it:



Layer 1 -- Boundary (Chapter 24). The universal restriction and the closure axiom define *what* values are permitted and *what* values exhaustively exist. This layer answers the question: *what is the world?*

Layer 2 -- Value (Chapter 25). The enumerated class and the `hasValue` restriction close the *set of possible values* and bind a *specific value* to be acted on. This layer answers: *what value fires the trigger?*

Layer 3 -- Integrity (Chapter 26, this Chapter). The cardinality restriction guarantees that the required *number* of values is present. This layer answers: *how much data do I need before I can act?*

Each layer depends on the one before it:

- **Without the boundary layer,** the value layer has no closed domain to draw from. A `hasValue` restriction that binds a property to a specific value is only meaningful if the value's type is exhaustively known.
- **Without the value layer,** the integrity layer has no specific value to count. A cardinality restriction that guarantees "at least three toppings" is only meaningful if the *kinds* of toppings that count are already closed.
- **Without the integrity layer,** the boundary and value layers are correct but incomplete: the ontology knows what is permitted and which value to act on, but cannot guarantee the data required to act is present.

The layering is not arbitrary. It reflects the logical dependency between the primitives. Boundary closure enables value binding. Value binding enables count. Count completes the executable specification.

Key Insight:

> "Executable knowledge is not a single axiom. It is three layers, built in order, each one making the next possible. The boundary layer says what is closed. The value layer says what to act on. The integrity layer says when to act."

26.4.3 The Anatomy in the EKA Framework

In the EKA framework (introduced in earlier chapters), Stage 7 of Semantic Knowledge Development Lifecycle (SKDL) establishes the Θ **(Trigger)** and Φ **(Execution)** layers. The anatomy of the five primitives maps directly onto these layers.

Θ **(Trigger) -- what fires.** The Θ layer is the set of semantic conditions that fire a trigger. It is built from:

- Universal restrictions (what is permitted)
- Closure axioms (what exhaustively exists)
- Enumerated classes (what value domain is bounded)

- `hasValue` restrictions (what specific value fires)

The Θ layer is not a single axiom. It is the *conjunction* of boundary and value axioms that together define a class whose instances fire a trigger. When `PlainAmericanaPizza` is classified as an `InterestingPizza`, the Θ layer has fired.

Φ (Execution) -- what executes. The Φ layer is the mechanism that, once the trigger has fired, executes the corresponding action. In the ontology itself, this appears as the cardinality restriction, which guarantees that the data required for execution is complete. Beyond the ontology, Φ is the runtime system that reads the classified instance and takes the corresponding action.

The integrity layer's contribution to Φ is subtle but essential. Without it, the trigger might fire on incomplete data. With it, the trigger fires only when the data is sufficient. The cardinality restriction is what *binds* the trigger to the execution: it says "this condition fires only when the data is complete enough to act on."

Key Insight:

> "The boundary and value layers are the semantic side of the trigger. The integrity layer is the operational side. Together, they establish the Θ-Trigger and Φ-Execution layers that close the loop between reasoning and action."

26.4.4 A Minimal Worked Example

Let us see the anatomy in action, using the `Pizza` ontology.

Boundary (Exercise 22, Exercise 23).

- `VegetarianPizza` $\equiv$ `Pizza and (hasTopping only (CheeseTopping or VegetableTopping))`. The universal restriction says "every topping must be cheese or vegetable."
- The closure axiom on `MargheritaPizza` -- `SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping)` -- says "exactly mozzarella and tomato are the toppings."

Together, they establish that anything called a `VegetarianPizza` has only vegetarian toppings -- and that `MargheritaPizza` and `SohoPizza` are provably vegetarian.

Value (Exercise 24, Exercise 25).

- `Spiciness` $\equiv$ `{Hot, Medium, Mild}` closes the spiciness domain.
- `JalapenoPepperTopping` $\sqsubseteq$ `hasSpiciness value Hot` binds a specific topping to a specific value.

`SpicyPizza` $\equiv$ `Pizza and (hasTopping some (hasSpiciness value Hot))` fires when a pizza has at least one hot topping.

Integrity (Exercise 26).

- `InterestingPizza` $\equiv$ `Pizza and (hasTopping min 3 PizzaTopping)` fires when a pizza has at least three toppings.

The cardinality restriction guarantees that the count is provable -- once the boundary and value layers have made distinctness provable.

Key Insight:

> "Compose the three layers, and the ontology can answer: *'Is this pizza both spicy and interesting?'* That is an actionable query. The answer is derivable. It is not intuition -- it is a decision."

26.4.5 What the Anatomy Enables

A **descriptive ontology** tells us what is true. An **executable ontology** tells us what to *do*.

The anatomy enables a small but profound shift:

- **Before Stage 7**, the ontology could classify `AmericanaHotPizza` as `SpicyPizza`. But that classification did not trigger anything -- it was a static fact in a library.
- **After Stage 7**, the ontology can classify `AmericanaHotPizza` as `SpicyPizza` *and* as `InterestingPizza`, *and* guarantee that the count and value conditions are met. The classification is now a dynamic *decision*, not a fact.

The anatomy is what makes this shift possible. Without the boundary layer, classification is open-ended. Without the value layer, classification has no specific trigger. Without the integrity layer, the trigger fires on incomplete data. All three together make the ontology

executable.

Key Insight:

> "A descriptive ontology knows. An executable ontology decides. The anatomy of the five primitives is what turns knowledge into decision."

26.4.6 From Anatomy to Perspective

The anatomy gives us the *structure* of an executable ontology. But it does not, by itself, explain *why* this structure works. That requires four complementary perspectives:

1. **Mathematical** (§26.5) -- the logical foundations of closure, enumeration, binding, and cardinality.
2. **Historical** (§26.6) -- how executability evolved from expert systems to modern rule engines.
3. **Industrial** (§26.7) -- how executable ontologies are applied in manufacturing, compliance, and process automation.
4. **EKA** (§26.8) -- how the five primitives map to the Θ-Trigger and Φ-Execution layers of the Executable Knowledge Architecture.

The following four sections develop each perspective in turn. Together with the anatomy, they give you a complete picture of Stage 7 -- not just what the primitives are, but why they matter.

We begin with the mathematical perspective -- the logical foundation on which everything else rests.

26.5 Mathematical Perspective

26.5.1 OWL as Set Theory

Before examining each primitive, we need a shared mental model: **OWL is set theory.**

- A **class** is a set of individuals. `Pizza` is the set of all pizzas.
- An **individual** is an element of a set. `MargheritaPizza1` is an element of `MargheritaPizza`.
- A **(object) property** is a binary relation. `hasTopping` relates pizzas to toppings.
- A **restriction** is a set defined by a condition on the property's value.
- An **axiom** is a constraint on the sets: "the set `VegetarianPizza` is a subset of `Pizza`" is a constraint.

This is not an analogy. It is the formal semantics of OWL. Every OWL axiom has a precise set-theoretic interpretation, and the reasoner computes the logical consequences of those set-theoretic constraints.

With this model in mind, each of the five primitives becomes a specific *set operation* directly.

26.5.2 Closure as Set Restriction

The closure axiom:

```
Class: MargheritaPizza
   SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping)
```

is a **set restriction** on the property's range.

Formally, `hasTopping only X` says:

> For every individual `p` in the class, every value `t` related to `p` via `hasTopping` is an element of `X`.

In set-theoretic notation:

$$\forall p \in \text{MargheritaPizza} : \text{hasTopping}(p) \subseteq \text{MozzarellaTopping} \cup \text{TomatoTopping}$$

This is a **universal restriction**. It constrains the *range* of `hasTopping` for every instance of the class. It does not say anything about how many value exist -- only that whatever exists must be in the specified set.

What closure does mathematically?

It transforms an *open* set of possible values into a *closed* set. Before the closure, `hasTopping(MargheritaPizza1)` could contain any topping; after the closure, it can only contain mozzarella and tomato. The set of possible values is bounded.

What closure does not do?

It does not tell the reasoner *how many* values exist. The closure of `hasTopping(MargheritaPizza1)` to `{mozzarella, tomato}` is compatible with zero values (an empty subset), one value, or two values. It does not force the existence of any particular value.

> This is why the closure axiom alone is not sufficient for cardinality-based classification. The closure restricts the *range*, but the cardinality requires the *count*. The two are orthogonal dimensions of the property's extension.

26.5.3 Enumeration as Finite Extension

The enumerated class:

```
Class: Spiciness
   EquivalentTo: {Hot, Medium, Mild}
```

defines a class whose extension is a **finite set**.

Formally:

$$\text{Spiciness} = \{\text{Hot}, \text{Medium}, \text{Mild}\}$$

This is a *definition by listing*. It says: the class `Spiciness` contains exactly these three individuals, and no others.

What enumeration does mathematically?

It converts an *open class* (whose membership is unknown) into a *closed set* (whose membership is fully specified). Before the enumeration, the reasoner knows `Spiciness` is a class; after the enumeration, it knows the exact extension.

Why enumeration is stronger than asserted membership?

If we simply asserted that `Hot`, `Medium`, and `Mild` are instances of `Spiciness`, the reasoner would know they are *members*. But it would not know they are the *only members* -- under OWA, there could be other instances not yet asserted.

The enumeration axiom $\text{Spiciness} \equiv \{\text{Hot}, \text{Medium}, \text{Mild}\}$ closes this gap. It says: for any individual `x`, $\text{x} \in \text{Spiciness}$ if and only if $\text{x} \in \{\text{Hot}, \text{Medium}, \text{Mild}\}$. This is an *equivalence*, not just an implication. It excludes any individual outside the enumerated set.

Mathematical consequence:

Enumeration makes the class extension **decidable**. Given any individual `x`, the reasoner can determine in constant time whether $\text{x} \in \text{Spiciness}$: check whether `x` is one of the three enumerated individuals. Without enumeration, membership in `Spiciness` might require reasoning over other axioms.

Why this matters for execution?

An executable rule that fires on "any spiciness" can now enumerate all possible cases (`Hot`, `Medium`, `Mild`). The rule is guaranteed to be *exhaustive* -- every value it might encounter has been enumerated. This is a prerequisite for deterministic execution.

26.5.4 `hasValue` as Singleton Binding

The `hasValue` restriction:

```
Class: JalapenoPepperTopping
   SubClassOf: hasSpiciness value Hot
```

binds the property `hasSpiciness` to the specific individual `Hot`.

Formally:

$$\forall j \in \text{JalapenoPepperTopping} : \text{hasSpiciness}(j) = \{\text{Hot}\}$$

This is a **singleton binding** -- a restriction of the property's value to a set with exactly one element.

What `hasValue` does mathematically?

It fixes the property's value to a specific individual. Where the closure axiom restricted the *range* to a set, and enumeration fixed a *class's extension*, `hasValue` fixes a *property's value* to a singleton.

Why `hasValue` is stronger than `only`?

The restriction `hasSpiciness only Spiciness` would say "any spiciness value must be a Spiciness." But that's compatible with $\text{hasSpiciness}(\text{j}) = \emptyset$ (no value). The `hasValue` restriction says the value is `Hot` -- exactly that one individual, no ambiguity.

Formally, `p value v` is equivalent to `p only {v}` combined with `p some {v}` -- it requires both the range restriction and the existence.

Mathematical consequence:

`hasValue` makes the property's value **deterministic**. Given an individual $\text{j} \in \text{JalapenoPepperTopping}$, its `hasSpiciness` value is fully determined: it is `Hot`. This is what enables the trigger semantics.

Why this matters for execution?

A trigger that fires on "spiciness is Hot" requires not just that the property *could* be Hot, but that it *is* Hot. The `hasValue` restriction provides that certainty.

26.5.5 Cardinality as Bounded Extension

The cardinality restriction:

```
Class: InterestingPizza
    EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
```

bounds the **size** of the property's extension.

Formally:

$$\forall i \in \text{InterestingPizza} : |\text{hasTopping}(i) \cap \text{PizzaTopping}| \geq 3$$

This says: for every instance of `InterestingPizza`, the set of its `PizzaTopping`-typed toppings has at least three elements.

What cardinality does mathematically?

It constrains the *number* of values a property takes. The three forms are:

- `min n`: `|hasTopping(i)|` $\geq$ `n` (at least `n`)
- `max n`: `|hasTopping(i)|` $\leq$ `n` (at most `n`)
- `exactly n`: `|hasTopping(i)| = n` (exactly `n`)

The qualified form.

The restriction we wrote uses `min 3 PizzaTopping` -- a *qualified* cardinality. It says "at least three toppings *of type* `PizzaTopping`." Because `hasTopping` has range `PizzaTopping`, every topping counts, and the qualification is trivially satisfied. But in general, the qualified form is more expressive -- it allows counting a subset of the property's values.

The distinctness requirement.

Counting is only meaningful if the elements being counted are **distinct**. The reasoner must be able to prove that the values of `hasTopping(i)` are pairwise different. This is what makes disjointness axioms (which we examined in §26.3.4) essential for cardinality: without them, the reasoner cannot prove that the toppings being counted are actually distinct individuals.

Mathematical consequence.

Cardinality requires that the ontology be able to prove **distinctness** of the values. This is where the set theory becomes subtle. Under the Open World Assumption, existential restrictions prove existence but not distinctness. Without additional axioms (disjointness, unique names,

or explicit `differentFrom`), the reasoner cannot count.

This is precisely the lesson of **Exercise 26**: closure alone (which restricts *what*) is not sufficient for cardinality (which counts *how many*). Distinctness is what bridges the two.

26.5.6 The Mathematical Arc of Stage 7

Read across the five primitives as set operations, and Stage 7 becomes a single mathematical progression:

Primitive	Set Operation	Effect
Universal Restriction (`only`)	Subset restriction	Constrains the *range* of a property
Closure Axiom	Complete subset restriction	Closes the *boundary* -- no values outside the set
Enumerated Class	Finite extension definition	Fixes a class's extension to a *finite set*
`hasValue` Restriction	Singleton binding	Fixes a property's value to a *single individual*
Cardinality Restriction	Bounded extension	Constrains the *size* of a property's extension

Each operation constrains a different dimension of the same underlying structure:

- **Range**: what values are permitted (`only`, closure).
- **Extension**: what values a class contains (enumeration).
- **Binding**: what specific value a property takes (`hasValue`).
- **Count**: how many values a property has (cardinality).

Together, the four dimensions fully specify the executable conditions of a class.

The mathematical foundation of the anatomy. The three-layer structure of §26.4 is not arbitrary. It reflects the *logical dependency* between the operations:

1. **Boundary closure** (Layer 1) must precede value binding, because binding a value requires knowing what values are permitted.
2. **Value binding** (Layer 2) must precede count, because counting a property requires knowing which specific values are being counted.
3. **Cardinality** (Layer 3) closes the specification, because the count guarantees that the data required for the trigger is present.

The mathematics mirrors the engineering:

> each layer is a set operation, and the operations compose in a specific order.

26.5.7 Distinctness -- The Underlying Condition

There is one mathematical condition that spans all five primitives, and it deserves explicit treatment: **distinctness.**

A set has distinct elements. An individual is not two individuals. But under the Open World Assumption, the reasoner does not automatically know that two named individuals are different -- **it must be told**.

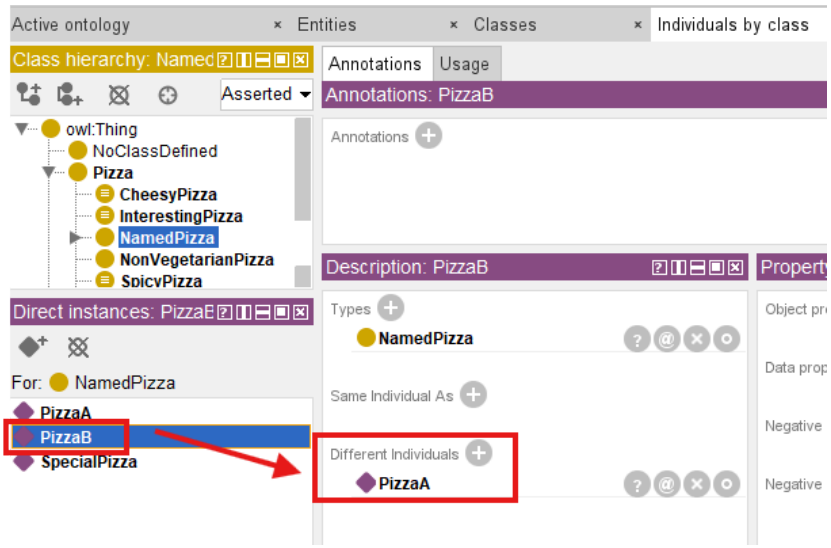
Distinctness enters the picture at multiple points:

- **Enumeration**: the enumeration `Spiciness` $\equiv$ `{Hot, Medium, Mild}` requires that `Hot`, `Medium`, and `Mild` are three distinct individuals. If they were the same, the enumeration would contain one element, not three.
- **`hasValue`**: the `hasValue` restriction binds a property to `Hot`. If `Hot` were the same individual as `Medium`, the binding would be to the same individual as if we had bound to `Medium`.
- **Cardinality**: the count "at least 3 toppings" requires that the toppings being counted are distinct. Without distinctness, three toppings could be one topping counted three times.

How distinctness is established?

There are three ways to prove distinctness in OWL:

1. **Unique Name Assumption (UNA)**: if UNA holds, every named individual is distinct. But OWL does not assume UNA.
2. **Explicit `owl:differentFrom`**: you can assert that two individuals are different



3. **Disjointness of classes**: if $A \perp B$, and $x \in A$, $y \in B$, then $x \neq y$. This is the form of distinctness we used in **Exercise 26**.

Here is the `owl:differentFrom` in the ontology's XML:

```
<owl:NamedIndividual rdf:about="http://www.semanticweb.org/xiaoqi/ontologies/2026/4/pizza-ebook#PizzaA">
    <rdf:type rdf:resource="http://www.semanticweb.org/xiaoqi/ontologies/2026/4/pizza-ebook#NamedPizza"/>
    <owl:differentFrom rdf:resource="http://www.semanticweb.org/xiaoqi/ontologies/2026/4/pizza-ebook#PizzaB"/>
    <pizza-ebook:hasTopping rdf:resource="http://www.semanticweb.org/xiaoqi/ontologies/2026/4/pizza-ebook#MozzarellaTopping"/>
</owl:NamedIndividual>
```

The disjointness approach is the most powerful for cardinality-based reasoning. By declaring $\text{MozzarellaTopping} \perp \text{PepperoniTopping}$, $\text{MozzarellaTopping} \perp \text{TomatoTopping}$, and $\text{PepperoniTopping} \perp \text{TomatoTopping}$, we make the reasoner able to prove that the three toppings in `AmericanaPizza` are distinct. Without those disjointness declarations, the reasoner would see three existential restrictions but would not be able to count them.

Why distinctness is the mathematical heart of Stage 7?

Every execution primitive depends on being able to prove distinctness -- because every primitive is a set operation, and sets contain *distinct* elements. Distinctness is what makes **set operations** well-defined.

This is why the closure and cardinality layers of the anatomy work together, and why the **Stage 1** taxonomy matters. The taxonomy declares the disjointness that makes distinctness provable. The closure and cardinality declare the set operations that *use* distinctness. The **Stage 7** primitives are not independent of **Stage 1** -- they are set operations *on the sets that **Stage 1** defines*.

Key Insight:

> "Every execution primitive is a set operation. Every set operation requires distinctness. Distinctness is what makes the sets well-defined. In an ontology, distinctness is what the taxonomy provides --and Stage 7 is what puts it to use."

26.5.8 What the Mathematics Teaches

The set-theoretic view of **Stage 7** is not just academic, it changes how you approach ontology engineering:

1. **Debugging is easier with the mathematical model**: when a classification does not fire, the first question is: *which set operation failed?* Is the range not closed (missing closure)? Is the extension not determined (missing enumeration)? Is the value not bound(missing `hasValue`)? Is the count not provable (missing cardinality or distinctness)? The mathematical model gives you a systematic checklist.
2. **Composition is compositional**: each primitive is a separate set operation. They compose without interfering -- but only if their *order* respects the dependency. Closure before binding, binding before count. This is why the anatomy's three layers must be built in order.
3. **Distinctness is the hidden requirement**: the most common failure mode in **Stage 7** is a cardinality that doesn't fire because distinctness is not provable. When this happens, go back to **Stage 1** and check the taxonomy -- it is almost always the source of the missing disjointness.

4. **The distinctions between primitives are subtle but real**: `only` restricts the range. `value` binds the value. `some` asserts existence. `min` / `max` / `exactly` bound the count. These are four *different* set operations, and confusing them is the source of many modeling errors.

Key Insight:

> "OWL is set theory applied with engineering discipline. Every primitive restricts a set in a specific way. Understanding which set is being restricted, and how, is the difference between an ontology that works and one that merely looks correct."

26.5.9 The Mathematical Arc Across Stage 7

Read across the three layers of the anatomy (§26.4.2), the mathematics tells a complete story:

Layer 1 -- Boundary. The set-restriction operations (`only` , closure) constrain *what* values are permitted. They establish the *domain* of the property's extension.

Layer 2 -- Value. The extension and binding operations (enumeration, `hasValue`) fix *which* values the property takes. They establish the *range* of the extension to a specific individual.

Layer 3 -- Integrity. The count operation (cardinality) constrains *how many* values the property has. It establishes the *size* of the extension.

Each layer operates on the same underlying set -- the property's extension -- but constrains a different dimension of it. Together, they fully specify the property: its domain, its range, and its size.

The mathematics is what makes this composition rigorous. Without the set-theoretic view, the three layers look like three unrelated modeling patterns. With the set-theoretic view, they are three operations on the same mathematical object, and their composition is what makes the ontology executable.

This is the mathematical foundation of **Stage 7**. The remaining perspectives -- historical, industrial, and EKA -- build on this foundation. Each of them shows how the primitives have been used, in different eras and different domains, to solve the same class of problem: **how to encode knowledge with sufficient precision to act on it.**

Key Insight:

> "The five primitives are not five separate tools. They are five set operations on the same mathematical object -- the property's extension. Composed in the right order, they fully specify what it means to execute."

26.6 Historical Perspective

The idea that knowledge should be *executable* is not new. It has two lineages, and **Stage 7** sits at their meeting point.

- The first lineage comes from **artificial intelligence** -- from the early production systems of the 1960s through expert systems in the 1980s to the rule engines and decision management systems of today. It answers the question: *how do we encode knowledge so that a machine can act on it?*
- The second lineage comes from **industrial engineering** -- from Taylor and Ford through the formal process standards of the twentieth century to the BPMN standard and Model-Driven Engineering. It answers the question: *how do we encode work so that it can be executed reliably at scale?*

These two lineages are often treated separately. AI is taught in computer science departments; process engineering is taught in industrial engineering departments. The split is artificial. In an executable ontology, both lineages are present simultaneously: the ontology encodes the *semantic conditions* that fire a trigger (the AI contribution) and, when it fires, drives the *workflow* that executes the corresponding action (the industrial contribution).

This section traces both lineages and shows where they coverage. Understanding the historical foundations makes the design patterns of **Stage 7** legible:

> they are not new inventions but the natural evolution of two disciplines that have been solved the same class of problem for different reasons.

26.6.1 The AI Lineage -- From Production Rules to Rule Engines

The first lineage begins with **production systems** in the 1960s and 1970s. The name is a bit of a misnomer: a "production system" is not a manufacturing system; it is a *rule-based* system. The term comes from the Chomskyan tradition in linguistics, where a "production" is a rewrite rule. Early AI researchers adapted the term to describe a system where knowledge is encoded as *condition-action pairs* -- "if condition X holds, then take action Y" -- and an *inference engine* fires rules wherever their conditions are met.

The canonical production systems of the era were:

- **PSG** (Post Production System) -- Emil Post's foundational formalism, 1943. A finite set of rewrite rules applied to strings.
- **AM** (Automated Mathematician) -- Douglas Lenat's 1976 system that discovered mathematical conjectures by applying heuristic rules.
- **OPS5** -- Charles Forgy's 1977 production system, the first to use the **Rete algorithm** for efficient rule firing. OPS5 became the foundation of nearly every commercial rule engine that followed.

The 1980s brought **expert systems** -- production systems that encoded domain-specific knowledge from human experts. Expert systems were the first successful commercial AI applications, and they established the principle that a system could *reason* about a domain if its knowledge were encoded explicitly enough:

- **MYCIN** (Stanford, 1970s) -- Diagnosed bacterial infections and recommended antibiotics. Its knowledge base contained ~450 rules of the form "If the patient's infection is X and their immune status is Y, then recommend antibiotic Z."
- **DENDRAL** (Stanford, 1965-1980) -- Identified organic molecules from mass spectra. Its knowledge base encoded the rules of mass spectrometry.
- **XCON** (DEC, 1980) -- Configured VAX computer systems for customer orders. At its peak, XCON was saving DEC tens of millions of dollars annually and was the first AI system to be widely credited with significant commercial impact.
- **CADUCEUS** (University of Pittsburgh, 1980s) -- Diagnosed internal medicine conditions from patient symptoms. At its peak, its knowledge base contained ~10,000 rules.

What did these systems have in common?

Each encoded domain knowledge as a **set of rules** -- condition-action pairs -- and used an **inference engine** to fire the rules when their conditions matched. The knowledge was *explicit* in the rules, and the runtime was *general* in the engine. The separation between knowledge and inference was the central design principle.

Expert systems fell out of fashion in the 1990s for engineering reasons. They were:

- **Brittle.** A rule that fired correctly in one context could fire incorrectly in another, and there was no way to express the boundary.
- **Hard to maintain.** Adding new rules could break existing rules; the knowledge base grew faster than the maintenance capacity.
- **Non-learning.** The system could not improve from data; every new case required a new rule.
- **Narrow.** Expert systems worked well on narrow, well-defined problems but failed on the messy, open-ended problems that characterize real enterprise domains.

Yet the core idea survived. It reappeared in several forms:

- **Business rule engines** -- Drools, FICO Blaze, IBM Operational Decision Manager. These are the direct descendants of OPS5. They are widely used in financial services, insurance, and retail for pricing, eligibility, and fraud detection.
- **Semantic Web Rule Language (SWRL)** -- Introduced by the W3C in 2004 as a rule language for OWL ontologies. SWRL adds rule-based reasoning to OWL DL, allowing the ontology to express inferences that DL alone cannot capture.
- **SHACL** -- Introduced by the W3C in 2017 as a constraint language for RDF. SHACL is not a rule language in the traditional sense, but it plays a similar role: it declares conditions that data must satisfy, and a validator fires on violations.
- **Decision Model and Notation (DMN)** -- Published by the OMG in 2015 as a standard for business decision modeling. DMN explicitly separates the decision's *logic* (a decision table) from the decision's *execution* (a runtime engine), which is a direct philosophical descendant of the expert-system separation.

Stage 7 is the semantic-web continuation of this lineage. Where expert systems encoded rules as ad hoc syntax, OWL encodes them as **logical axioms**. Where expert systems needed a specific engine per system, OWL axioms are executed by any conformant reasoner. Where expert systems were brittle because they had no way to express boundary conditions, OWL offers the universal restriction and the closure axiom precisely to express those boundaries.

The contribution of the AI lineage to Stage 7.

The AI lineage gives us three things:

1. **The idea of explicit knowledge as the basis for action.** The rule-based tradition established that a system can act on knowledge only if that knowledge is explicitly encoded.
2. **The separation of knowledge and inference.** The knowledge base contains the rules; the inference engine fires them. **Stage 7** presents this separation: the ontology contains the axioms; the reasoner derives the classifications.
3. **The recognition of failure modes.** Expert systems' brittleness and maintenance burden are the historical lessons that **Stage 7** addresses through *closure*, *disjointness*, and *cardinality*.

Key Insight:

> "The AI lineage gave us the trigger. From production rules through expert systems to modern rule engines, the discipline has been the same: encode knowledge explicitly, and let a general engine fire it. **Stage 7** is the semantic-web continuation of this discipline."

26.6.2 The Industrial Lineage -- From Taylor to BPMN to Model-Driven Engineering

The second lineage begins with the **First and Second Industrial Revolutions**.

Frederick Winslow Taylor published *The Principles of Scientific Management* in 1911. Taylor's argument was simple but revolutionary: complex work could be *rationalized* -- broken into discrete, standardized, measurable steps -- and the rationalized process could be executed much more efficiently than artisanal craft. Taylor's time-and-motion studies at Bethlehem Steel showed that reorganizing the work of loading pig iron onto rail cars could raise output per worker by nearly 400%.

Henry Ford applied Taylor's principles to the assembly line in 1913. The moving assembly line at the Highland Park plant reduced the time required to build a Model T from twelve hours to ninety-three minutes. Ford's contribution was not a new theory but a new application: the process was rationalized, the steps were standardized, and the sequence was enforced by physical constraints (the conveyor).

Taylor and Ford established what the twentieth century would call **mass production**: the ability to manufacture complex products in large volumes with consistent quality and predictable cost. The core asset was not the machinery; it was the **process definition and orchestration** -- the explicit, standardized, measurable sequence of steps.

The industrial lineage continued through the twentieth century with several waves:

- **Statistical Process Control** -- Walter Shewhart (1930s) and W. Edwards Deming (1950s) introduced the idea that process *variation* could be measured and controlled. The control chart became the standard tool for detecting when a process was drifting out of specification.
- **Lean Manufacturing** -- Toyota's production system (Taiichi Ohno, 1950s-1970s) introduced the idea of *continuous improvement*: process execution should generate data, and that data should drive the next iteration of process design.
- **Six Sigma** -- Motorola (1980s) and GE (1990s) formalized the idea of *quality as a process metric*. The DMAIC cycle (Define, Measure, Analyze, Improve, Control) is a closed-loop process of executing, measuring, and improving.
- **Business Process Re-engineering** -- Michael Hammer and James Champy (1993) argued that process should not just be improved but *redesigned*. The insight was that some processes were fundamentally wrong and could not be fixed by incremental improvement.

The unifying theme of all these waves is the same: **process execution is only as good as the process definition**. A statistical process control chart requires a defined process; a Lean improvement cycle requires a defined process; a Six Sigma measurement requires a defined process. The process model is the *specification* against which execution is measured.

In 2011, the Object Management Group published **BPMN 2.0** (Business Process Model and Notation). BPMN 2.0 was not the first process notation -- it had predecessors in flowcharting (1920s), IDEF (1980s), and ARIS (1990s) -- but it was the first to be designed with **formal execution semantics** from the start.

The critical distinction is this: a flowchart is a (static) picture of a process. A BPMN 2.0 model is a *formal specification* that can be compiled to an executable workflow. Every BPMN construct -- sequence flow, parallel gateway, exclusive gateway, boundary event, compensation event -- has a defined mapping to an execution semantics. A BPMN-compliant workflow engine can read a BPMN 2.0 model and execute it without additional interpretation.

This is the industrial equivalent of what **Stage 7** does for ontologies. Where OWL axioms are *executed* by a reasoner, BPMN models are *executed* by a workflow engine. In both cases, the formal semantics are what make execution possible.

Dr. Nicolas Figay (2026) has recently argued that the industrial heritage of business process modeling is often forgotten, and that this is a mistake.[1] In an era of cloud-native software and knowledge work, formal process modeling is sometimes dismissed as bureaucracy. But Figay points out that this view mistakes *process execution* for *process documentation*. Without an explicit process architecture, digital transformation degenerates into what he calls "digitized chaos" -- duplicated efforts, unintegrated software silos, and operational fragility.

Figay's argument for BPMN 2.0 is directly parallel to this book's argument for OWL:

> "BPMN 2.0 provides a rigorous mathematical and logical grammar designed specifically to rationalize work. Grounded in formal execution semantics and workflow patterns, BPMN allows system architects to model task dependencies, concurrency (gateways), exception handling, and resource assignments without ambiguity."

Replace "BPMN 2,.0" with "OWL" and "Process Model" with "Ontology", and the argument is identical. Both formalisms exist because *formal semantics enables engineering discipline*. A process description without formal semantics is a **picture**; a process model with formal semantics is a **specification**. An ontology without formal semantics is a **vocabulary**; an ontology with formal semantics is a **reasoner-executable specification**.

The industrial lineage then extends into **Model-Driven Engineering (MDE)**, the discipline of treating formal models as *primary engineering artifacts*. In MDE, a BPMN model is not a post-hoc illustration of an already-built workflow; it is the artifact from which the workflow is generated, validated, tested, and orchestrated. The model is linked to the ontology (for semantic validation), the data schemas (for structure), and the system architecture (for deployment). The traceability is *preserved* from business goal to ground-level execution.

This is the industrial counterpart to the semantic-web stack.

Where SKDL treats the ontology as the primary artifact and the reasoner as its executor, MDE treats the process model as the primary artifact and the workflow engine as its executor. Both discipline share the same core insight: **the formal model is not documentation; it is the executable specification.**

The contribution of the industrial lineage to Stage 7.

The industrial lineage gives us three things:

1. **The idea of process as a specification, not a description.** The industrial engineers established that a process model must be explicit, standardized, and measurable if it is to be executed reliably at scale.
2. **The formal semantics of execution.** BPMN 2.0 was the first process notation designed with execution semantics from the start. It established that a process language can have a precise meaning -- a meaning that a machine can act on.
3. **The closing of the loop.** Lean, Six Sigma, and Process Mining all established that execution should feed back into process design. The process model is not static, it evolves based on what the runtime reveals.

Key Insight:

> "The industrial lineage gave us the workflow. From Taylor's stopwatch through Ford's assembly line to BPMN's formal execution semantics, the discipline has been the same: **rationalize the work, specify it formally, and let a machine execute it.** Stage 7's Φ-Execution layer is the semantic counterpart to this discipline."

26.6.3 The Convergence at Stage 7

The two lineages converge at **Stage 7**.

The AI lineage supplies the **semantic conditions** -- the logical axioms that determine when a rule fires. The industrial lineage supplies the **process orchestration** -- the sequence of actions that follow once the rules fires.

Together, they form the two halves of the Executable Knowledge Architecture:

- Θ **(Trigger)** -- established by the AI lineage. The ontology defines *what to fire on*: `InterestingPizza` is a semantic condition, derived from the boundary and value layers, that fires when a pizza has at least three toppings.
- Φ **(Execution)** -- established by the industrial lineage. The workflow defines *what to do*: once a pizza is classified as `InterestingPizza`, the corresponding workflow takes the corresponding action (log it, tag it, escalate it, or produce it).

Neither half is sufficient on its own.

- A trigger without a workflow fires on nothing: the ontology knows the pizza is `InterestingPizza`, but nothing happens.

- A workflow without a trigger fires at the wrong time: the process starts executing the moment a pizza exists, whether or not it satisfies the semantic condition.

The two lineages are not just complementary. They are *mutually necessary*. The AI lineage provides the semantic precision that the industrial lineage needs to fire reliably; the industrial lineage provides the operational execution that the AI lineage cannot provide on its own.

This is what Stage 7 delivers: a single architecture in which the semantic condition and the executable process are two sides of the same specification.

The ontology is the formal model of the semantic conditions; the process model is the formal model of the corresponding actions. Together, they form a system that is *both* semantically precise *and* operationally executable.

Key Insight:

> "Executability is the meeting point of two engineering traditions. The AI tradition gives us the trigger; the industrial tradition gives us the workflow. **Stage 7** is where they became one architecture. Without either half, the other half is incomplete."

26.6.4 What the Historical Perspective Teaches

The historical perspective is not just academic. It teaches three concrete lessons for SKDL's **Stage 7** engineering:

1. Formal semantics are what make execution possible. Every successful era of both lineages -- production systems, expert systems, rule engines in the AI lineage; Taylorism, Lean, Six Sigma, BPMN in the industrial lineage -- was built on formal semantics. The eras that failed -- hand-written ad hoc rules, bureaucratic process documentation -- were eras where the formal semantics were lost.

2. The two lineages are complementary, not competitive. Contemporary conversations sometimes pit AI against process discipline as if one were replacing the other. The historical record shows the opposite: the AI lineage and the industrial lineage are two halves of the same architecture. Neither replaces the other; both are necessary for a system that executes reliably at scale.

3. The failure modes of the past are what Stage 7 addresses. Expert systems were brittle; **Stage 7**'s closure axioms and disjointness declarations address the boundary conditions that expert systems could not express. Bureaucratic process documentation was disconnected from execution; **Stage 7**'s cardinality restrictions and Θ-Trigger semantics ensure that execution is grounded in the process model. The history of the two disciplines is the record of the failure modes that **Stage 7** was designed to address.

Key Insight:

> "History is not memory. It is the record of what failed, and why. **Stage 7**'s primitives are the response to the failure modes of two disciplines that have been solving the same class of problem for over a century."

26.6.5 From Lineage to Industrial Practice

The historical perspective sets up the industrial perspective. If the AI lineage gave us the trigger and the industrial lineage gave us the workflow, the natural question is: *how are these two lineages applied together, in real industrial systems?*

This is the subject of §26.7.

The industrial perspective examines three legs of executable knowledge in practice:

1. **Manufacturing** -- the ISA-95 standard and modern MOM platforms (Rhize) where the reasoner infers classifications and the MES fires actions.
2. **Compliance and Safety** -- where the closed-world obligation of regulation meets the open-world reasoning of OWL through layered constraints (OWL + SHACL).
3. **Process Graphs** -- where BPMN 2.0, Model-Driven Engineering, and Process Mining form the industrial infrastructure that Figay and others have argued is the foundation of executable operations.

Together, these three legs show how **Stage 7**'s anatomy is applied at industrial scale. The historical perspective has shown *why* the two lineages matter; the industrial perspective shows *how* they are put to work.

26.7 Industrial Perspective

The historical perspective (§26.6) established that **Stage 7** is the convergence of two lineages: the AI lineage, which gives us the semantic trigger (Θ), and the industrial lineage, which gives us the executable workflow (Φ). The historical perspective explains *why* these two halves exist. The industrial perspective explains *how* they are put to work.

This section examines three legs of executable knowledge in industrial practice, as mentioned in §26.6.5.

26.7.1 The Manufacturing Leg: ISA-95 and the Reasoning-to-Execution Pipeline

Manufacturing is the domain where the $\Theta \rightarrow \Phi$ loop is most mature, because the industrial lineage (§26.6.2) originated there. The modern manufacturing enterprise is organized around a layered architecture defined by the **ISA-95** standard (IEC 62264), which separates the enterprise level (ERP) from the manufacturing operations level (MES) and the control level (SCADA/PLC).

The critical interface for executable knowledge is the boundary between **Level 3 (Manufacturing Operations Management, MOM)** and **Level 4 (Enterprise Resource Planning, ERP)**. This is where semantic decisions must be made and where the reasoner's output must be grounded in the physical process.

The ISA-95 Hierarchy

ISA-95 defines a functional hierarchy that maps directly onto the anatomy of an executable ontology:

ISA-95	Function	Stage 7 Equivalent
Level 4	Business planning, logistics, ERP	The **value layer** -- what the business *wants* to happen (e.g., "produce 100 units of Product A")
Level 3	Manufacturing operations, MES, quality	The **integrity layer** -- the data completeness and cardinality checks that guarantee the process can execute
Level 2	Monitoring, supervisory control	The **boundary layer** -- the universal restrictions that define what is permitted on the shop floor
Level 1	Sensing and manipulating	The physical execution (Φ)
Level 0	The production process itself	The actual world

In an executable ontology, the **reasoner** operates at the boundary Level 4 and Level 3. It reads the semantic conditions (the boundary and value layers) and infers classifications that the MES (Level 3) uses to fire actions.

The Reasoning-to-Execution Pipeline

The pipeline in a modern MOM platform (such as **Rhize**, which implements the ISA-95 standard as a cloud-native MOM) has five stages. Each stage corresponds to one of the five primitives of **Stage 7**:

Stage	Explanation
Stage 1: The Boundary (Universal Restrictions `only`)	The ontology declares what is permitted on the shop floor. For example, a `ProductionOrder` may only be assigned to a `Machine` that has the required `Capability`. This is a universal restriction: `ProductionOrder SubClassOf: assignedTo only (Machine and (hasCapability some RequiredCapability))`.
Stage 2: The Value (Enumeration and `hasValue`)	The ontology closes the set of possible values for critical properties. For example, a `QualityGrade` is enumerated as `{A, B, C, Scrap}`. A `Batch` has a `hasQualityGrade` value from that set. This is the value layer: it tells the system *what* value to act on.
Stage 3: The Integrity	The ontology guarantees that the data required for execution is present. For example, a `Batch` must have at least three `QualityCheck` records before it can be released. This is the cardinality restriction:

Stage	Explanation
(Cardinality)	`Batch SubClassOf: hasQualityCheck min 3 QualityCheck`. Without this, the MES might release a batch with incomplete quality data -- a quiet failure that the integrity layer prevents.
Stage 4: The Trigger (Θ)	The reasoner synchronizes. When a `Batch` satisfies the boundary, value, and integrity conditions, it is classified as a `ReleasableBatch`. This is the Θ layer: the semantic condition that fires the trigger.
Stage 5: The Execution (Φ)	The MES reads the inferred classification (`ReleasableBatch`) and fires the corresponding action: it releases the batch to the next production stage, prints the certificate of analysis, and updates the ERP. This is the Φ layer: the operational execution that the industrial lineage provides.

Why the Pipeline Fails Without Stage 7?

Without the integrity layer, the pipeline has a quiet failure mode. The MES will release a batch if the reasoner classifies it as `ReleasableBatch`. But under the Open World Assumption, the reasoner will *not* classify a batch as `ReleasableBatch` if the quality checks are missing -- it will simply *not fire the rule*. The MES will then wait indefinitely, or the batch will be stuck in a "pending" state. The integrity layer (the cardinality restriction) makes the count provable, so the reasoner can fire the trigger deterministically.

Key Insight:

> "In manufacturing, the quiet failure mode of OWL is expensive. A batch that is not released because the reasoner cannot prove the count is a line stoppage. The cardinality restriction is not an academic exercise -- it is the difference between a working MES and a silent factory."

26.7.2 The Compliance Leg: Closed-World Regulation vs. Open-World Reasoning

The second leg is compliance and safety. This is where the industrial perspective meets a fundamental tension: regulations are closed-world obligations, but OWL is an open-world language.

A regulation says: "Every batch must have a quality check before release." This is a *closed-world* statement: if the quality check is not present, the batch is non-compliant. OWL, under the Open-World Assumption, cannot make this statement directly. It can only say: "If a batch has a quality check, it is a `CheckedBatch`." It cannot say: "If a batch does *not* have a quality check, it is non-compliant."

This tension is resolved through layered constraints: OWL for the semantic reasoning, and SHACL (Shapes Constraint Language) for the closed-world validation.

The OWL + SHACL Layering

Layer	Language	World Assumption	Role
Semantic	OWL 2 DL	Open World	Infers classifications, derives new knowledge, handles the Θ trigger
Validation	SHACL	Closed World	Validates that the data satisfies the regulation's obligations
Execution	BPMN / MES	Closed World	Fires the action, blocks the release, triggers the exception

In practice, the OWL ontology defines the *semantic conditions* for compliance.

For example, a `CompliantBatch` is defined as a `Batch` that has at least one `QualityCheck` with a `hasResult` value of `Pass`. The reasoner infers `CompliantBatch` when the conditions are met.

SHACL then validates the *closed-world obligation*. A SHACL shape declares: "Every `Batch` must have at least one `QualityCheck`." If the batch does not have one, the SHACL validator fires a violation. The MES reads the violation and blocks the release.

This layering is the industrial answer to the open-world/closed-world tension. OWL provides the semantic precision that the regulation needs (the Θ trigger). SHACL provides the closed-world enforcement that the regulation demands (the Φ execution). Neither is sufficient alone.

The Compliance Pipeline in Practice

1. **OWL Reasoner**: Infers `CompliantBatch` for batches that satisfy the semantic conditions (boundary + value + integrity).
2. **SHACL Validator**: Checks the closed-world obligations (e.g., "every batch must have a quality check"). Fires violations for non-compliant batches.

3. **MES / Workflow Engine**: Reads the inferred classification and the SHACL violations. Releases compliant batches; blocks non-compliant batches and triggers an exception workflow.

This is the same $\Theta \rightarrow \Phi$ loop as in manufacturing, but with an additional validation layer that handles the closed-world obligation of regulation.

Key Insight:

> "Regulation is closed-world. OWL is open-world. The industrial answer is not to choose one over the other -- it is to layer them. OWL infers the semantic condition; SHACL enforces the closed-world obligation; the workflow engine executes the action. This is the three-layer anatomy applied to compliance."

26.7.3 The Process Graph Leg: BPMN, DMN, and the Separation of Concerns

The third leg is the process graph. This is where the industrial lineage's formal process modeling (§26.6.2) meets the semantic layer. The process graph is the executable specification of the workflow that follows the trigger.

In the previous editions of this chapter, this leg was described as **BPMN + MDE**. However, the modern industrial standard for executable knowledge is actually a **"triple crown"** of complementary OMG specifications: **BPMN** (process), **DMN** (decision), and **CMMN** (case). For **Stage 7**, the critical relationship is between BPMN and DMN.

BPMN 2.0 as the Execution Specification

BPMN 2.0 is the industrial standard for process modeling with formal execution semantics. Every BPMN construct -- sequence flow, boundary event, parallel gateway, exclusive gateway -- has a defined mapping to an execution semantics. A BPMN-compliant workflow engine can read a BPMN 2.0 model and execute it without additional interpretation.

In the **Stage 7** architecture, the BPMN model is the Φ layer. It defines *what to do* once the reasoner has fired the trigger.

For example:

- **Trigger**: The reasoner classifies a `Batch` as `ReleasableBatch`.
- **BPMN Process**: The `BatchReleaseProcess` starts. It has a sequence flow: `CheckQuality` $\rightarrow$ `GenerateCertificate` $\rightarrow$ `UpdateERP` $\rightarrow$ `ReleaseBatch`.
- **Execution**: The workflow engine executes the BPMN process, calling the MES APIs at each step.

The BPMN model is not a post-hoc illustration. It is the executable specification that the workflow engine runs.

DMN: The Missing Link for Business Rules

While BPMN handles the *flow* of work over time (stateful, concurrency, timers), it is poorly suited for modeling complex, multi-criteria business rules. Encoding a large decision table (matrix) into a series of nested BPMN exclusive gateways creates brittle, unmaintainable models. [2][3]

This is where **DMN (Decision Model and Notation)** enters the **Stage 7** architecture. DMN is an OMG standard designed to model *decisions at a point in time* (stateless, pure expression evaluation). It provides a graphical Decision Requirements Diagram (DRD) and a standardized decision table format (using the FEEL expression language).[4][5]

The integration point is the **BPMN Business Rule Task**. Instead of hard-coding the decision logic into the process flow, the BPMN process contains a Business Rule Task that *calls* a DMN decision service. The DMN engine evaluates the decision table and returns the result to the BPMN process.[6]

How BPMN and DMN Work Together in Stage 7?

Aspect	BPMN	DMN
Domain	Process flow (over time)	Decision logic (at a point)
State	Stateful (tokens, variables)	Stateless (input $\rightarrow$ output)
Stage 7 Role	The Φ **layer** (execution orchestration)	The Θ **evaluation engine** for complex rules

Aspect	BPMN	DMN
Integration	Contains a Business Rule Task	Called by the Business Rule Task

The reasoner (OWL) still handles the *semantic trigger (Θ)* -- it decides *if* a batch is `ReleasableBatch`. But once the trigger fires, the BPMN process starts. If that process needs to determine a *discount percentage* or a *shipping priority* based on multiple criteria, it delegates to a DMN decision table. The DMN engine evaluates the rules and returns the result, which the BPMN process then uses.

Model-Driven Engineering (MDE)

Model-Driven Engineering (MDE) extends this principle: the formal models (BPMN and DMN) are the *primary engineering artifacts*.

In MDE, the BPMN model is linked to

- The **ontology** (for semantic validation: the BPMN process must be consistent with the OWL axioms),
- The **DMN models** (for decision logic: the BPMN process uses decision tables from the DMN models),
- The **data schemas** (for structure: the BPMN process must use the data types defined in the ontology), and
- The **system architecture** (for deployment: the BPMN process is deployed to the workflow engine).

The traceability is preserved from the business goal (the OWL ontology) to the ground-level execution (the BPMN + DMN runtime).

This is the industrial counterpart to the semantic-web stack: SKDL treats the ontology as the primary artifact and the reasoner as its executor; MDE treats the process/decision models as the primary artifacts and the workflow/decision engine as its executors.

Process Mining: The Feedback Loop

The final piece of the process graph leg is **Process Mining**. Process mining reads the event logs from the workflow engine and reconstructs the actual process that was executed. It compares the actual process to the BPMN model and identifies deviations.

This closes the loop that the industrial lineage established (§26.6.2): **execution feeds back into process design**. The process model is not static; it evolves based on what the runtime reveals. In the **Stage 7** architecture, this feedback loop is the mechanism by which the ontology, the BPMN process, and the DMN decision logic are continuously improved.

Key Insight:

> "The process graph is the bridge between the semantic trigger and the physical execution. BPMN provides the flow, DMN provides the decision logic, and MDE provides the traceability from business goal to deployment. Process mining provides the feedback loop that closes the cycle. Together, they form the industrial infrastructure that makes **Stage 7**'s Φ layer executable at scale."

26.7.4 The Three Legs Together

The three legs of the industrial perspective are not separate. They are three views of the same $\Theta \rightarrow \Phi$ loop:

Leg	What it provides	Stage 7 Layer
Manufacturing (ISA-95)	The reasoning-to-execution pipeline; the MES fires actions based on inferred classifications	$\Theta \rightarrow \Phi$
Compliance (OWL + SHACL)	The closed-world validation that regulation requires; the SHACL validator fires violations	$\Theta \rightarrow \text{Validation} \rightarrow \Phi$
Process Graph (BPMN + DMN + MDE)	The executable specification of the workflow (BPMN) and the business rules (DMN); the workflow engine executes the process	Φ (with DMN as the Θ evaluation engine)

Together, they show how **Stage 7**'s anatomy is applied at industrial scale.

1. The manufacturing leg shows the pipeline;
2. the compliance leg shows the validation layer; and
3. the process graph leg shows the execution specification (BPMN) and the decision evaluation (DMN).

The historical perspective (§26.6) has shown *why* the two lineages matter. The industrial perspective shows *how* they are put to work.

Key Insight:

> "The industrial perspective is the proof that **Stage 7** is not an academic exercise. The ISA-95 pipeline, the OWL+SHACL compliance stack, and the BPMN+DMN process graph are the three legs on which executable knowledge stands in the real world. Each leg is a different application of the same $\Theta \rightarrow \Phi$ loop."

26.8 EKA Perspective -- Stage 7 Establishes Θ + Φ

We have now examined **Stage 7** from three complementary perspectives: the **mathematical** (§26.5) explained *why* the five primitives work; the **historical** (§26.6) explained *where* they came from; and the **industrial** (§26.7) explained *how* they are applied. This section delivers the fourth and final perspective: the **EKA perspective** -- how the five primitives establish the **Θ (Trigger)** and **Φ (Execution)** layers of the Executable Knowledge Architecture.

The EKA framework was introduced in earlier chapters. In this section, we close the loop: we show that **Stage 7** is not merely a collection of OWL constructs, but the architectural layer that connects semantic reasoning to operational action.

26.8.1 The Two Halves of the Executable Knowledge Architecture

The historical perspective (§26.6.3) established that executability is the convergence of two lineages: the AI lineage, which gives us the semantic trigger (Θ), and the industrial lineage, which gives us the executable workflow (Φ). The EKA perspective formalizes this convergence.

Θ (Trigger) -- The Semantic Condition

The Θ layer is the set of semantic conditions that fire a trigger. It is built from four of the give **Stage 7** primitives:

- **Universal restrictions** (§26.3.4, **Exercise 22**): what values are permitted.
- **Closure axioms** (§26.3.4, **Exercise 23**): what values exhaustively exist.
- **Enumerated classes** (§26.5.3, **Exercise 24**): what value domain is bounded.
- `hasValue` **restrictions** (§26.5.4, **Exercise 25**): what specific value fires.

The Θ layer answers the question: *"Under what semantic conditions should an action be taken?"* It is the "if" part of the execution rule. When a reasoner classifies an individual as a member of a class defined by these primitives, the Θ layer has fired.

Φ (Execution) -- The Operational Mechanism

The Φ layer is the mechanism that, once the trigger has fired, executes the corresponding action. It is built from the fifth primitive:

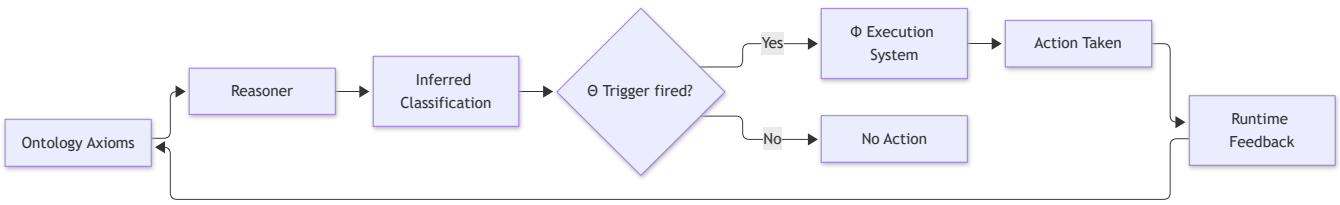
- **Cardinality restrictions** (§26.5.5, **Exercise 26**): how many values are required before execution is safe.

The Φ layer answers the question: *"What action should be taken, and when is the data sufficient to take it?"* It is the "then" part of the execution rule. The cardinality restriction is the semantic construct that guarantees the data required for execution is complete.

Beyond the ontology itself, Φ is the runtime system -- the MES, the workflow engine, the rule engine -- that reads the classified instance and takes the corresponding action.

The $\Theta \rightarrow \Phi$ Loop

Together, Θ and Φ form a closed loop:



The ontology supplies the semantic conditions (Θ); the runtime supplies the action (Φ). The loop closes when runtime feedback (process mining, event logs) feeds back into the ontology, refining the axioms for the next iteration.

Key Insight:

> "The Θ layer is the semantic side of the trigger. The Φ layer is the operational side. Together, they close the loop between reasoning and action.

26.8.2 The Five Primitives in the EKA Framework

The five **Stage 7** primitives map directly onto the Θ and Φ layers. The table below summarizes the mapping:

#	Primitive	EKA Layer	Role in the Architecture
1	**Universal Restriction (`only`)**	Θ	Defines the boundary of the trigger: what values are permitted
2	**Closure Axiom**	Θ	Closes the boundary: what values exhaustively exist
3	**Enumerated Class**	Θ	Closes the value domain: the finite set of possible trigger values
4	**`hasValue` Restriction**	Θ	Binds the value: the specific value that fires the trigger
5	**Cardinality Restriction**	Φ	Enforces completeness: how many values are required before execution is safe

The first four primitives build the **semantic trigger**. The fifth primitive builds the **operational guarantee**. Without the first four, there is no trigger. Without the fifty, the trigger may fire on incomplete data.

The Θ Layer: What Fires?

The Θ layer is not a single axiom. It is the *conjunction* of boundary and value axioms that together define a class whose instances fire a trigger. Consider the `SpicyPizza` definition from **Chapter 25**:

```
Class: SpicyPizza
    EquivalentTo: Pizza and (hasTopping some (hasSpiciness value Hot))
```

This definition uses two primitives:

- The **enumerated class** `Spiciness = {Hot, Medium, Mild}` (established in **Exercise 24**) closes the value domain.
- The **`hasValue` restriction** `hasSpiciness value Hot` (established in **Exercise 25**) binds the specific value.

Together, they define the Θ condition: *"A pizza is a `SpicyPizza` if and only if it has at least one topping whose spiciness is `Hot`."* When the reasoner classifies a pizza as a `SpicyPizza`, the Θ layer has fired.

The Φ Layer: What Executes?

The Φ layer is the mechanism that, once the trigger has fired, executes the corresponding action. In the ontology, it appears as the **cardinality restriction**. Consider the `InterestingPizza` definition from **Chapter 26**:

```
Class: InterestingPizza
    EquivalentTo: Pizza and (hasTopping min 3 PizzaTopping)
```

This definition uses the cardinality primitive:

- The **cardinality restriction** `hasTopping min 3 PizzaTopping` (established in **Exercise 26**) guarantees that the required number of toppings is present.

The cardinality restriction is what *binds* the trigger to the execution. It says: *"This condition fires only when the data is complete enough to act on."* Without it, the trigger might fire on incomplete data -- a pizza with three toppings asserted but not proven distinct.

Key Insight:

> "The boundary and value layers are the semantic side of the trigger. The integrity layer is the operational side. Together, they establish the $\Theta - Trigger$ and $\Phi - Execution$ layers that close the loop between reasoning and action."

26.8.3 The Anatomy Revisited: EKA in Action

In §26.4, we described the anatomy of an executable ontology as three layers: **boundary**, **value**, and **integrity**. The EKA perspective reveals that this three-layer anatomy *is* the $\Theta \rightarrow \Phi$ architecture.

Layer 1 -- Boundary (Chapter 24): The Semantic Foundation

The boundary layer (universal restrictions + closure axioms) establishes the *domain* of the trigger. It answers: *"What is the world in which the trigger operates?"*

Without a closed boundary, the trigger has no well-defined scope. The boundary layer is the first half of the Θ condition.

Layer 2 -- Value (Chapter 25): The Specific Trigger

The value layer (enumerated classes + `hasValue` restrictions) establishes the *specificity* of the trigger. It answers: *"What specific value fires the trigger?"*

Without a closed value domain, the trigger cannot be exhaustive. Without a `hasValue` binding, the trigger cannot be specific. The value layer is the second half of the Θ condition.

Layer 3 -- Integrity (Chapter 26): The Operational Guarantee

The integrity layer (cardinality restrictions) establishes the *completeness* of the data required for execution. It answers: *"How much data do I need before I can act?"*

Without cardinality, the trigger may fire on incomplete data. The integrity layer is the Φ guarantee.

The table below maps the three layers to the EKA framework:

Anatomy Layer	EKA Layer	Question Answered	Primitives
Boundary	Θ	What is the world?	Universal restriction, closure axiom
Value	Θ	What value fires?	Enumerated class, `hasValue`
Integrity	Φ	How much data is required?	Cardinality restriction

The key observation is that the boundary and value layer both belong to Θ. They are not separate architectural layers; they are two halves of the same semantic condition. The integrity layer belongs to Φ; it is the operational guarantee that the trigger is ready to execute.

Key Insight:

> "The anatomy of an executable ontology is not three unrelated layers. It is two Θ layers and one Φ layer -- the semantic condition and its operational guarantee."

26.8.4 The EKA Perspective Across the Book

The EKA framework is not unique to **Stage 7**. It is the architectural thread that runs through the entire SKDL lifecycle. **Stage 7** is the stage where the Θ and Φ layers are *established* -- where the ontology becomes actually executable. But the layers themselves were prepared by earlier stages.

How Earlier SKDL Stages Prepare Θ and Φ?

SKDL Stage	Contribution to Θ	Contribution to Φ
Stage 1: Conceptual Modeling	Defined the classes and properties that the trigger operates on	Defined the taxonomy that makes distinctness provable (essential for cardinality restriction)
Stage 2: Semantic Description	Created the natural language definitions of the trigger conditions	Documented the execution rules
Stage 3: Knowledge Reuse	Reused established design patterns (closure, enumeration, `hasValue`)	Reused established execution patterns (cardinality, state machines)

SKDL Stage	Contribution to Θ	Contribution to Φ
Stage 4: Semantic Governance	Ensured the trigger conditions meet organizational standards	Ensured the execution rules are auditable
Stage 5: Semantic Validation	Verified that the trigger conditions are logically consistent	Verified that the execution data is complete
Stage 6: Semantic Reasoning	Ran the reasoner to derive classifications	Verified that the inferred classifications match the intended execution rules
Stage 7: Executable Knowledge	**Established the Θ layer**	**Established the Φ layer**

This table reveals a critical insight: **Stage 7 does not create the Θ and Φ layers from nothing**. It *establishes* them by adding the final execution primitives (enumerated classes, `hasValue`, cardinality) to the foundation built by earlier stages.

If Stage 1 has a modeling error (e.g., `PepperoniTopping` under `VegetableTopping` instead of `MeatTopping`), the Θ layer will fire on the wrong conditions. If Stage 1 lacks disjointness declarations, the Φ layer cannot prove disjointness and cardinality will fail. The EKA perspective makes this dependency explicit: **the Θ and Φ layers are only as reliable as the stages that prepared them**.

Key Insight:

> "Stage 7 establishes Θ and Φ -- but it does not build them from scratch. It inherits the semantic foundation from Stage 1-6. An error at Stage 1 propagates to Θ and Φ."

The EKA Architecture in Industrial Practice

The industrial perspective (§26.7) showed how Θ and Φ are applied in three domains: manufacturing (ISA-95), compliance (OWL + SHACL), and process graphs (BPMN + DMN + MDE). The EKA perspective reveals that all three are instances of the same architecture.

Manufacturing (ISA-95)

In manufacturing, the Θ layer is the reasoner's classification of a `Batch` as a `ReleasableBatch`. The Φ layer is the MES' action: releasing the batch, printing the certificate, updating the ERP. The cardinality restriction (`hasQualityCheck min 3 QualityCheck`) guarantees that the batch has enough quality data for the Φ layer to execute safety.

Compliance (OWL + SHACL)

In compliance, the Θ layer is the OWL reasoner's classification of a `Batch` as a `CompliantBatch`. The Φ layer is the SHACL validator's check: does the batch satisfy the closed-world obligation? If not, the workflow engine blocks the release and triggers an exception. The EKA perspective reveals that OWL and SHACL are not competing formalisms; they are two halves of the same $\Theta \rightarrow \Phi$ loop.

Process Graphs (BPMN + DMN + MDE)

In process graphs, the Θ layer is the OWL reasoner's classification of a `Batch` as a `ReleasableBatch`. The Φ layer is the BPMN process: $CheckQuality \rightarrow GenerateCertificate \rightarrow UpdateERP \rightarrow ReleaseBatch$. The DMN decision tables provide the decision logic for the Φ layer: *"What discount percentage applies? What shipping priority?"* The EKA perspective reveals that BPMN, DMN, and MDE are all Φ-layer mechanisms; they orchestrate the execution that Θ triggers.

The table below summarizes the EKA mapping across the three industrial legs:

Industrial Leg	Θ Layer	Φ Layer	Cardinality Guarantee
**Manufacturing (ISA-95)	Reasoner classifies `ReleasableBatch`	MES releases batch	`hasQualityCheck min 3 QualityCheck`
Compliance (OWL + SHACL)	Reasoner classifies `CompliantBatch`	SHACL validator checks obligation	SHACL shape: `minCount 1` on `QualityCheck`
Process Graph (BPMN + DMN)	Reasoner classifies `ReleasableBatch`	BPMN process + DMN decision table	BPMN gateway checks for require data

Each leg is a different application of the same $\Theta \rightarrow \Phi$ architecture. The EKA perspective reveals the common structure beneath the domain-specific details.

Key Insight:

> "Manufacturing, compliance, and process graphs are not three separate domains,. They are three instances of the same $\Theta \rightarrow \Phi$ architecture. The EKA framework is the common language that connects them."

26.8.6 What the EKA Perspective Teaches

The EKA perspective is not just a restatement of the anatomy. It teaches three concrete lessons for SKDL's **Stage 7** engineering:

1. Θ and Φ are two halves of the same architecture.

The AI lineage (trigger) and the industrial lineage (workflow) are not separate concerns. They are Θ and Φ of the same executable knowledge architecture. An ontology that provides Θ but not Φ is a library, not a system. An ontology that provides Φ but not Θ is a workflow engine without semantic conditions. Both are incomplete. **Stage 7** delivers both.

2. The $\Theta \rightarrow \Phi$ loop is closed by runtime feedback.

The EKA framework is not a one-way pipeline. It is a closed loop. The execution system (Φ) generates data (even logs, process mining) that feeds back into the ontology (Θ), refining the semantic conditions for the next iteration. This is the industrial lineage's insight: execution should feed back into process design. The EKA perspective formalizes this feedback loop as part of the architecture.

3. The reliability of Θ and Φ depends on the earlier stages.

The Θ layer is only as reliable as the taxonomy as **Stage 1** defines. The Φ layer is only as reliable as the disjointness declarations that **Stage 1** provides. The EKA perspective makes this dependency explicit: **Stage 7 establishes Θ and Φ, but it does not build them from nothing.** It inherits the semantic foundations from **Stage 1-6**. An error at **Stage 1** propagates to Θ and Φ. This is why the SKDL is a lifecycle, not a checklist.

Key Insight:

> "The EKA perspective is the architectural synthesis of **Stage 7**. It shows that the five primitives are not a checklist of OWL constructs -- they are the Θ and Φ layers of an executable knowledge architecture. The boundary and value layers build Θ. The integrity layer builds Φ. Together, they close the loop between reasoning and action."

26.8.7 Looking Ahead -- From EKA to SKDL Synthesis

The EKA perspective closes the fourth and final perspective of **Chapter 26**. We have now examined **Stage 7** from four complementary angles:

1. **Mathematical (§26.5)** -- the logical foundations of closure, enumeration, binding, and cardinality.
2. **Historical (§26.6)** -- how executability evolved from expert systems to modern rule engines.
3. **Industrial (§26.7)** -- how executable ontologies are applied in manufacturing, compliance, and process automation.
4. **EKA (§26.8)** -- how the five primitives establish the Θ (Trigger) and Φ (Execution) layers of the Executable Knowledge Architecture.

Together, these four perspectives give a complete picture of **Stage 7** -- not just what the primitives are, but why they matter.

The next section, **§26.9 Engineering Guidelines for Executable Knowledge**, distills these perspectives into a set of concrete guidelines for engineers building executable ontologies. Then **§26.10 Key Concepts** and **§26.11 Chapter Summary** consolidate the chapter, and **§26.12 Looking Ahead** prepares the synthesis that follows in **Chapter 27**.

The arc of **Stage 7** is nearly complete. From the boundary layer (**Chapter 24**) through the value layer (**Chapter 25**) to the integrity layer (**Chapter 26, this chapter**), we have built an executable ontology from the ground up. The EKA perspective reveals that this is not just a technical construction -- it is an architecture that connects semantic reasoning to operational action. That architecture is the foundation of the SKDL synthesis that follows.

Key Insight:

> "The EKA perspective is the final perspective of **Stage 7**. It shows that the five primitives are not just OWL constructs -- they are the Θ and Φ layers of an executable knowledge architecture. **Stage 7** establishes these layers. **Chapter 27** will synthesize them."

26.9 Engineering Guidelines for Executable Knowledge

26.10 Key Concepts

26.11 Chapter Summary

26.12 Looking Ahead -- Toward SKDL Synthesis

Last updated at 2026-11-04

Reference

---

1. Dr. Nicolas Figay, *Rethinking Business Process Modeling: Rediscovering the Industrial Essence of BPMN for Work Rationalization*, https://medium.com/@nfigay/rethinking-business-process-modeling-rediscovering-the-industrial-essence-of-bpmn-for-work-aae3f1ccb101 ↩
2. Ekaterina Bazhenova, Francesca Zerbato, Barbara Oliboni, Mathias Weske, *From BPMN process models to DMN decision models*, https://www.sciencedirect.com/science/article/abs/pii/S0306437918300231 ↩
3. Rickard Bízik, *BPMN vs DMN: Processes Over Time, Decisions at a Point*, https://quantumbpm.com/blog/bpmn-vs-dmn ↩
4. Faruk Hasić, Johannes De Smedt, Jan Vanthienen, *Redesigning Processes for Decision-Awareness: Strategies for Integrated Modelling*, https://ieeexplore.ieee.org/document/8590196 ↩
5. *What is FEEL?*, https://docs.camunda.io/docs/components/modeler/feel/what-is-feel/ ↩
6. Thierry Biard, Alexandre Le Mauff, Michel Bigand & Jean-Pierre Bourey, *Separation of Decision Modeling from Business Process Modeling Using New "Decision Model and Notation" (DMN) for Automating Operational Decision-Making*, https://link.springer.com/chapter/10.1007/978-3-319-24141-8_45 ↩